

Market_Sentiment_Predictor_Pipeline

April 10, 2026

1 Market Sentiment Predictor

Geovanni Jones, Josh Previte, Casey Barrasso

The goal of this project is to predict the directional price movement of a specific stock using both historical price data and news information. Two separate models will predict price movements based on historical data and analyze news sentiment.

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import re
import warnings
import torch
import gc

#from pandas.errors import SettingWithCopyWarning

from collections import Counter

from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.seasonal import STL

#!pip install imblearn
from imblearn.over_sampling import SMOTE
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression as LR
from sklearn.svm import SVC
from xgboost import XGBClassifier
from sklearn.model_selection import StratifiedKFold, GridSearchCV
from sklearn.utils import resample
```

```

from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from sklearn.metrics import make_scorer, accuracy_score, f1_score

from scipy.sparse import hstack, csr_matrix

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.metrics import classification_report as CR
#from sklearn.pipeline import Pipeline
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE

from sklearn.metrics import accuracy_score, f1_score

#install transformers if not already done
#!pip install transformers
from transformers import AutoTokenizer, AutoModelForSequenceClassification

import warnings
warnings.filterwarnings("ignore")

```

```
[2]: os.getcwd()
```

```
[2]: '/home/jupyter-barrasc/market-sentiment-predictor/notebooks'
```

```
[3]: #suppress pesky filter warnings
#warnings.filterwarnings("ignore", category=SettingWithCopyWarning)
```

2 Week 2 - Data Ingestion

There are two main datasets to this project. The first is a dataset containing news article content including titles, content, and sentiment scores from various news sources around the AAPL stock. The second dataset (introduced later in the notebook) contains a time series of AAPL stock prices. The final dataset will combine the two datasets on date.

2.1 Sentiment Dataset

```
[4]: #Github didn't allow us to upload a file more than 25MB. This is combining all
      ↳ files into one file
def sentiment_data_creation():
    raw_data_1 = pd.read_csv('../data/apple_news_data_1.csv', encoding='latin-1')
    raw_data_2 = pd.read_csv('../data/apple_news_data_2.csv', encoding='latin-1')
    raw_data_3 = pd.read_csv('../data/apple_news_data_3.csv', encoding='latin-1')
    raw_data_4 = pd.read_csv('../data/apple_news_data_4.csv', encoding='latin-1')
    raw_data_5 = pd.read_csv('../data/apple_news_data_5.csv', encoding='latin-1')
```

```

    raw_data = pd.concat([raw_data_1, raw_data_2, raw_data_3, raw_data_4,
→raw_data_5])[['date', 'title', 'content', 'link', 'symbols', 'tags',
→'sentiment_polarity', 'sentiment_neg', 'sentiment_neu', 'sentiment_pos']]
    return raw_data

raw_data = sentiment_data_creation()

```

```

[5]: #view a small subsection of the data to ensure formatting looks correct
raw_data.head(10)

```

```

[5]:
      date \
0  2021-04-30T21:38:00+00:00
1  2021-04-30T20:41:52+00:00
2  2021-04-30T20:18:00+00:00
3  2021-04-30T19:32:30+00:00
4  2021-04-30T18:49:00+00:00
5  2021-04-30T18:30:06+00:00
6  2021-04-30T18:19:10+00:00
7  2021-04-30T18:13:06+00:00
8  2021-04-30T17:54:31+00:00
9  2021-04-30T17:34:59+00:00

      title \
0  Techas Big Five Had Fantastic Pandemics. Her...
1  Valuation Remains a Concern for Churchill Capi...
2  Big techâs trillion-dollar pandemic year may...
3  What Appleâs blowout earnings means for Berk...
4  EU Agrees With Spotify: Apple Is Abusing Its A...
5  ETFs to Stack as Reopening Poses No Hindrance ...
6  Big Tech Stocks Flash Red Flag to Regulators A...
7  Top Stock Reports for Apple, Alphabet & Bl...
8  Why Stem Stock Looks Very Attractive Following...
9  âTechnology is the largest single growth tre...

      content \
0  Alphabet, Amazon.com, Apple, Facebook, and Mic...
1  Churchill Capital IV (NYSE:CCIV) stock has gra...
2  This week concludes the first quarter earnings...
3  Krish Sankar, Managing Dir. & Cowen Senior Res...
4  The EU's European Commission just released pre...
5  The tech-heavyÂ Nasdaq Composite Index was a s...
6  If â Big Techâ was a gamefish, like Faceboo...
7  Friday, April 30, 2021\n\nThe Zacks Research D...
8  Not infrequently, Wall Street focuses excessiv...
9  Keith Fitz-Gerald, Fitz-Gerald Group Chief Inv...

      link \

```

```

0 https://finance.yahoo.com/m/bbe3f7c9-c7af-3c9f...
1 https://finance.yahoo.com/news/valuation-remai...
2 https://finance.yahoo.com/m/6e441d74-37c1-3642...
3 https://finance.yahoo.com/video/apple-blowout-...
4 https://finance.yahoo.com/m/07524444-63a7-34aa...
5 https://finance.yahoo.com/news/etfs-stack-reop...
6 https://finance.yahoo.com/news/big-tech-stocks...
7 https://finance.yahoo.com/news/top-stock-repor...
8 https://finance.yahoo.com/news/why-stem-stock-...
9 https://finance.yahoo.com/video/technology-lar...

```

```

                                symbols \
0 AAPL.US, AAPL34.SA, AMZN.US, FB.US, GOOGL.US
1 AAPL.US, CCIV-UN.US, CCIV.US
2 AAPL.US, AAPL34.SA, AMZN.US, MSFT.US
3 AAPL.US, AAPL34.SA, BRK-A.US, BRK-B.US, DJI.IN...
4 AAPL.US, AAPL34.SA, SPOT.US
5 AAPL.US, AAPL34.SA, COMP.US, FB.US, GOOGL.US, ...
6 AAPL.US, CRM.US, FB.US, GME.US, GOOGL.US, MSFT...
7 AAPL.US, AAPL34.SA, BLK.US, F.US, GOOGL.US, ZT...
8 AAPL.US, AMZN.US, BX.US, FSLR.US, GE.US, GEOO3...
9 AAPL.US, AAPL34.SA, AMZN.US, DJI.INDX, FB.US, ...

```

```

                                tags sentiment_polarity \
0 AMAZON, APPLE, FACEBOOK, HARLEY FINKELSTEIN, M... 0.395
1 BUSINESS COMBINATION, CCIV, CHURCHILL CAPITAL ... 0.998
2 AMAZON, APPLE INC, BIG TECH, MARKET CAPITALIZA... 0
3 ALEXIS CHRISTOFOROUS, BERKSHIRE HATHAWAY, COWE... 0
4 APP STORE, APPLE, EUROPEAN COMMISSION, MUSIC S... -0.71
5 APPLE, CONSENSUS ESTIMATE, EARNINGS PER SHARE,... 0.999
6 BITCOIN, DIGITAL TRANSFORMATION, FACEBOOK, MIC... 0.999
7 ALPHABET, APPLE, APPLE WATCH, BLACKROCK, RESEA... 0.999
8 CLEAN-ENERGY, STEM STOCK 0.999
9 ALEXIS CHRISTOFOROUS, KEITH FITZ-GERALD, SIBIL... 0

```

```

sentiment_neg sentiment_neu sentiment_pos
0 0 0.871 0.129
1 0.023 0.85 0.127
2 0 1 0
3 0 1 0
4 0.093 0.885 0.022
5 0.027 0.826 0.148
6 0.026 0.861 0.113
7 0.042 0.775 0.183
8 0.015 0.83 0.154
9 0 1 0

```

```
[6]: # Sort raw_data by date column from earliest to latest
raw_data = raw_data.sort_values('date', ascending=True).reset_index(drop=True)

raw_data.head(10)
#raw_data.tail(10)
```

```
[6]:
```

	date \		title \		content \		link \
0	And I can tell you now that team presents to ...		our Board and us as a company.		NaN		NaN
1	And but their objective was in a very lower f...		they had an initial spend goal which were sig...		and the results were so great that they were ...		you know
2	And by midyear		we're supposed to be fully ramped		then we'll reevaluate. We'll just see where w...		the expectation is we have what we need.
3	And like I said		the only other call-out I would give is that ...		NaN		NaN
4	And part of that will be growth of e-commerce...		it's a really high value demand and helps us ...		NaN		
5	And so we can get those leading indicators an...		to the extent that we've seen international c...		we're going to look at that and invest in tha...		
6	And so we're just making sure that we're doin...		very closely. So we're on point.		NaN		
7	And so when those renewals come up		we're certainly showing the advantage of not ...		but then an Apple-led and Apple-focused ecosy...		
8	And then I guess to wrap it up would be secur...		every device can have one device management p...		that gives us a good opportunity and really a...		
9	And then also		I also wanted to ask about retention rates		which you mentioned benefit from training you...		

```

4
5
6
7 is resonating with our customer base. So we c...
8
9

```

```

                                symbols                                tags \
0                                NaN                                NaN
1 doubling came from someplace else but I was happy to see it.
2                                NaN                                NaN
3                                NaN                                NaN
4                                NaN                                NaN
5                                NaN                                NaN
6                                NaN                                NaN
7 we were even just as if not even more optimistic about it.
8                                NaN                                NaN
9                                NaN                                NaN

```

```

sentiment_polarity sentiment_neg sentiment_neu sentiment_pos
0 NaN NaN NaN NaN
1 NaN NaN NaN NaN
2 NaN NaN NaN NaN
3 NaN NaN NaN NaN
4 NaN NaN NaN NaN
5 NaN NaN NaN NaN
6 NaN NaN NaN NaN
7 NaN NaN NaN NaN
8 NaN NaN NaN NaN
9 NaN NaN NaN NaN

```

```

[7]: #check how many NA values there are. We care most about sentiment, date, and
      ↪content columns
raw_data.isna().sum().sort_values(ascending=False)

```

```

[7]: tags 17790
sentiment_pos 5853
sentiment_neu 5812
sentiment_neg 5767
sentiment_polarity 5721
symbols 5539
link 5340
content 4901
title 4255
date 2808
dtype: int64

```

```
[8]: #Drop Clumns we won't be focusing on (link and tags)
raw_data = raw_data.drop(columns=['link', 'tags'])
```

```
[9]: # force sentiment scores to be a numerical value, NA if not
for col in
    ↳['sentiment_pos', 'sentiment_neu', 'sentiment_neg', 'sentiment_polarity']:
    raw_data[col] = pd.to_numeric(raw_data[col], errors='coerce')

# extract valid calendar dates only (ignore time + BST noise)
raw_data['date'] = pd.to_datetime(
    raw_data['date'].astype(str).str.extract(r'(\d{4}-\d{2}-\d{2})')[0],
    errors='coerce'
) #This section of the code was inputed to handle warnings from pandas that
↳could lead to future breakage of the code.

# convert content to string
raw_data['content'] = raw_data['content'].astype('str')
```

```
[10]: # Setting "date" column to datetime format
raw_data['date'] = pd.to_datetime(raw_data['date'])

print(raw_data.shape)

#Data range for raw_data
print(f"Date range for raw_data: {raw_data['date'].min()} to {raw_data['date'].
↳max()}")
```

(35604, 8)

Date range for raw_data: 2016-02-19 00:00:00 to 2024-11-27 00:00:00

```
[11]: #this data is messy, drop observations that have NAs in critical columns
raw_data = raw_data.
↳dropna(subset=['sentiment_pos', 'sentiment_neu', 'sentiment_neg', 'sentiment_polarity', 'content'])

#print out the data type of each column to ensure conversions from above worked
for col in raw_data.columns:
    print(f'Column: {col} , Data Type: {raw_data[col].dtype}')
```

```
Column: date , Data Type: datetime64[ns]
Column: title , Data Type: object
Column: content , Data Type: object
Column: symbols , Data Type: object
Column: sentiment_polarity , Data Type: float64
Column: sentiment_neg , Data Type: float64
Column: sentiment_neu , Data Type: float64
Column: sentiment_pos , Data Type: float64
```

The “content” column will be the predictor variable in the first model, and the string will be transformed into a numerical matrix using feature engineering methods such as TF-IDF. The “snetiment

polarity” variable will be the target variable. The goal of the first model will be to predict the sentiment polarity of a news article given its content.

```
[12]: print(raw_data.shape)
```

(29700, 8)

```
[13]: # Dropping rows before January 1, 2018 and after October 31, 2024
raw_data = raw_data[(raw_data['date'] >= '2018-01-01') & (raw_data['date'] <=
    →'2024-10-31')]

print(raw_data.shape)

#Data range for raw_data
print(f"Date range for raw_data: {raw_data['date'].min()} to {raw_data['date'].
    →max()}")
```

(29635, 8)

Date range for raw_data: 2018-01-31 00:00:00 to 2024-10-31 00:00:00

This dataset has 29,635 rows and 9 columns (plus the net sentiment score column created by us)

2.2 Price Timeseries Dataset

```
[14]: ts_data = pd.read_csv('../data/AAPL Stock Data.csv')
```

```
[15]: ts_data.head(10)
```

```
[15]:
```

	Dates	AAPL Share Price
0	Jan-03-2005	1.13
1	Jan-04-2005	1.14
2	Jan-05-2005	1.15
3	Jan-06-2005	1.15
4	Jan-07-2005	1.24
5	Jan-10-2005	1.23
6	Jan-11-2005	1.15
7	Jan-12-2005	1.17
8	Jan-13-2005	1.25
9	Jan-14-2005	1.25

```
[16]: #check how many NA values there are
ts_data.isna().sum().sort_values(ascending=False)
```

```
[16]: Dates          0
AAPL Share Price    0
dtype: int64
```

```
[17]: #Convert values to the expected formatting
ts_data['Dates'] = pd.to_datetime(ts_data['Dates']).dt.date
```



```
ts_data['AAPL Share Price'] = ts_data['AAPL Share Price'].astype('float')
```

```
[18]: for col in ts_data.columns:  
       print(f'Column: {col} , Data Type: {type(ts_data[col][0])}')
```

```
Column: Dates , Data Type: <class 'datetime.date'>
```

```
Column: AAPL Share Price , Data Type: <class 'numpy.float64'>
```

```
[19]: print(ts_data.shape)
```

```
(5298, 2)
```

```
[20]: # Setting "Dates" column to datetime format  
ts_data['Dates'] = pd.to_datetime(ts_data['Dates'])  
  
# Dropping rows before January 1, 2018 and after October 31, 2024  
ts_data = ts_data[(ts_data['Dates'] >= '2018-01-01') & (ts_data['Dates'] <=   
    ↪ '2024-10-31')]  
  
print(ts_data.shape)  
print(f'Date range: {ts_data['Dates'].min()} to {ts_data['Dates'].max()}')
```

```
(1720, 2)
```

```
Date range: 2018-01-02 00:00:00 to 2024-10-31 00:00:00
```

There are 1,720 rows in this dataset and two columns (date and price).

3 Week 3 - Split and EDA

This week, we will focus on splitting the data into a train, validation, and test set and performing exploratory data analysis (EDA).

3.1 Split Sentiment Dataset

```
[21]: raw_data = raw_data.sort_values("date").reset_index(drop=True)  
  
raw_train = raw_data[(raw_data["date"] >= "2018-01-01") & (raw_data["date"] <=   
    ↪ "2022-10-31")]  
raw_val    = raw_data[(raw_data["date"] >= "2022-11-01") & (raw_data["date"] <=   
    ↪ "2023-10-31")]  
raw_test   = raw_data[(raw_data["date"] >= "2023-11-01") & (raw_data["date"] <=   
    ↪ "2024-10-31")]  
  
print(raw_train.shape)  
print(raw_val.shape)  
print(raw_test.shape)  
  
raw_train.head(10)
```

(15474, 8)
(7598, 8)
(6563, 8)

```
[21]:      date                                     title \
0 2018-01-31  Investor Expectations to Drive Momentum within...
1 2018-03-16  Top 100 Reputable Companies Around the Globe A...
2 2018-03-27  Universal Display Corporation Stock Is Way Und...
3 2018-04-16  Detailed Research: Economic Perspectives on Ge...
4 2018-06-22  Apple iPhone Spared Tariffs, But Could Face Ch...
5 2018-10-23  Report: Exploring Fundamental Drivers Behind T...
6 2018-11-30  New Research Coverage Highlights Omeros, Apple...
7 2019-01-03  Apple, AAPL Investment Losses Alert: Bernstein...
8 2019-01-03  SHAREHOLDER ALERT: Bronstein, Gewirtz & Grossm...
9 2019-02-22  Factors of Influence in 2019, Key Indicators a...
```

```
      content \
0 NEW YORK, Jan. 31, 2018 (GLOBE NEWSWIRE) -- ...
1 Boston, MA, March 16, 2018 (GLOBE NEWSWIRE) ...
2 Since topping out in January, shares of Univer...
3 NEW YORK, April 16, 2018 (GLOBE NEWSWIRE) --...
4 In an escalating trade war between China and t...
5 NEW YORK, Oct. 23, 2018 (GLOBE NEWSWIRE) -- ...
6 NEW YORK, Nov. 30, 2018 (GLOBE NEWSWIRE) -- ...
7 NEW YORK, Jan. 02, 2019 (GLOBE NEWSWIRE) -- ...
8 NEW YORK, Jan. 03, 2019 (GLOBE NEWSWIRE) -- ...
9 NEW YORK, Feb. 22, 2019 (GLOBE NEWSWIRE) -- ...
```

```
      symbols  sentiment_polarity \
0      AAPL.US, AVHI.US, FARM.US, GM.US, SGRY.US      0.995
1 AAPL.US, AMZN.US, BA.US, BUD.US, CAJ.US, CAT.U...      0.991
2      AAPL.US, OLED.US      -0.244
3 AAPL.US, FB.US, GD.US, GRMN.US, HD.US, WBA.US      0.997
4      AAPL.US, HNHPPF.US      -0.807
5 AAPL.US, BTX.US, CTLT.US, HD.US, PDLI.US, QURE.US      0.995
6 AAPL.US, BIO.US, COT.US, DAR.US, OMER.US, OR.US      0.998
7      AAPL.US      0.992
8      AAPL.US      0.967
9 AAPL.US, CBAY.US, FTK.US, GVA.US, IMGN.US, UAL.US      0.997
```

```
      sentiment_neg  sentiment_neu  sentiment_pos
0      0.009      0.937      0.054
1      0.011      0.917      0.072
2      0.134      0.764      0.101
3      0.008      0.930      0.062
4      0.138      0.862      0.000
5      0.008      0.941      0.051
```

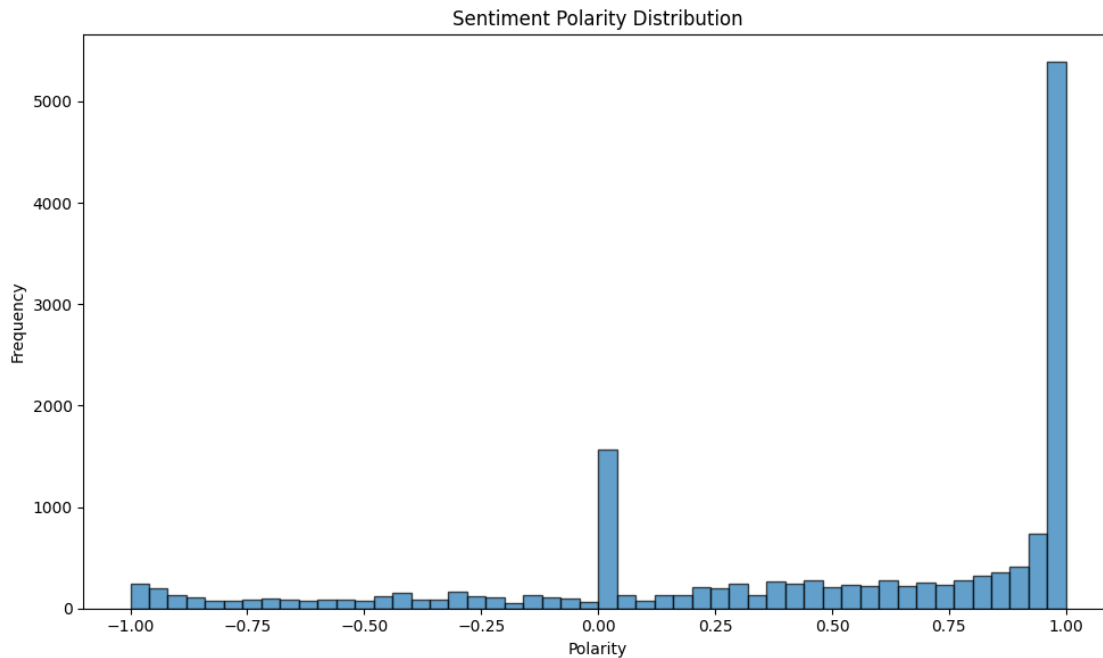
6	0.009	0.925	0.066
7	0.038	0.816	0.147
8	0.050	0.839	0.111
9	0.009	0.928	0.064

3.2 EDA Sentiment Dataset

Visual Analysis:

- Sentiment polarity distribution histogram
- Sentiment scores comparison (neg/neu/pos) boxplot
- Articles count over time (monthly trend)
- Average sentiment polarity over time with zero baseline

```
[22]: # Sentiment polarity distribution
plt.figure(figsize=(10, 6))
plt.hist(raw_train['sentiment_polarity'], bins=50, edgecolor='black', alpha=0.7)
plt.title('Sentiment Polarity Distribution')
plt.xlabel('Polarity')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()
```

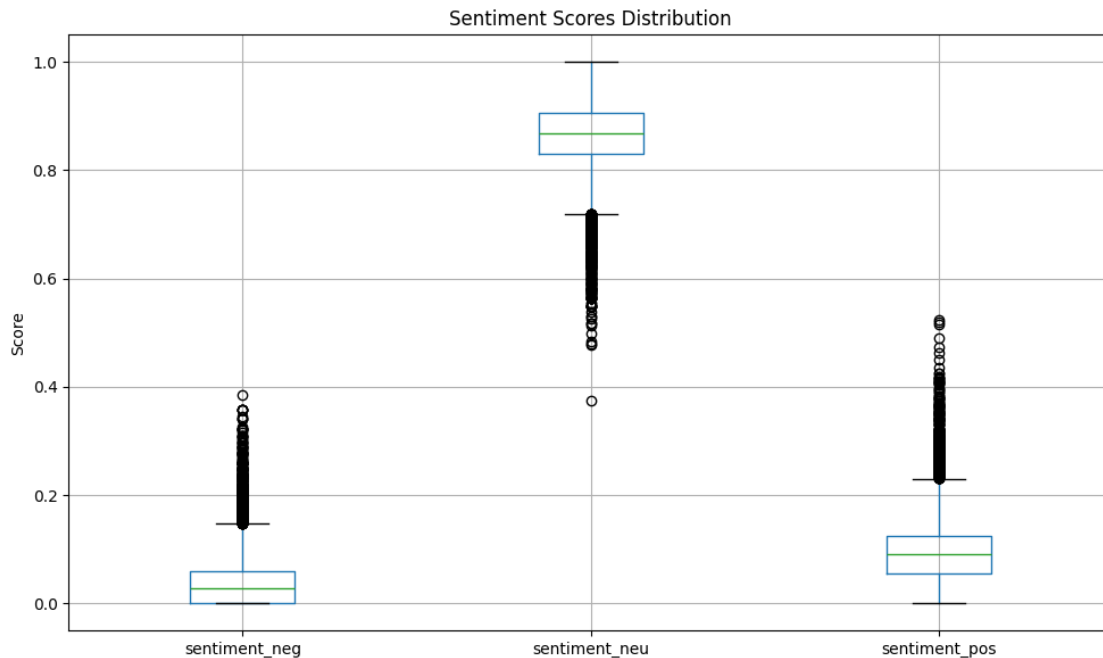


```
[23]: # Sentiment scores comparison
plt.figure(figsize=(10, 6))
sentiment_cols = ['sentiment_neg', 'sentiment_neu', 'sentiment_pos']
```

```

raw_train[sentiment_cols].boxplot()
plt.title('Sentiment Scores Distribution')
plt.ylabel('Score')
plt.tight_layout()
plt.show()

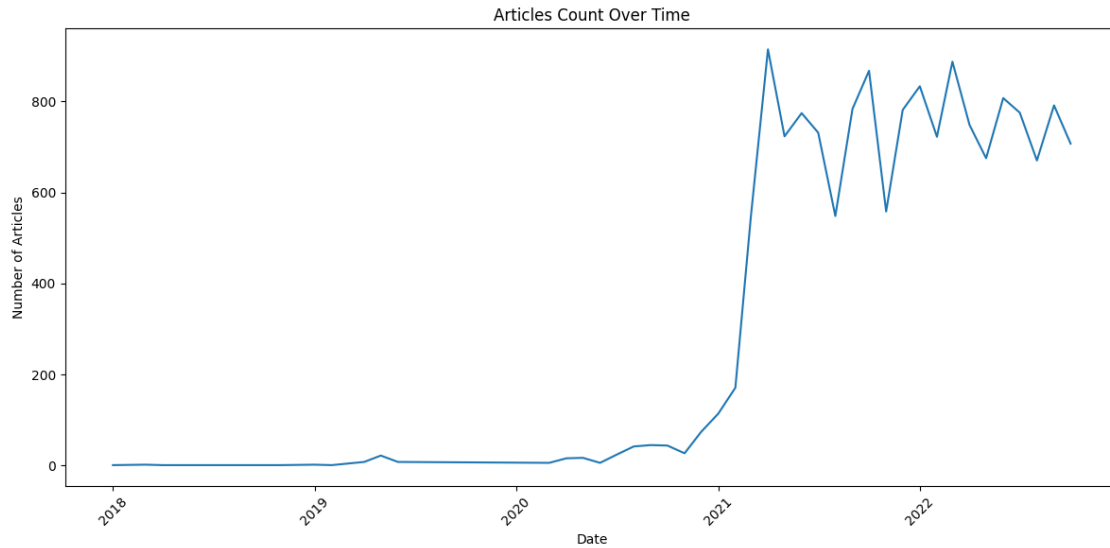
```



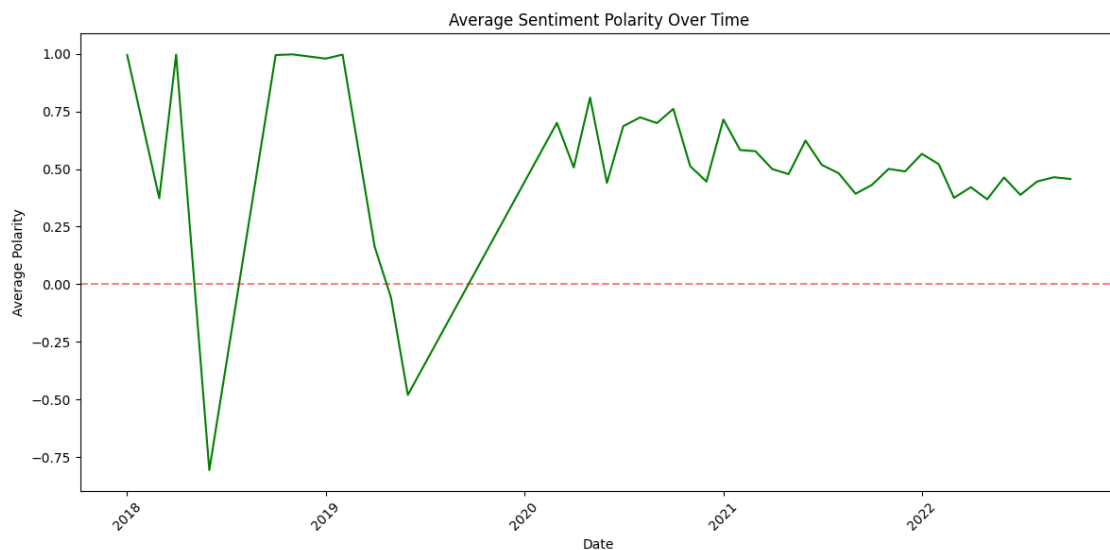
```

[24]: # Articles per date
raw_train['date'] = pd.to_datetime(raw_train['date'])
articles_per_date = raw_train.groupby(raw_train['date'].dt.to_period('M')).size()
plt.figure(figsize=(12, 6))
plt.plot(articles_per_date.index.to_timestamp(), articles_per_date.values)
plt.title('Articles Count Over Time')
plt.xlabel('Date')
plt.ylabel('Number of Articles')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



```
[25]: # Average sentiment over time
avg_sentiment = raw_train.groupby(raw_train['date'].dt.
    ↳to_period('M'))['sentiment_polarity'].mean()
plt.figure(figsize=(12, 6))
plt.plot(avg_sentiment.index.to_timestamp(), avg_sentiment.values, color='green')
plt.title('Average Sentiment Polarity Over Time')
plt.xlabel('Date')
plt.ylabel('Average Polarity')
plt.axhline(y=0, color='r', linestyle='--', alpha=0.5)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



The sentiment polarity plot shows many missing values at the start of the time series. Then, there are some extreme highs and lows. Many of these are close to 1 (maximum) and -1 (minimum) and a few are exactly 1.

There are some low outliers in 2020. These could be explained by the COVID-19 pandemic, which brought uncertainty to the stock market and the economy as a whole.

Most of the sentiment scores, especially in the latter half, are overall positive.

Also, since we know that sentiment polarity scores in this dataset are all in the -1 to 1 range, we can confirm that there are no illogical values in the train data by looking at this plot.

Next, we can examine the extent of missing values, specifically within the sentiment polarity feature.

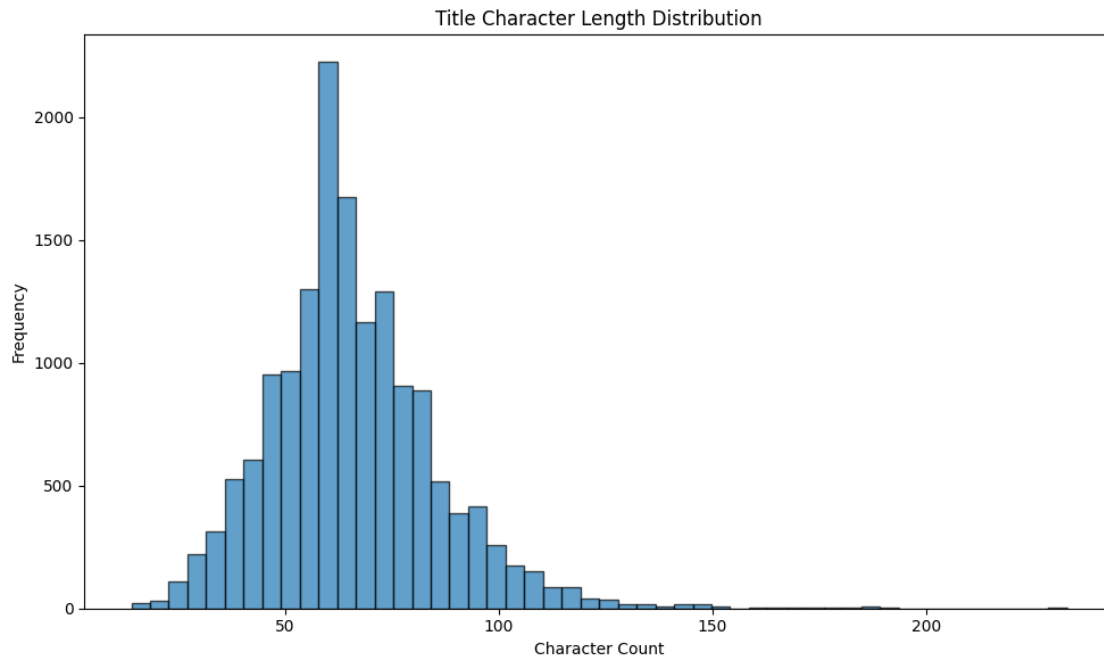
NLP Analysis on Title:

- Character length and word count distributions
- Average statistics

```
[26]: # Title length distribution
raw_train['title_length'] = raw_train['title'].fillna('').str.len()
      ↪ # Title word count

# Title character length
plt.figure(figsize=(10, 6))
plt.hist(raw_train['title_length'], bins=50, edgecolor='black', alpha=0.7)
plt.title('Title Character Length Distribution')
plt.xlabel('Character Count')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

print(f"\nAverage title length: {raw_train['title_length'].mean():.2f} ↪
      ↪characters")
```

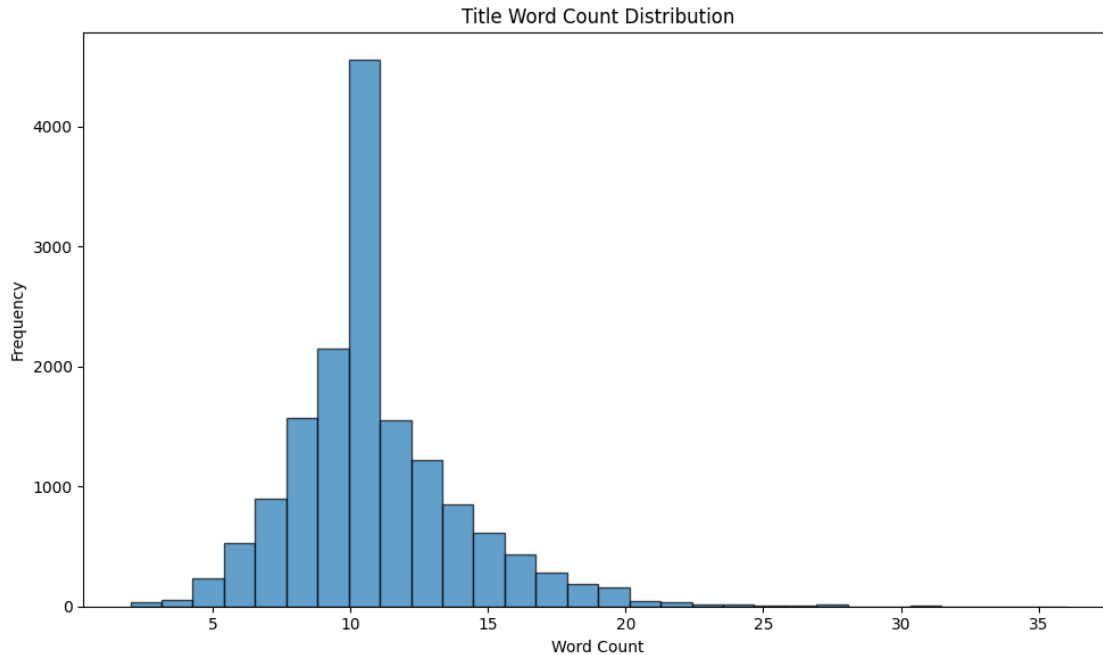


Average title length: 65.83 characters

```
[27]: # Title word count
raw_train['title_word_count'] = raw_train['title'].fillna('').str.split().str.
    ↪len()

# Title word count
plt.figure(figsize=(10, 6))
plt.hist(raw_train['title_word_count'], bins=30, edgecolor='black', alpha=0.7)
plt.title('Title Word Count Distribution')
plt.xlabel('Word Count')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

print(f"Average title word count: {raw_train['title_word_count'].mean():.2f}␣
    ↪words")
```



Average title word count: 10.85 words

- Top 20 most common words
- NLP Analysis on Content:

Character length and word count distributions Average statistics and missing content count Correlation Analysis:

Sentiment vs title/content length scatter plots

```
[28]: # Most common words in titles
all_title_words = ' '.join(
    raw_train['title']
    .fillna('')
    .astype(str)    # AI Added code to help prevent code breakage in future due
    ↳to warning errors.
).lower()

words = re.findall(r'\b[a-z]+\b', all_title_words)
word_freq = Counter(words)
common_words = word_freq.most_common(20)

print("\nTop 20 most common words in titles:")
for word, count in common_words:
    print(f" {word}: {count}")
```

Top 20 most common words in titles:


```

apple: 5885
to: 5033
s: 3821
stocks: 2929
the: 2874
in: 2615
stock: 2316
and: 1933
on: 1873
as: 1798
for: 1730
buy: 1617
of: 1600
a: 1587
is: 1494
tech: 1490
market: 1359
dow: 1321
earnings: 1098
with: 973

```

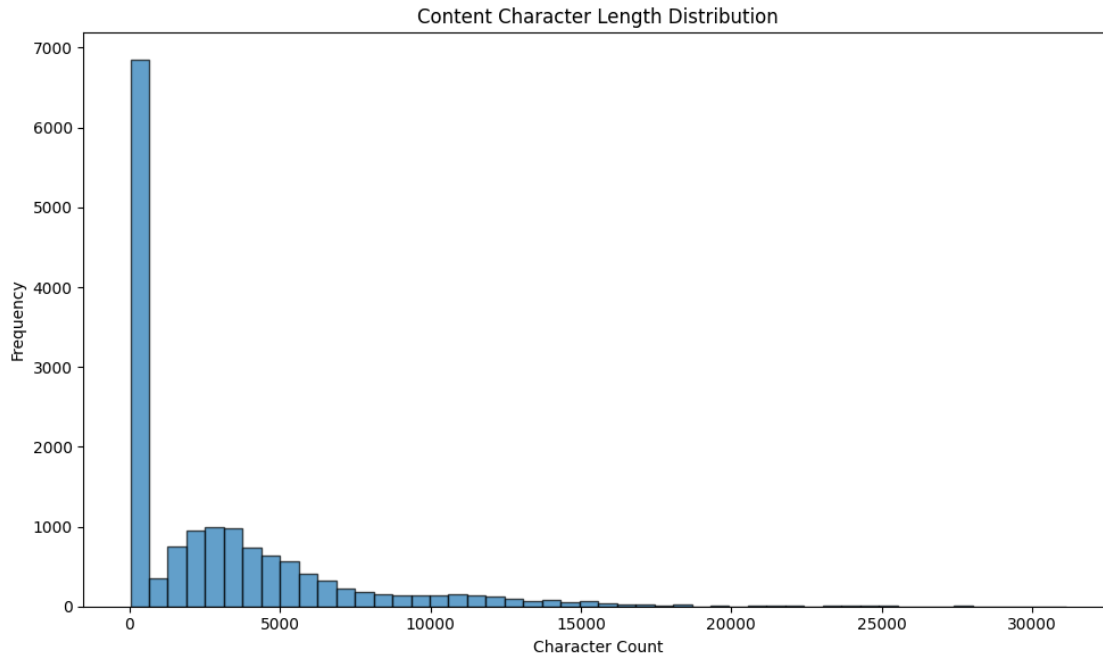
```

[29]: # Content length distribution
raw_train.loc[:, 'content_length'] = (
    raw_train['content']
    .fillna('')
    .astype(str)
    .str.len()
) # AI Added code to help prevent code breakage in future due to warning errors.

# Content character length
plt.figure(figsize=(10, 6))
plt.hist(raw_train['content_length'], bins=50, edgecolor='black', alpha=0.7)
plt.title('Content Character Length Distribution')
plt.xlabel('Character Count')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

print(f"\nAverage content length: {raw_train['content_length'].mean():.2f}␣
↪characters")

```

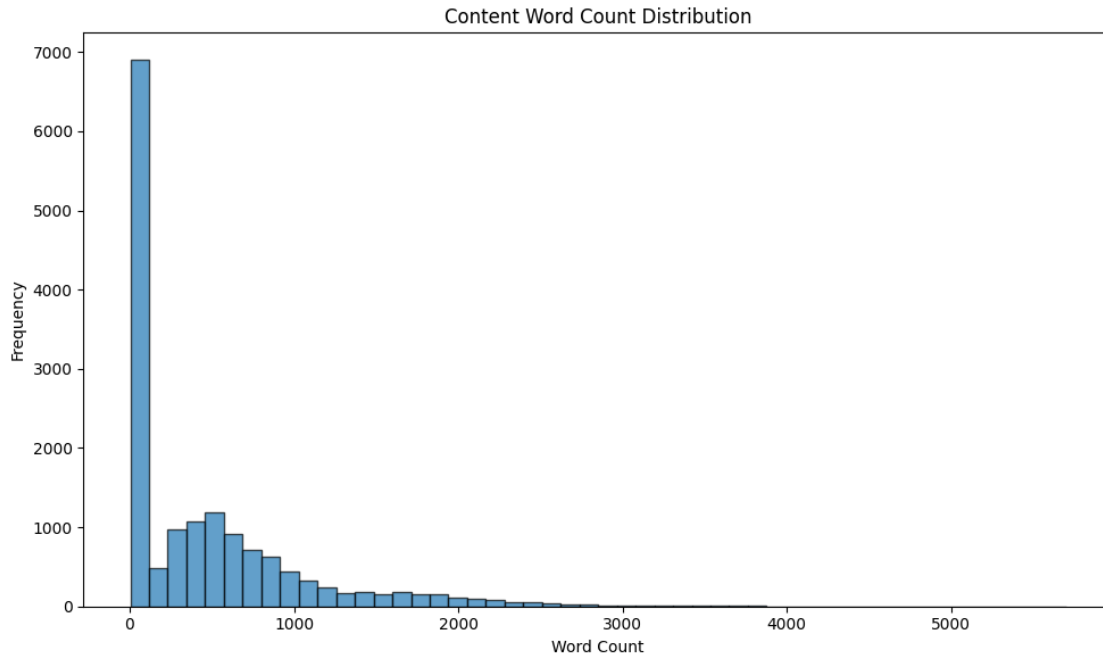


Average content length: 3020.90 characters

```
[30]: # Content word count
raw_train.loc[:, 'content_word_count'] = (
    raw_train['content']
    .fillna('')
    .astype(str)
    .str.split()
    .str.len()
) # AI Added code to help prevent code breakage in future due to warning errors.

# Content word count distribution
plt.figure(figsize=(10, 6))
plt.hist(raw_train['content_word_count'], bins=50, edgecolor='black', alpha=0.7)
plt.title('Content Word Count Distribution')
plt.xlabel('Word Count')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

print(f"Average content word count: {raw_train['content_word_count'].mean():.2f} \n
      ↪ words")
```

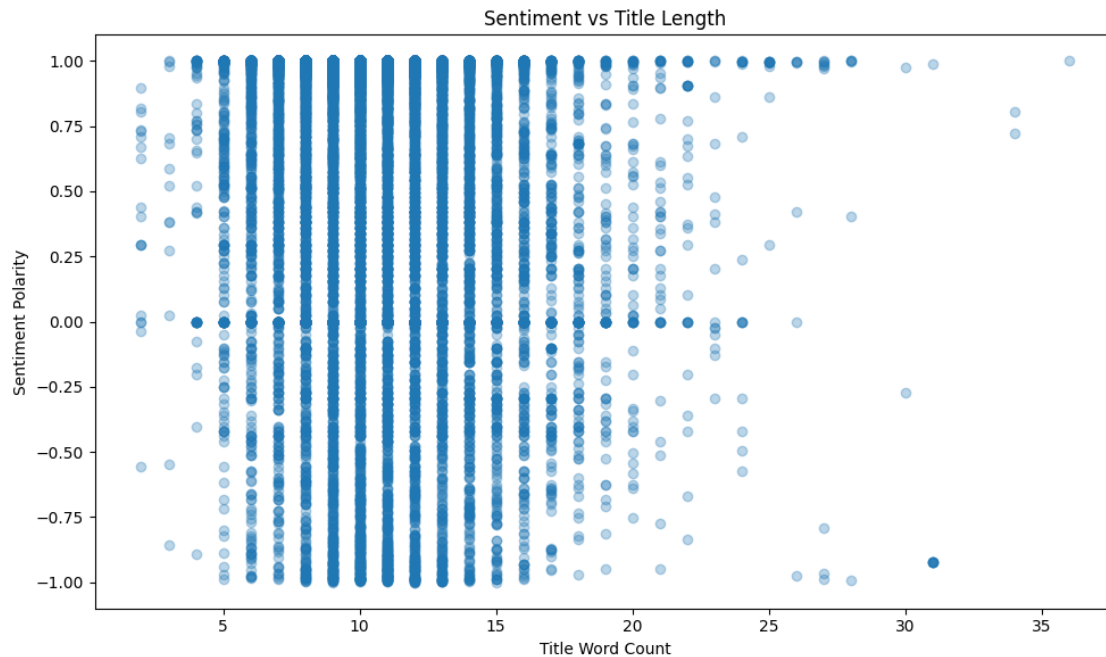


Average content word count: 481.55 words

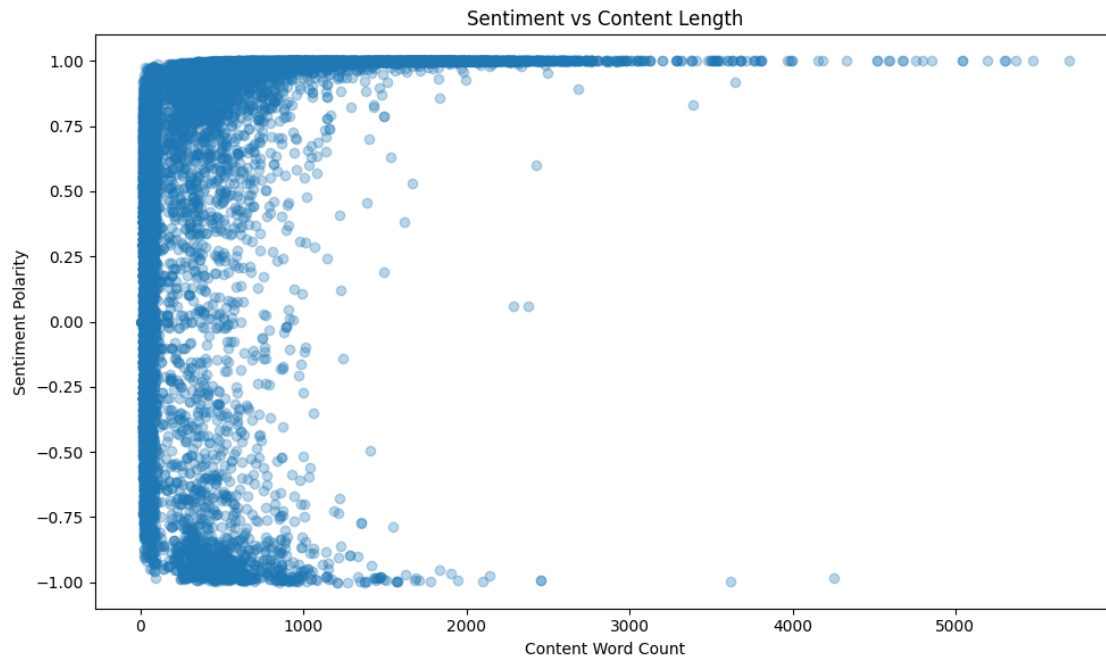
```
[31]: print(f"Missing content: {raw_train['content'].isna().sum()} articles")
```

Missing content: 0 articles

```
[32]: # Sentiment vs Title Length
plt.figure(figsize=(10, 6))
plt.scatter(raw_train['title_word_count'], raw_train['sentiment_polarity'], alpha=0.3)
plt.title('Sentiment vs Title Length')
plt.xlabel('Title Word Count')
plt.ylabel('Sentiment Polarity')
plt.tight_layout()
plt.show()
```



```
[33]: # Sentiment vs Content Length
plt.figure(figsize=(10, 6))
plt.scatter(raw_train['content_word_count'], raw_train['sentiment_polarity'], alpha=0.3)
plt.title('Sentiment vs Content Length')
plt.xlabel('Content Word Count')
plt.ylabel('Sentiment Polarity')
plt.tight_layout()
plt.show()
```



3.3 Split Price Dataset

```
[34]: ts_data = ts_data.sort_values("Dates").reset_index(drop=True)

ts_train = ts_data[(ts_data["Dates"] >= "2018-01-01") & (ts_data["Dates"] <=
↪ "2022-10-31")]
ts_val   = ts_data[(ts_data["Dates"] >= "2022-11-01") & (ts_data["Dates"] <=
↪ "2023-10-31")]
ts_test  = ts_data[(ts_data["Dates"] >= "2023-11-01") & (ts_data["Dates"] <=
↪ "2024-10-31")]

print(ts_train.shape)
print(ts_val.shape)
print(ts_test.shape)

ts_train.head(10)
```

```
(1217, 2)
```

```
(251, 2)
```

```
(252, 2)
```

```
[34]:      Dates  AAPL Share Price
0 2018-01-02      43.07
1 2018-01-03      43.06
2 2018-01-04      43.26
```

3	2018-01-05	43.75
4	2018-01-08	43.59
5	2018-01-09	43.58
6	2018-01-10	43.57
7	2018-01-11	43.82
8	2018-01-12	44.27
9	2018-01-16	44.05

3.4 EDA Price Dataset

```
[35]: ts_train.describe()
```

```
[35]:
```

	Dates	AAPL Share Price
count	1217	1217.000000
mean	2020-06-01 17:34:15.875102720	96.583131
min	2018-01-02 00:00:00	35.550000
25%	2019-03-20 00:00:00	50.690000
50%	2020-06-03 00:00:00	81.300000
75%	2021-08-17 00:00:00	142.000000
max	2022-10-31 00:00:00	182.010000
std	NaN	46.350782

```
[36]: # Check for missing values
print("Missing values in ts_train:")
print(ts_train.isnull().sum())
print(f"\nPercentage of missing values:")
print((ts_train.isnull().sum() / len(ts_train) * 100).round(2))
```

Missing values in ts_train:

```
Dates      0
AAPL Share Price  0
dtype: int64
```

Percentage of missing values:

```
Dates      0.0
AAPL Share Price  0.0
dtype: float64
```

```
[37]: # Time Series Plot of Close Price
plt.figure(figsize=(15, 6))
plt.plot(ts_train['Dates'], ts_train['AAPL Share Price'], linewidth=1.5,
         color='blue')
plt.title('Stock Close Price Over Time (Training Data)', fontsize=16,
         fontweight='bold')
plt.xlabel('Date', fontsize=12)
plt.ylabel('Close Price ($)', fontsize=12)
plt.grid(True, alpha=0.3)
plt.tight_layout()
```

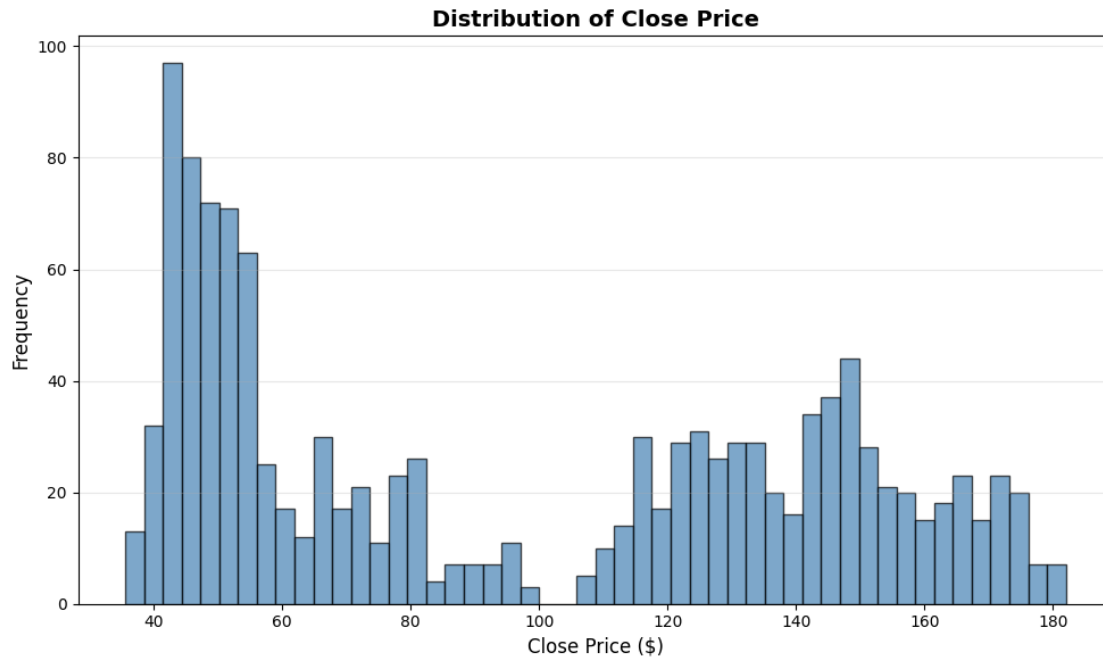
```
plt.show()
```



```
[38]: # Distribution of Close Price
plt.figure(figsize=(10, 6))

# Histogram
plt.hist(ts_train['AAPL Share Price'], bins=50, color='steelblue',
        edgecolor='black', alpha=0.7)
plt.title('Distribution of Close Price', fontsize=14, fontweight='bold')
plt.xlabel('Close Price ($)', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()
```



```
[39]: # STL Decomposition (Seasonal-Trend decomposition using LOESS)
# Prepare time series for STL
ts_for_stl = ts_train.set_index('Dates')['AAPL Share Price']

# Perform STL decomposition
stl = STL(ts_for_stl, seasonal=13, period=252) # 252 trading days per year
stl_result = stl.fit()

# Plot decomposition
fig, axes = plt.subplots(4, 1, figsize=(15, 12))

# Original
axes[0].plot(stl_result.observed, color='blue', linewidth=1)
axes[0].set_ylabel('Observed', fontsize=12)
axes[0].set_title('STL Decomposition of Close Price', fontsize=16,
    ↳fontweight='bold')
axes[0].grid(True, alpha=0.3)

# Trend
axes[1].plot(stl_result.trend, color='green', linewidth=1.5)
axes[1].set_ylabel('Trend', fontsize=12)
axes[1].grid(True, alpha=0.3)

# Seasonal
axes[2].plot(stl_result.seasonal, color='orange', linewidth=1)
```



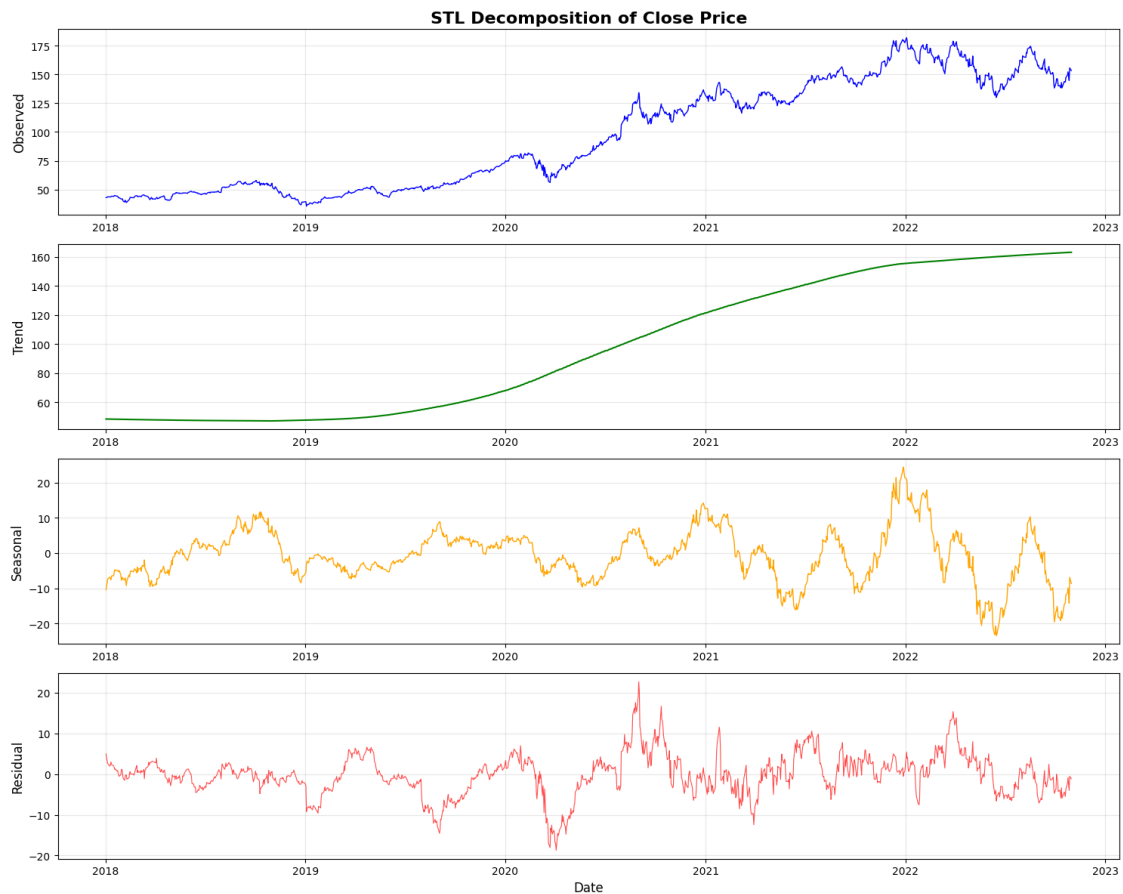
```

axes[2].set_ylabel('Seasonal', fontsize=12)
axes[2].grid(True, alpha=0.3)

# Residual
axes[3].plot(stl_result.resid, color='red', linewidth=0.8, alpha=0.7)
axes[3].set_ylabel('Residual', fontsize=12)
axes[3].set_xlabel('Date', fontsize=12)
axes[3].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



```

[40]: # Rolling Statistics (Moving Average and Standard Deviation)
window_sizes = [7, 30, 90] # 1 week, 1 month, 3 months

fig, axes = plt.subplots(2, 1, figsize=(15, 10))

# Rolling Mean

```

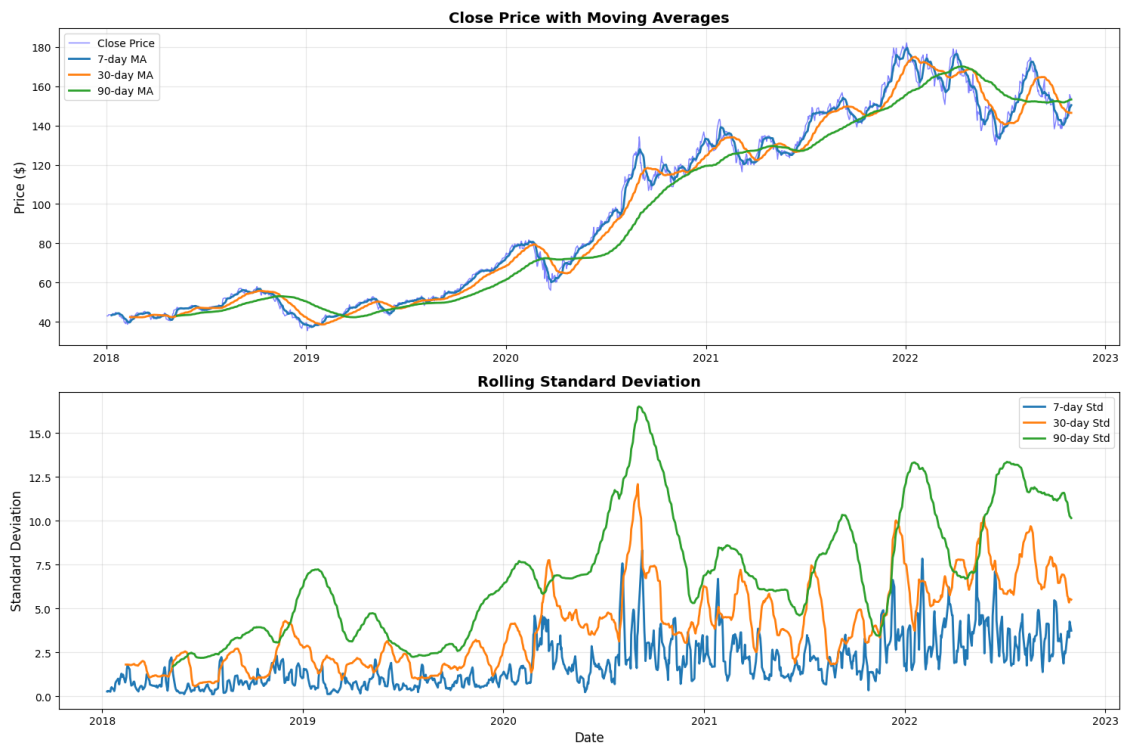
```

axes[0].plot(ts_train['Dates'], ts_train['AAPL Share Price'], label='Close_
↪Price', linewidth=1, alpha=0.5, color='blue')
for window in window_sizes:
    rolling_mean = ts_train['AAPL Share Price'].rolling(window=window).mean()
    axes[0].plot(ts_train['Dates'], rolling_mean, label=f'{window}-day MA',
↪linewidth=2)
axes[0].set_title('Close Price with Moving Averages', fontsize=14,
↪fontweight='bold')
axes[0].set_ylabel('Price ($)', fontsize=12)
axes[0].legend(loc='best')
axes[0].grid(True, alpha=0.3)

# Rolling Standard Deviation
for window in window_sizes:
    rolling_std = ts_train['AAPL Share Price'].rolling(window=window).std()
    axes[1].plot(ts_train['Dates'], rolling_std, label=f'{window}-day Std',
↪linewidth=2)
axes[1].set_title('Rolling Standard Deviation', fontsize=14, fontweight='bold')
axes[1].set_xlabel('Date', fontsize=12)
axes[1].set_ylabel('Standard Deviation', fontsize=12)
axes[1].legend(loc='best')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



During the period of time covered by the train data, AAPL stock fluctuates, but increases overall from July, 2020 to about January 2022. Then, it continues to fluctuate but trends down until July, 2022.

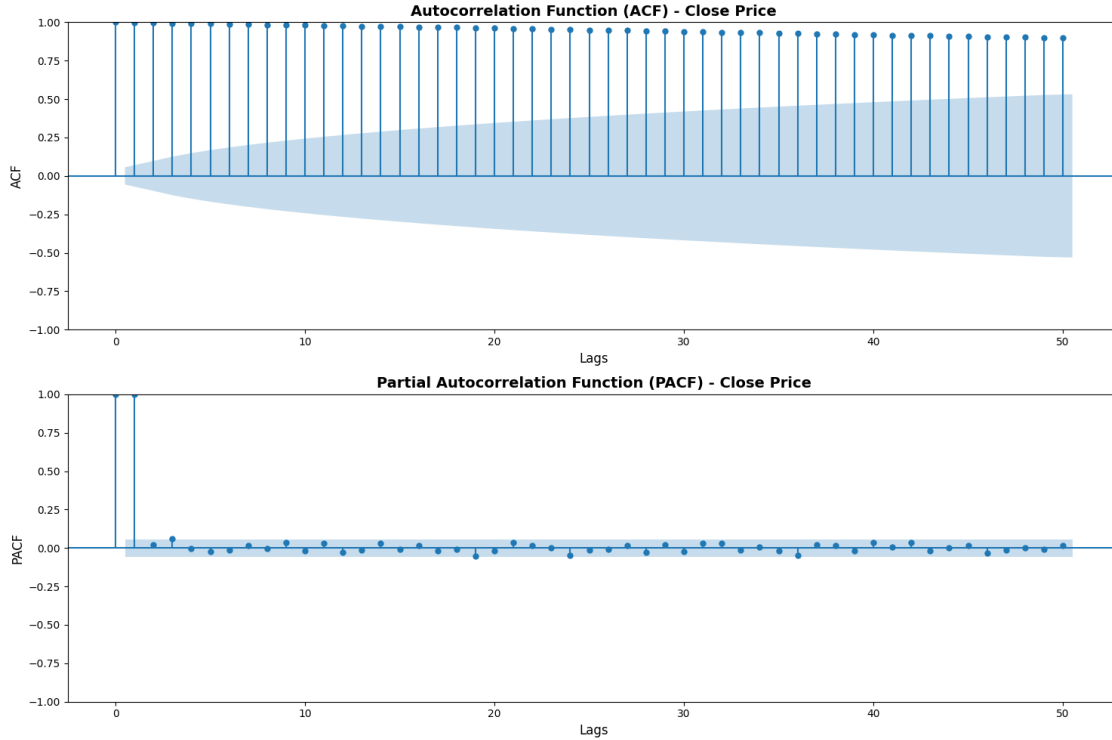
We can also build a plot of the sentiment polarity over the same period of time and compare the two charts.

```
[41]: # ACF and PACF Plots
fig, axes = plt.subplots(2, 1, figsize=(15, 10))

# ACF
plot_acf(ts_train['AAPL Share Price'].dropna(), lags=50, ax=axes[0])
axes[0].set_title('Autocorrelation Function (ACF) - Close Price', fontsize=14,
    ↳fontweight='bold')
axes[0].set_xlabel('Lags', fontsize=12)
axes[0].set_ylabel('ACF', fontsize=12)

# PACF
plot_pacf(ts_train['AAPL Share Price'].dropna(), lags=50, ax=axes[1])
axes[1].set_title('Partial Autocorrelation Function (PACF) - Close Price',
    ↳fontsize=14, fontweight='bold')
axes[1].set_xlabel('Lags', fontsize=12)
axes[1].set_ylabel('PACF', fontsize=12)

plt.tight_layout()
plt.show()
```



4 Week 4: Make Data Model Ready

4.1 Sentiment Dataset Cleaning

In order to make the sentiment dataset ready for modeling we will first combine our title and content columns into one full text column. We will remove the columns (title, content, net sentiment, title_length, title_word_count, content_length, content_word_count) that will not be necessary for training our model and aggregate full_text column by date to create a daily text feature for modeling and aggregate sentiment_polarity, sentiment_neg, sentiment_neu and sentiment_pos by mean for each date. This will create a daily summary of the news articles and their sentiment scores.

Combine each dataset (train, validation and test) with the corresponding dates in timeseries dataset (ts_train, ts_val, ts_test) to create the final training datasets that contains the daily aggregated full_text, sentiment_neg, sentiment_neu, and sentiment_pos along with the AAPL Share Price for each date. We use time series dates as the main column dates so that all sentiment datasets dates are aligned with ts_train dates. This will allow us to have a complete dataset for modeling that includes both the text features and the target variable.

Finally we will fill missing data in the full_text column by replacing NaN values with a blank string to avoid putting in placeholders that would otherwise introduce noise into the model. We will forward fill sentiment values up to 7 days as news diminishes overtime and will leave the remaining values at 0.

4.1.1 Data Pre-Processing

```
[42]: #Combine title and content into a single text column for NLP processing and
      ↪consistency across train, val, and test sets.
raw_train['full_text'] = raw_train['title'] + ' ' + raw_train['content']
raw_val['full_text'] = raw_val['title'] + ' ' + raw_val['content']
raw_test['full_text'] = raw_test['title'] + ' ' + raw_test['content']

[43]: # Remove unnecessary columns from train, val, and test sets to focus on text and
      ↪sentiment features.
raw_train = raw_train.drop(columns=['title', 'content', 'title_length',
      ↪'title_word_count', 'content_length', 'content_word_count'])
raw_val = raw_val.drop(columns=['title', 'content'])
raw_test = raw_test.drop(columns=['title', 'content'])

[44]: # print out the columns and data types for each dataset to ensure they are
      ↪consistent and correctly formatted for modeling.
print('Training set columns and data types:')
for col in raw_train.columns:
    print(f'Column: {col} , Data Type: {raw_train[col].dtype}')

print('\nValidation set columns and data types:')
for col in raw_val.columns:
    print(f'Column: {col} , Data Type: {raw_val[col].dtype}')

print('\nTest set columns and data types:')
for col in raw_test.columns:
    print(f'Column: {col} , Data Type: {raw_test[col].dtype}')
```

```
Training set columns and data types:
Column: date , Data Type: datetime64[ns]
Column: symbols , Data Type: object
Column: sentiment_polarity , Data Type: float64
Column: sentiment_neg , Data Type: float64
Column: sentiment_neu , Data Type: float64
Column: sentiment_pos , Data Type: float64
Column: full_text , Data Type: object
```

```
Validation set columns and data types:
Column: date , Data Type: datetime64[ns]
Column: symbols , Data Type: object
Column: sentiment_polarity , Data Type: float64
Column: sentiment_neg , Data Type: float64
Column: sentiment_neu , Data Type: float64
Column: sentiment_pos , Data Type: float64
Column: full_text , Data Type: object
```

```
Test set columns and data types:
```

```

Column: date , Data Type: datetime64[ns]
Column: symbols , Data Type: object
Column: sentiment_polarity , Data Type: float64
Column: sentiment_neg , Data Type: float64
Column: sentiment_neu , Data Type: float64
Column: sentiment_pos , Data Type: float64
Column: full_text , Data Type: object

```

```

[45]: # Aggregate columns by date to create a daily text feature for modeling
      # Aggregate sentiment columns by mean for each date.
raw_train = raw_train.groupby('date').agg({
    'full_text': lambda x: ' '.join(x), # Combine all text for the day
    'sentiment_neg': 'mean',
    'sentiment_neu': 'mean',
    'sentiment_pos': 'mean',
    'sentiment_polarity': 'mean'
})

raw_val = raw_val.groupby('date').agg({
    'full_text': lambda x: ' '.join(x), # Combine all text for the day
    'sentiment_neg': 'mean',
    'sentiment_neu': 'mean',
    'sentiment_pos': 'mean',
    'sentiment_polarity': 'mean'
})

raw_test = raw_test.groupby('date').agg({
    'full_text': lambda x: ' '.join(x), # Combine all text for the day
    'sentiment_neg': 'mean',
    'sentiment_neu': 'mean',
    'sentiment_pos': 'mean',
    'sentiment_polarity': 'mean'
})

```

```

[46]: raw_train.head(10)

```

```

[46]:
      date      full_text  sentiment_neg \
2018-01-31  Investor Expectations to Drive Momentum within...      0.009
2018-03-16  Top 100 Reputable Companies Around the Globe A...      0.011
2018-03-27  Universal Display Corporation Stock Is Way Und...      0.134
2018-04-16  Detailed Research: Economic Perspectives on Ge...      0.008
2018-06-22  Apple iPhone Spared Tariffs, But Could Face Ch...      0.138
2018-10-23  Report: Exploring Fundamental Drivers Behind T...      0.008
2018-11-30  New Research Coverage Highlights Omeros, Apple...      0.009
2019-01-03  Apple, AAPL Investment Losses Alert: Bernstein...      0.044
2019-02-22  Factors of Influence in 2019, Key Indicators a...      0.009

```

	sentiment_neu	sentiment_pos	sentiment_polarity
date			
2018-01-31	0.9370	0.054	0.9950
2018-03-16	0.9170	0.072	0.9910
2018-03-27	0.7640	0.101	-0.2440
2018-04-16	0.9300	0.062	0.9970
2018-06-22	0.8620	0.000	-0.8070
2018-10-23	0.9410	0.051	0.9950
2018-11-30	0.9250	0.066	0.9980
2019-01-03	0.8275	0.129	0.9795
2019-02-22	0.9280	0.064	0.9970
2019-04-01	0.9430	0.049	0.9940

4.1.2 NLP Pre-Processing

```
[47]: # lowercase the text for consistency in NLP processing
raw_train['full_text'] = raw_train['full_text'].str.lower()
raw_val['full_text'] = raw_val['full_text'].str.lower()
raw_test['full_text'] = raw_test['full_text'].str.lower()

[48]: # Remove punctuation and special characters from the text
raw_train['full_text'] = raw_train['full_text'].str.replace(r'[^\w\s]', '',
    ↪regex=True)
raw_val['full_text'] = raw_val['full_text'].str.replace(r'[^\w\s]', '',
    ↪regex=True)
raw_test['full_text'] = raw_test['full_text'].str.replace(r'[^\w\s]', '',
    ↪regex=True)

[49]: # Remove extra whitespace
raw_train['full_text'] = raw_train['full_text'].str.replace(r'\s+', ' ',
    ↪regex=True).str.strip()
raw_val['full_text'] = raw_val['full_text'].str.replace(r'\s+', ' ', regex=True).
    ↪str.strip()
raw_test['full_text'] = raw_test['full_text'].str.replace(r'\s+', ' ',
    ↪regex=True).str.strip()

[50]: # Remove stop words (common words that may not add much meaning to the text)
raw_train['full_text'] = raw_train['full_text'].apply(
    lambda x: ' '.join([word for word in x.split() if word not in
    ↪ENGLISH_STOP_WORDS])
)
raw_val['full_text'] = raw_val['full_text'].apply(
    lambda x: ' '.join([word for word in x.split() if word not in
    ↪ENGLISH_STOP_WORDS])
)

```

```

raw_test['full_text'] = raw_test['full_text'].apply(
    lambda x: ' '.join([word for word in x.split() if word not in
        ↪ENGLISH_STOP_WORDS])
)

```

```

[51]: # Tokensize the text data.
raw_train['tokens'] = raw_train['full_text'].str.split()
raw_val['tokens'] = raw_val['full_text'].str.split()
raw_test['tokens'] = raw_test['full_text'].str.split()

```

4.1.3 Final Train, Val and Test sets

```

[52]: # Combine each dataset with corresponding stock price data.
Final_Raw_X_train = pd.merge(ts_train[['Dates', 'AAPL Share Price']], raw_train,
    ↪left_on='Dates', right_index=True, how='left')
Final_Raw_X_val = pd.merge(ts_val[['Dates', 'AAPL Share Price']], raw_val,
    ↪left_on='Dates', right_index=True, how='left')
Final_Raw_X_test = pd.merge(ts_test[['Dates', 'AAPL Share Price']], raw_test,
    ↪left_on='Dates', right_index=True, how='left')

```

```

[53]: #For Nan values in the full_text column, fill with an empty string to allow for
    ↪NLP processing without errors.
Final_Raw_X_train['full_text'] = Final_Raw_X_train['full_text'].fillna('')
Final_Raw_X_val['full_text'] = Final_Raw_X_val['full_text'].fillna('')
Final_Raw_X_test['full_text'] = Final_Raw_X_test['full_text'].fillna('')

# For Nan values in the sentiment columns, forwardfill up to 7 days ahead and
    ↪replace remaining NaNs with 0.
for col in ['sentiment_neg', 'sentiment_neu', 'sentiment_pos',
    ↪'sentiment_polarity']:
    #Final_Raw_X_train[col] = Final_Raw_X_train[col].fillna(method='ffill',
    ↪limit=7).fillna(0)
    #Final_Raw_X_val[col] = Final_Raw_X_val[col].fillna(method='ffill', limit=7).
    ↪fillna(0)
    #Final_Raw_X_test[col] = Final_Raw_X_test[col].fillna(method='ffill',
    ↪limit=7).fillna(0)

    Final_Raw_X_train[col] = Final_Raw_X_train[col].ffill(limit=7).fillna(0)
    Final_Raw_X_val[col] = Final_Raw_X_val[col].ffill(limit=7).fillna(0)
    Final_Raw_X_test[col] = Final_Raw_X_test[col].ffill(limit=7).fillna(0)

```

```

[54]: # Create stock price direction target variable: 'up' if current day's price is
    ↪higher than previous day, 'down' if lower or equal.
Final_Raw_X_train['Stock Price Direction'] = (Final_Raw_X_train['AAPL Share
    ↪Price'] > Final_Raw_X_train['AAPL Share Price'].shift(1)).map({True: 'up',
    ↪False: 'down'})

```



```
Final_Raw_X_val['Stock Price Direction'] = (Final_Raw_X_val['AAPL Share Price']_
↪> Final_Raw_X_val['AAPL Share Price'].shift(1)).map({True: 'up', False:_
↪'down'})
Final_Raw_X_test['Stock Price Direction'] = (Final_Raw_X_test['AAPL Share_
↪Price'] > Final_Raw_X_test['AAPL Share Price'].shift(1)).map({True: 'up',_
↪False: 'down'})
```

```
[55]: Final_Raw_X_train.head(10)
```

```
[55]:      Dates  AAPL Share Price full_text  sentiment_neg  sentiment_neu \
0 2018-01-02      43.07      0.0      0.0
1 2018-01-03      43.06      0.0      0.0
2 2018-01-04      43.26      0.0      0.0
3 2018-01-05      43.75      0.0      0.0
4 2018-01-08      43.59      0.0      0.0
5 2018-01-09      43.58      0.0      0.0
6 2018-01-10      43.57      0.0      0.0
7 2018-01-11      43.82      0.0      0.0
8 2018-01-12      44.27      0.0      0.0
9 2018-01-16      44.05      0.0      0.0
```

	sentiment_pos	sentiment_polarity	tokens	Stock Price Direction
0	0.0	0.0	NaN	down
1	0.0	0.0	NaN	down
2	0.0	0.0	NaN	up
3	0.0	0.0	NaN	up
4	0.0	0.0	NaN	down
5	0.0	0.0	NaN	down
6	0.0	0.0	NaN	down
7	0.0	0.0	NaN	up
8	0.0	0.0	NaN	up
9	0.0	0.0	NaN	down

```
[56]: Final_Raw_X_train.tail(10)
```

```
[56]:      Dates  AAPL Share Price \
1207 2022-10-18      143.75
1208 2022-10-19      143.86
1209 2022-10-20      143.39
1210 2022-10-21      147.27
1211 2022-10-24      149.45
1212 2022-10-25      152.34
1213 2022-10-26      149.35
1214 2022-10-27      144.80
1215 2022-10-28      155.74
1216 2022-10-31      153.34
```

	full_text	sentiment_neg \
1207	10 best sampp 500 stocks buy according hedge f...	0.023971
1208	apple aapl expands portfolio redesigned amp ne...	0.046526
1209	snap stock tumbles sales growth slows advertis...	0.053167
1210	snap stock got crushed early friday catalyst s...	0.036026
1211	apple raises prices services media executives ...	0.036469
1212	investors watch recession clues big tech earni...	0.038718
1213	sony gears report q2 earnings heres expect son...	0.044714
1214	tech live inside appleâs thinking privacy char...	0.037659
1215	stockswall street jumps 2 upbeat apple results...	0.050182
1216	chinese workers flee covid lockdown huge iphon...	0.045520

	sentiment_neu	sentiment_pos	sentiment_polarity \
1207	0.883971	0.092000	0.550735
1208	0.850947	0.102632	0.661947
1209	0.834833	0.111667	0.457667
1210	0.865795	0.098205	0.408718
1211	0.882906	0.080500	0.510625
1212	0.873256	0.087923	0.429282
1213	0.863750	0.091536	0.364393
1214	0.869439	0.092902	0.546146
1215	0.854227	0.095621	0.421227
1216	0.870900	0.083720	0.359380

	tokens	Stock Price Direction
1207	[10, best, sampp, 500, stocks, buy, according,...	up
1208	[apple, aapl, expands, portfolio, redesigned, ...	up
1209	[snap, stock, tumbles, sales, growth, slows, a...	down
1210	[snap, stock, got, crushed, early, friday, cat...	up
1211	[apple, raises, prices, services, media, execu...	up
1212	[investors, watch, recession, clues, big, tech...	up
1213	[sony, gears, report, q2, earnings, heres, exp...	down
1214	[tech, live, inside, appleâs, thinking, privac...	down
1215	[stockswall, street, jumps, 2, upbeat, apple, ...	up
1216	[chinese, workers, flee, covid, lockdown, huge...	down

```
[57]: # Initialize the TF-IDF Vectorizer with a maximum of 10000 features and English
      ↪ stop words removal.
```

```
tfidf_vectorizer = TfidfVectorizer(max_features=10000, stop_words='english')

# Fit on training data and transform
X_train_tfidf = tfidf_vectorizer.fit_transform(Final_Raw_X_train['full_text'])
X_val_tfidf = tfidf_vectorizer.transform(Final_Raw_X_val['full_text'])
X_test_tfidf = tfidf_vectorizer.transform(Final_Raw_X_test['full_text'])
```

```
[58]: # Check distribution of Stock Price Direction
      print("Training Set - Stock Price Direction Distribution:")
```

```
print(Final_Raw_X_train['Stock Price Direction'].value_counts())
print(Final_Raw_X_train['Stock Price Direction'].value_counts(normalize=True).
      ↳round(3))
```

Training Set - Stock Price Direction Distribution:

Stock Price Direction

up 643

down 574

Name: count, dtype: int64

Stock Price Direction

up 0.528

down 0.472

Name: proportion, dtype: float64

4.2 Price Dataset Cleaning

Next, we will clean the price dataset. The price dataset only contains time series AAPL price. The main purpose of this dataset will be to predict the direction of AAPL stock using only historical price data.

Our EDA indicated that there were no noticeable outliers in the stock price data. As a result, no measures were taken to remove outliers.

There are no missing values in the training, validation, or test data, so the step of imputing missing values can be skipped altogether.

Finally, the dataset is transformed to prepare it for a classification problem. With this dataset, we will use the directional movement of AAPL stock in previous days to predict future directional movement.

```
[59]: def price_cleaning(df_price: pd.DataFrame) -> pd.DataFrame:
        df_price = df_price.copy()
        df_price = df_price.sort_values("Dates")

        # Create price lags
        for i in range(1, 6):
            df_price.loc[:, f"aapl_price_lag{i}"] = df_price["AAPL Share Price"].
            ↳shift(i)

        # Create direction lags (1 = Up, 0 = Down)
        for i in range(1, 5):
            df_price.loc[:, f"aapl_price_direction_lag{i}"] = (
                (df_price[f"aapl_price_lag{i}"] - df_price[f"aapl_price_lag{i+1}"])
            ↳
            ↳> 0
                ).astype(int)

        # Create target direction variable
        df_price.loc[:, "Stock Price Direction"] = (
            (df_price["AAPL Share Price"] - df_price["aapl_price_lag1"]) > 0
```

```

).map({True: "Up", False: "Down"})

# Finalize columns
final_cols = [
    "Stock Price Direction", "Dates",
    "aapl_price_direction_lag1", "aapl_price_direction_lag2",
    "aapl_price_direction_lag3", "aapl_price_direction_lag4"
]
return df_price.loc[:, final_cols].reset_index(drop=True)

```

The training, validation, and test sets are cleaned separately, but in the same fashion, to prevent data leakage.

```

[60]: # Apply to training set
cleaned_ts_train = price_cleaning(ts_train)

# Apply to validation and test sets seperately to avoid data leakage
cleaned_ts_val = price_cleaning(ts_val)
cleaned_ts_test = price_cleaning(ts_test)
cleaned_ts_train.head()

```

```

[60]: Stock Price Direction      Dates  aapl_price_direction_lag1  \
0                Down 2018-01-02                0
1                Down 2018-01-03                0
2                 Up 2018-01-04                0
3                 Up 2018-01-05                1
4                Down 2018-01-08                1

      aapl_price_direction_lag2  aapl_price_direction_lag3  \
0                0                0
1                0                0
2                0                0
3                0                0
4                1                0

      aapl_price_direction_lag4
0                0
1                0
2                0
3                0
4                0

```

4.3 Combined Dataset Creation and Cleaning

This section of the notebook will ensure that the combined dataset (including both the text and price series data) is cleaned and model ready. We encapsulate all cleaning and preprocessing methods in one `data_cleaning()` function, which will be called identically on the validation and test datasets.

The first decision is whether to remove outliers. Referencing the line graph above, there are a

few outlying observations with a negative sentiment polarity that are extremely different from the preceding and following days. However, there is still some value in keeping these observations. Many of them coincide with dips in the AAPL stock price, and this could be very valuable information for the classification model. As a result, all outliers will remain in this dataset.

The next decision is on missing data. All missing data comes from the sentiment news dataset as there are some days where no articles on the AAPL stock were published. One basic assumption we can make is that the sentiment in media will be relatively stable until a new article is published with new sentiment. To follow this logic, we institute a forward fill function where the most recent sentiment score will fill in any missing observation. However, we put a cap of a week, insinuating that articles more than a week old are too antiquated to reflect current sentiment.

The next decision is whether to standardize the sentiment news scores. These scores are already bounded (between -1 and 1) and centered (0 is a perfectly neutral post). The current distribution is also very intuitive, a negative number means a negative sentiment, a positive number is a positive sentiment. As a result, this variable does not need to be standardized further, it can remain as is. This may be reconsidered if we use a neural network model as these models strongly prefer standardized data.

```
[61]: def data_cleaning(sentiment_data, ts_data):  
    # Create a copy to avoid modifying the original DataFrame  
    sentiment_data = sentiment_data.copy()  
  
    if 'Stock Price Direction' in sentiment_data.columns:  
        sentiment_data.drop(columns = 'Stock Price Direction', inplace = True)  
        #join on date  
        df = pd.merge(sentiment_data, ts_data, how = 'right', left_on = 'Dates',  
            ↪right_on = 'Dates')  
  
    # Drop 'tokens' column if present (contains lists which are unhashable)  
    if 'tokens' in df.columns:  
        df = df.drop(columns=['tokens'])  
  
    for i in range(1, 6):  
        #create lags for both price and sentiment polarity variables  
        df[f'sentiment_polarity_lag{i}'] = df['sentiment_polarity'].shift(i)  
        df[f'sentiment_pos_lag{i}'] = df['sentiment_pos'].shift(i)  
        df[f'sentiment_neu_lag{i}'] = df['sentiment_neu'].shift(i)  
        df[f'sentiment_neg_lag{i}'] = df['sentiment_neg'].shift(i)  
  
        #drop rows that are missing lagged variables  
        df.dropna(inplace = True)  
  
        #drop duplicates, but there shouldn't be any because of the primary key date  
        df.drop_duplicates(inplace = True)  
  
        #pick the final columns for our dataset
```

```

    final_df = df[['Stock Price Direction', 'AAPL Share Price', 'Dates',
    ↳ 'sentiment_polarity', 'sentiment_polarity_lag1', 'sentiment_polarity_lag2',
    ↳ 'sentiment_polarity_lag3', 'sentiment_polarity_lag4',
    ↳ 'sentiment_polarity_lag5', 'sentiment_pos_lag1', 'sentiment_pos_lag2',
    ↳ 'sentiment_pos_lag3', 'sentiment_pos_lag4', 'sentiment_pos_lag5',
    ↳ 'sentiment_neu_lag1', 'sentiment_neu_lag2', 'sentiment_neu_lag3',
    ↳ 'sentiment_neu_lag4', 'sentiment_neu_lag5', 'sentiment_neg_lag1',
    ↳ 'sentiment_neg_lag2', 'sentiment_neg_lag3', 'sentiment_neg_lag4',
    ↳ 'sentiment_neg_lag5', 'aapl_price_direction_lag1',
    ↳ 'aapl_price_direction_lag2', 'aapl_price_direction_lag3',
    ↳ 'aapl_price_direction_lag4']]

    return final_df

```

```

[62]: #perform data cleaning for training dataset
cleaned_com_train = data_cleaning(Final_Raw_X_train, cleaned_ts_train)
cleaned_com_train.head(10)

```

```

[62]:   Stock Price Direction  AAPL Share Price      Dates  sentiment_polarity \
5           Down          43.58 2018-01-09          0.0
6           Down          43.57 2018-01-10          0.0
7            Up          43.82 2018-01-11          0.0
8            Up          44.27 2018-01-12          0.0
9           Down          44.05 2018-01-16          0.0
10          Up          44.78 2018-01-17          0.0
11          Up          44.82 2018-01-18          0.0
12          Down          44.62 2018-01-19          0.0
13          Down          44.25 2018-01-22          0.0
14          Up          44.26 2018-01-23          0.0

      sentiment_polarity_lag1  sentiment_polarity_lag2  sentiment_polarity_lag3 \
5                0.0                0.0                0.0
6                0.0                0.0                0.0
7                0.0                0.0                0.0
8                0.0                0.0                0.0
9                0.0                0.0                0.0
10               0.0                0.0                0.0
11               0.0                0.0                0.0
12               0.0                0.0                0.0
13               0.0                0.0                0.0
14               0.0                0.0                0.0

      sentiment_polarity_lag4  sentiment_polarity_lag5  sentiment_pos_lag1  ... \
5                0.0                0.0                0.0  ...
6                0.0                0.0                0.0  ...

```

7	0.0	0.0	0.0	...
8	0.0	0.0	0.0	...
9	0.0	0.0	0.0	...
10	0.0	0.0	0.0	...
11	0.0	0.0	0.0	...
12	0.0	0.0	0.0	...
13	0.0	0.0	0.0	...
14	0.0	0.0	0.0	...

	sentiment_neu_lag5	sentiment_neg_lag1	sentiment_neg_lag2	\
5	0.0	0.0	0.0	
6	0.0	0.0	0.0	
7	0.0	0.0	0.0	
8	0.0	0.0	0.0	
9	0.0	0.0	0.0	
10	0.0	0.0	0.0	
11	0.0	0.0	0.0	
12	0.0	0.0	0.0	
13	0.0	0.0	0.0	
14	0.0	0.0	0.0	

	sentiment_neg_lag3	sentiment_neg_lag4	sentiment_neg_lag5	\
5	0.0	0.0	0.0	
6	0.0	0.0	0.0	
7	0.0	0.0	0.0	
8	0.0	0.0	0.0	
9	0.0	0.0	0.0	
10	0.0	0.0	0.0	
11	0.0	0.0	0.0	
12	0.0	0.0	0.0	
13	0.0	0.0	0.0	
14	0.0	0.0	0.0	

	aapl_price_direction_lag1	aapl_price_direction_lag2	\
5	0	1	
6	0	0	
7	0	0	
8	1	0	
9	1	1	
10	0	1	
11	1	0	
12	1	1	
13	0	1	
14	0	0	

	aapl_price_direction_lag3	aapl_price_direction_lag4
5	1	0

6	1	1
7	0	1
8	0	0
9	0	0
10	1	0
11	1	1
12	0	1
13	1	0
14	1	1

[10 rows x 28 columns]

```
[63]: for col in cleaned_com_train.columns:
      print(f'Column: {col} , Data Type: {type(cleaned_com_train[col][20])}')
```

```
Column: Stock Price Direction , Data Type: <class 'str'>
Column: AAPL Share Price , Data Type: <class 'numpy.float64'>
Column: Dates , Data Type: <class 'pandas._libs.tslibs.timestamps.Timestamp'>
Column: sentiment_polarity , Data Type: <class 'numpy.float64'>
Column: sentiment_polarity_lag1 , Data Type: <class 'numpy.float64'>
Column: sentiment_polarity_lag2 , Data Type: <class 'numpy.float64'>
Column: sentiment_polarity_lag3 , Data Type: <class 'numpy.float64'>
Column: sentiment_polarity_lag4 , Data Type: <class 'numpy.float64'>
Column: sentiment_polarity_lag5 , Data Type: <class 'numpy.float64'>
Column: sentiment_pos_lag1 , Data Type: <class 'numpy.float64'>
Column: sentiment_pos_lag2 , Data Type: <class 'numpy.float64'>
Column: sentiment_pos_lag3 , Data Type: <class 'numpy.float64'>
Column: sentiment_pos_lag4 , Data Type: <class 'numpy.float64'>
Column: sentiment_pos_lag5 , Data Type: <class 'numpy.float64'>
Column: sentiment_neu_lag1 , Data Type: <class 'numpy.float64'>
Column: sentiment_neu_lag2 , Data Type: <class 'numpy.float64'>
Column: sentiment_neu_lag3 , Data Type: <class 'numpy.float64'>
Column: sentiment_neu_lag4 , Data Type: <class 'numpy.float64'>
Column: sentiment_neu_lag5 , Data Type: <class 'numpy.float64'>
Column: sentiment_neg_lag1 , Data Type: <class 'numpy.float64'>
Column: sentiment_neg_lag2 , Data Type: <class 'numpy.float64'>
Column: sentiment_neg_lag3 , Data Type: <class 'numpy.float64'>
Column: sentiment_neg_lag4 , Data Type: <class 'numpy.float64'>
Column: sentiment_neg_lag5 , Data Type: <class 'numpy.float64'>
Column: aapl_price_direction_lag1 , Data Type: <class 'numpy.int64'>
Column: aapl_price_direction_lag2 , Data Type: <class 'numpy.int64'>
Column: aapl_price_direction_lag3 , Data Type: <class 'numpy.int64'>
Column: aapl_price_direction_lag4 , Data Type: <class 'numpy.int64'>
```

```
[64]: cleaned_com_train['sentiment_polarity'].describe()
```



```
[64]: count      1212.000000
      mean         0.342105
      std          0.443149
      min        -0.993000
      25%          0.000000
      50%          0.358211
      75%          0.654409
      max           1.000000
      Name: sentiment_polarity, dtype: float64
```

The sentiment polarity looks to be performing as expected. Scores range from -.99 to 1.00, but most are positive (which is expected due to Apple's stellar performance during this time period).

All models will use next-day stock price direction as the main target variable. There will be three possible forecasting models, the traditional model will exclusively use previous stock prices as predictor variables, the second model exclusively uses sentiment analysis as the predictor variable, while the third model will combine both.

```
[65]: cleaned_com_train['sentiment_polarity'].isna().sum()
```

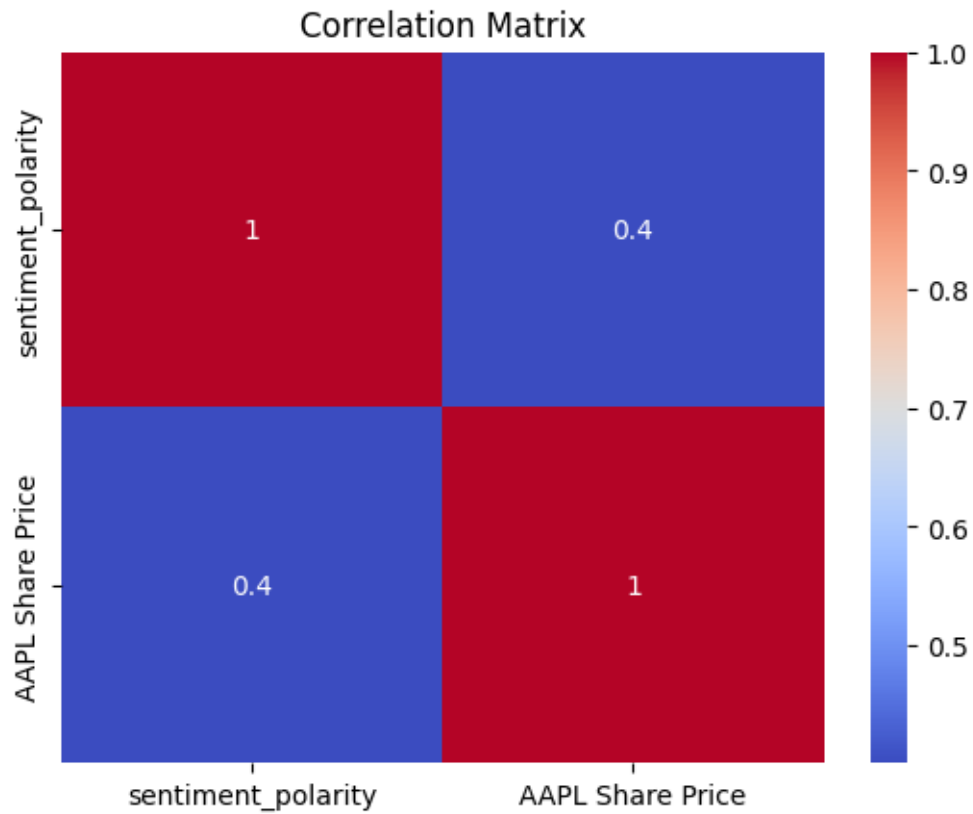
```
[65]: np.int64(0)
```

There are no missing sentiment polarity scores. This is ideal since it will be feature in the final model.

We can also view a correlation matrix to look at how our two variables are correlated.

```
[66]: corr_matrix = cleaned_com_train[["sentiment_polarity", "AAPL Share Price"]].
      ↪corr()

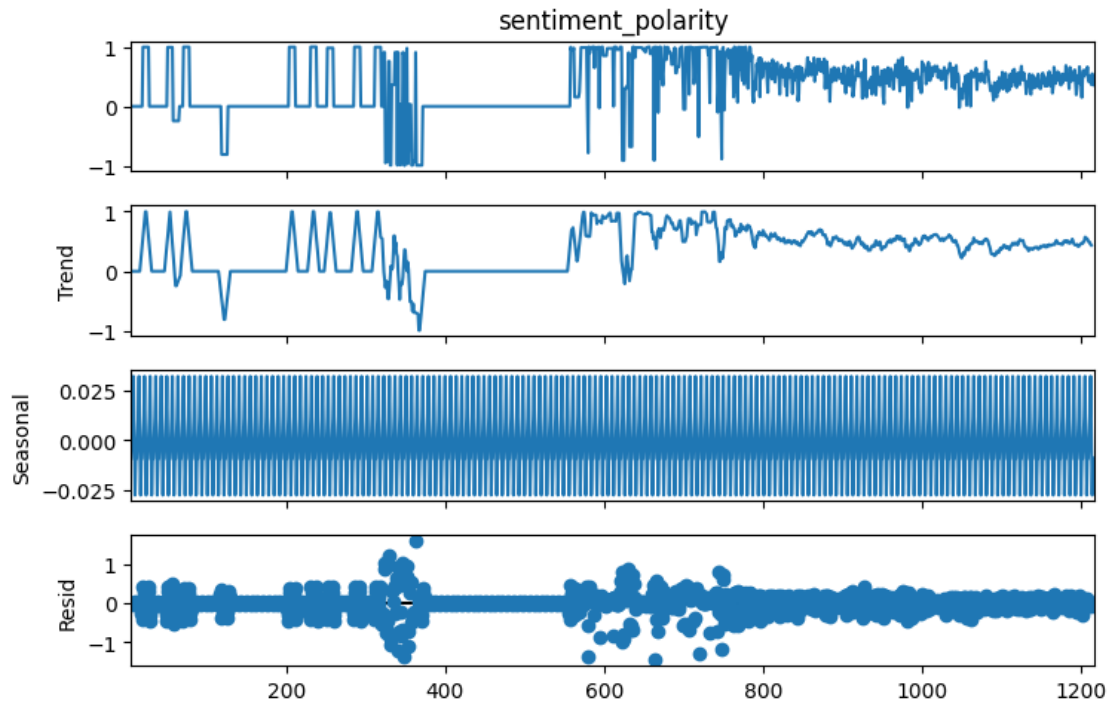
      sns.heatmap(corr_matrix, annot=True, cmap="coolwarm")
      plt.title("Correlation Matrix")
      plt.show()
```



Finally, we can observe a seasonal composition of the train data.

```
[67]: y = cleaned_com_train['sentiment_polarity'].dropna()

decomp = seasonal_decompose(y, model='additive', period=7)
decomp.plot()
plt.gcf().set_size_inches(8, 5)
plt.show()
```



```
[68]: #now that we've performed our EDA, we should drop these columns since they won't
      ↪ be used in the model
cleaned_com_train.drop(columns = ['sentiment_polarity', 'AAPL Share Price',
      ↪ 'Dates'], inplace = True)
```

```
[69]: #perform data cleaning for validation set
cleaned_com_val = data_cleaning(Final_Raw_X_val, cleaned_ts_val)
cleaned_com_val.drop(columns = ['sentiment_polarity', 'AAPL Share Price',
      ↪ 'Dates'], inplace = True)
cleaned_com_val.head(10)
```

```
[69]:
```

	Stock Price Direction	sentiment_polarity_lag1	sentiment_polarity_lag2 \
5	Up	0.102262	0.444375
6	Down	0.398720	0.102262
7	Up	0.399429	0.398720
8	Up	0.482846	0.399429
9	Down	0.707300	0.482846
10	Up	0.284654	0.707300
11	Down	0.404769	0.284654
12	Up	0.624091	0.404769
13	Up	0.613143	0.624091
14	Down	0.465308	0.613143

```

      sentiment_polarity_lag3 sentiment_polarity_lag4 sentiment_polarity_lag5 \

```

5	0.347913	0.283481	0.326217
6	0.444375	0.347913	0.283481
7	0.102262	0.444375	0.347913
8	0.398720	0.102262	0.444375
9	0.399429	0.398720	0.102262
10	0.482846	0.399429	0.398720
11	0.707300	0.482846	0.399429
12	0.284654	0.707300	0.482846
13	0.404769	0.284654	0.707300
14	0.624091	0.404769	0.284654

	sentiment_pos_lag1	sentiment_pos_lag2	sentiment_pos_lag3 \
5	0.055049	0.078083	0.095217
6	0.077240	0.055049	0.078083
7	0.084333	0.077240	0.055049
8	0.078179	0.084333	0.077240
9	0.128350	0.078179	0.084333
10	0.072192	0.128350	0.078179
11	0.088500	0.072192	0.128350
12	0.089591	0.088500	0.072192
13	0.088000	0.089591	0.088500
14	0.103231	0.088000	0.089591

	sentiment_pos_lag4	...	sentiment_neu_lag5	sentiment_neg_lag1 \
5	0.084000	...	0.890783	0.064508
6	0.095217	...	0.860148	0.047560
7	0.078083	...	0.862739	0.046905
8	0.055049	...	0.882292	0.043359
9	0.077240	...	0.880410	0.034300
10	0.084333	...	0.875240	0.048538
11	0.078179	...	0.868714	0.042423
12	0.128350	...	0.878564	0.025318
13	0.072192	...	0.837300	0.036357
14	0.088500	...	0.879192	0.040462

	sentiment_neg_lag2	sentiment_neg_lag3	sentiment_neg_lag4 \
5	0.039750	0.042043	0.055815
6	0.064508	0.039750	0.042043
7	0.047560	0.064508	0.039750
8	0.046905	0.047560	0.064508
9	0.043359	0.046905	0.047560
10	0.034300	0.043359	0.046905
11	0.048538	0.034300	0.043359
12	0.042423	0.048538	0.034300
13	0.025318	0.042423	0.048538
14	0.036357	0.025318	0.042423

	sentiment_neg_lag5	aapl_price_direction_lag1	aapl_price_direction_lag2	\
5	0.040957	1	0	
6	0.055815	1	1	
7	0.042043	0	1	
8	0.039750	1	0	
9	0.064508	1	1	
10	0.047560	0	1	
11	0.046905	1	0	
12	0.043359	0	1	
13	0.034300	1	0	
14	0.048538	1	1	

	aapl_price_direction_lag3	aapl_price_direction_lag4
5	0	0
6	0	0
7	1	0
8	1	1
9	0	1
10	1	0
11	1	1
12	0	1
13	1	0
14	0	1

[10 rows x 25 columns]

```
[70]: #perform data cleaning for test dataset
cleaned_com_test = data_cleaning(Final_Raw_X_test, cleaned_ts_test)
cleaned_com_test.drop(columns = ['sentiment_polarity', 'AAPL Share Price', '
↳ 'Dates'], inplace = True)
cleaned_com_test.head(10)
```

	Stock Price Direction	sentiment_polarity_lag1	sentiment_polarity_lag2	\
5	Up	0.451958	0.493862	
6	Down	0.413867	0.451958	
7	Up	0.481171	0.413867	
8	Down	0.556571	0.481171	
9	Up	0.572346	0.556571	
10	Up	0.652667	0.572346	
11	Up	0.757655	0.652667	
12	Down	0.732615	0.757655	
13	Up	0.557379	0.732615	
14	Down	0.130737	0.557379	

	sentiment_polarity_lag3	sentiment_polarity_lag4	sentiment_polarity_lag5	\
5	0.410016	0.458235	0.558027	
6	0.493862	0.410016	0.458235	

7	0.451958	0.493862	0.410016
8	0.413867	0.451958	0.493862
9	0.481171	0.413867	0.451958
10	0.556571	0.481171	0.413867
11	0.572346	0.556571	0.481171
12	0.652667	0.572346	0.556571
13	0.757655	0.652667	0.572346
14	0.732615	0.757655	0.652667

	sentiment_pos_lag1	sentiment_pos_lag2	sentiment_pos_lag3 \
5	0.097333	0.115655	0.103000
6	0.119333	0.097333	0.115655
7	0.098257	0.119333	0.097333
8	0.093250	0.098257	0.119333
9	0.114115	0.093250	0.098257
10	0.095786	0.114115	0.093250
11	0.112310	0.095786	0.114115
12	0.111346	0.112310	0.095786
13	0.114448	0.111346	0.112310
14	0.055421	0.114448	0.111346

	sentiment_pos_lag4 ...	sentiment_neu_lag5	sentiment_neg_lag1 \
5	0.079216 ...	0.840865	0.035458
6	0.103000 ...	0.884745	0.044700
7	0.115655 ...	0.841859	0.037257
8	0.097333 ...	0.838103	0.023393
9	0.119333 ...	0.867292	0.028577
10	0.098257 ...	0.835967	0.028738
11	0.093250 ...	0.864514	0.020690
12	0.114115 ...	0.883286	0.026115
13	0.095786 ...	0.857385	0.031414
14	0.112310 ...	0.875571	0.021474

	sentiment_neg_lag2	sentiment_neg_lag3	sentiment_neg_lag4 \
5	0.046103	0.055172	0.035961
6	0.035458	0.046103	0.055172
7	0.044700	0.035458	0.046103
8	0.037257	0.044700	0.035458
9	0.023393	0.037257	0.044700
10	0.028577	0.023393	0.037257
11	0.028738	0.028577	0.023393
12	0.020690	0.028738	0.028577
13	0.026115	0.020690	0.028738
14	0.031414	0.026115	0.020690

	sentiment_neg_lag5	aapl_price_direction_lag1	aapl_price_direction_lag2 \
5	0.037216	1	1

6	0.035961	1	1
7	0.055172	0	1
8	0.046103	1	0
9	0.035458	0	1
10	0.044700	1	0
11	0.037257	1	1
12	0.023393	1	1
13	0.028577	0	1
14	0.028738	1	0

	aapl_price_direction_lag3	aapl_price_direction_lag4
5	0	1
6	1	0
7	1	1
8	1	1
9	0	1
10	1	0
11	0	1
12	1	0
13	1	1
14	1	1

[10 rows x 25 columns]

5 Week 5

We created two new features in last week's notebook. These features are lagging, meaning they show positive, neutral, negative and polarity sentiment scores from previous days. This is important because the stock price might not just depend on the previous day's sentiment scores, but other days leading up to it.

We also created lagging variables for AAPL price direction, so that the model has some historical information in price movement while predicting the next day price movement.

Our dataset is already small in number of observations, so dimension reduction is not necessary. However, to have as a later option for a feature set, principal component analysis is completed here to capture 99% of the variance of the original dataset.

```
[71]: #standardize values according to training_dataset
my_scaler = StandardScaler()

cleaned_com_train_scaled = my_scaler.fit_transform(cleaned_com_train.
↳drop(columns = ['Stock Price Direction']))
cleaned_com_val_scaled   = my_scaler.transform(cleaned_com_val.drop(columns =
↳['Stock Price Direction']))
cleaned_com_test_scaled  = my_scaler.transform(cleaned_com_test.drop(columns =
↳['Stock Price Direction']))
```

```
[72]: #use PCA, but only use training dataset to fit, preventing data leakage
pca = PCA(n_components=0.99) #keep 99% of variance

com_train_pca = pca.fit_transform(cleaned_com_train_scaled)
com_val_pca   = pca.transform(cleaned_com_val_scaled)
com_test_pca  = pca.transform(cleaned_com_test_scaled)
```

6 Week 6 - Logistic Regression

Each of the three datasets will use a logistic regression as a baseline model. The model will be trained on the training dataset and evaluated on the validation set.

6.1 Sentiment Dataset

```
[73]: # Prepare target variable - map 'up'/'down' to 1/0
y_train_sentiment = Final_Raw_X_train['Stock Price Direction'].map({'up': 1,
    ↳ 'down': 0})
y_val_sentiment = Final_Raw_X_val['Stock Price Direction'].map({'up': 1, 'down':
    ↳ 0})

# Drop any rows with NaN target values (first row due to shift)
train_mask = ~y_train_sentiment.isna()
X_train_tfidf = X_train_tfidf[train_mask.values]
y_train_sentiment = y_train_sentiment[train_mask]

val_mask = ~y_val_sentiment.isna()
X_val_tfidf = X_val_tfidf[val_mask.values]
y_val_sentiment = y_val_sentiment[val_mask]
```

```
[74]: # Extract sentiment features and combine with TF-IDF
sentiment_cols = ['sentiment_pos', 'sentiment_neu', 'sentiment_neg']
X_train_sentiment = Final_Raw_X_train.loc[train_mask, sentiment_cols].values
X_val_sentiment = Final_Raw_X_val.loc[val_mask, sentiment_cols].values

# Combine TF-IDF features with sentiment features
X_train_combined = hstack([X_train_tfidf, csr_matrix(X_train_sentiment)])
X_val_combined = hstack([X_val_tfidf, csr_matrix(X_val_sentiment)])

print(f"Training set size: {X_train_combined.shape[0]}")
print(f"Validation set size: {X_val_combined.shape[0]}")
print(f"Number of TF-IDF features: {X_train_tfidf.shape[1]}")
print(f"Number of sentiment features: {len(sentiment_cols)}")
print(f"Total combined features: {X_train_combined.shape[1]}")
```

```
Training set size: 1217
Validation set size: 251
Number of TF-IDF features: 10000
```


Number of sentiment features: 3
Total combined features: 10003

```
[75]: # Initialize and train the logistic regression model on TF-IDF features
lr_tfidf = LR(max_iter=500, C=1, random_state=42)
lr_tfidf.fit(X_train_combined, y_train_sentiment)
```

```
[75]: LogisticRegression(C=1, max_iter=500, random_state=42)
```

```
[76]: # Make predictions on validation set
y_pred_combined = lr_tfidf.predict(X_val_combined)

# Evaluate the model
print("Logistic Regression - TF-IDF Text Features Results")

# Accuracy
accuracy = accuracy_score(y_val_sentiment, y_pred_combined)
print(f"\nAccuracy: {accuracy:.4f}")

# Classification Report (Precision, Recall, F1-Score, Support)
print("\nClassification Report:")
print(CR(y_val_sentiment, y_pred_combined, target_names=['Down (0)', 'Up (1)']))

# Confusion Matrix
print("\nConfusion Matrix:")
cm = confusion_matrix(y_val_sentiment, y_pred_combined, labels=lr_tfidf.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Down', 'Up'])

fig, ax = plt.subplots(figsize=(8, 6))
disp.plot(ax=ax, cmap='viridis')
plt.title("Confusion Matrix - Logistic Regression (TF-IDF Features)")
plt.tight_layout()
plt.show()
```

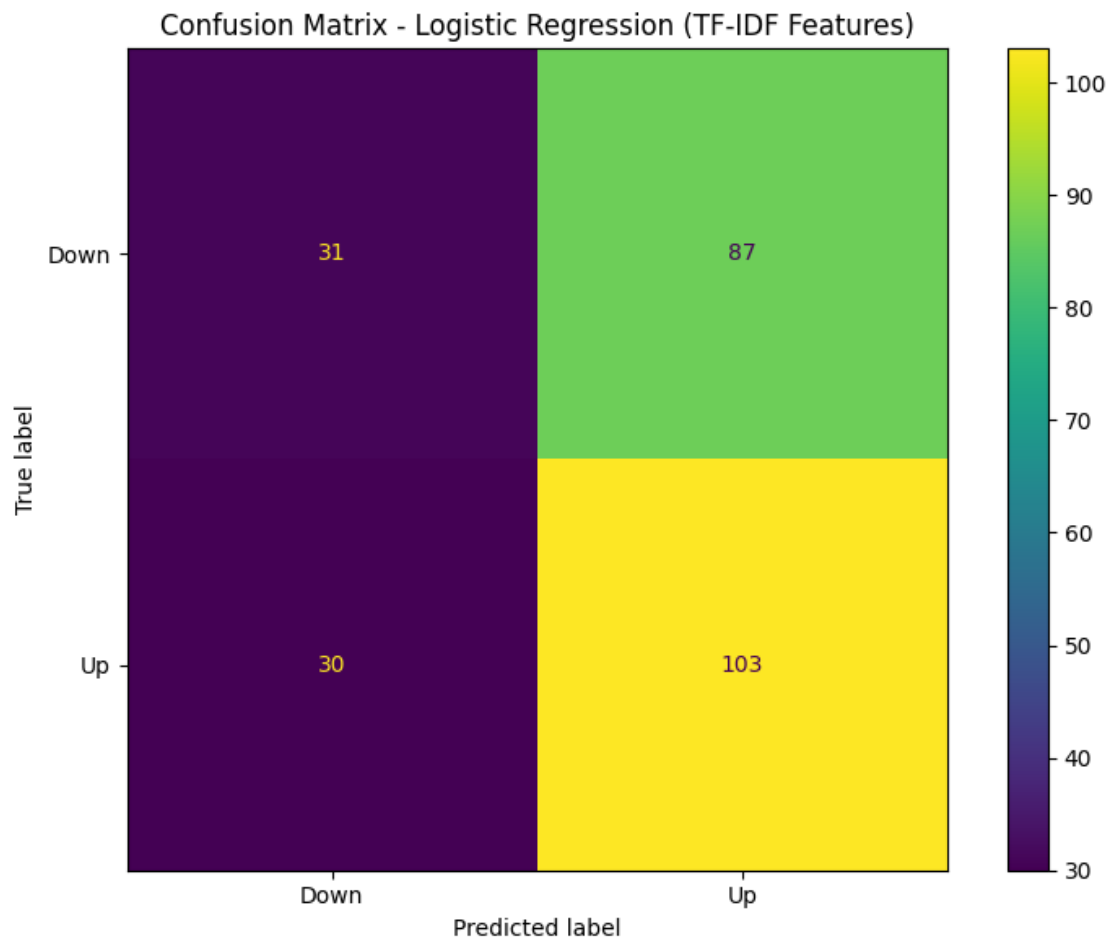
Logistic Regression - TF-IDF Text Features Results

Accuracy: 0.5339

Classification Report:

	precision	recall	f1-score	support
Down (0)	0.51	0.26	0.35	118
Up (1)	0.54	0.77	0.64	133
accuracy			0.53	251
macro avg	0.53	0.52	0.49	251
weighted avg	0.53	0.53	0.50	251

Confusion Matrix:



```
[77]: def gridsearch_sentiment():  
    # Define the hyperparameter grid to search  
    param_grid_sentiment = {  
        'C': [0.01, 0.1, 1, 10],          # Regularization strength  
        'max_iter': [200, 500, 1000],     # Maximum iterations  
        'solver': ['lbfgs', 'liblinear'],  # Optimization algorithm  
        'penalty': ['l2']                  # Regularization type  
    }  
  
    # Initialize the logistic regression model  
    lr_base = LR(random_state=42)  
  
    # Set up GridSearchCV with cross-validation  
    grid_search_sentiment = GridSearchCV(  

```

```

    estimator=lr_base,
    param_grid=param_grid_sentiment,
    cv=5,                                     # 5-fold cross-validation
    scoring='accuracy',
    verbose=1
)

# Fit grid search on training data
print("Starting Grid Search for Logistic Regression on TF-IDF Features...")
grid_search_sentiment.fit(X_train_combined, y_train_sentiment)

# Display best parameters and score
print("Grid Search Results - Sentiment Dataset (TF-IDF)")
print(f"\nBest Parameters: {grid_search_sentiment.best_params_}")
print(f"Best Cross-Validation Accuracy: {grid_search_sentiment.best_score_:.4f}")

# Get the best estimator and CV results
cv_results = grid_search_sentiment.cv_results_

# Create a results table for each parameter combination
results_data = []

for i, params in enumerate(cv_results['params']):
    # Create a model with these parameters
    model = LR(**params, random_state=42)
    model.fit(X_train_tfidf, y_train_sentiment)

    # Training predictions and metrics
    y_train_pred = model.predict(X_train_tfidf)
    train_acc = accuracy_score(y_train_sentiment, y_train_pred)
    train_f1 = f1_score(y_train_sentiment, y_train_pred, average='weighted')

    # Validation predictions and metrics
    y_val_pred = model.predict(X_val_tfidf)
    val_acc = accuracy_score(y_val_sentiment, y_val_pred)
    val_f1 = f1_score(y_val_sentiment, y_val_pred, average='weighted')

    results_data.append({
        'C': f"{params['C']:.3f}",
        'Max Iterations': params['max_iter'],
        'Training Accuracy': f"{train_acc:.3%}",
        'Training F1': f"{train_f1:.3f}",
        'Validation Accuracy': f"{val_acc:.3%}",
        'Validation F1': f"{val_f1:.3f}"
    })

```

```

# Create DataFrame
results_df = pd.DataFrame(results_data)

# Sort by C (descending) and Max Iterations (ascending)
results_df = results_df.sort_values(['C', 'Max Iterations'],
↪ascending=[False, True]).reset_index(drop=True)

best_lr_sentiment = grid_search_sentiment.best_estimator_

#delete grid search to save memory
del grid_search_sentiment
gc.collect()

return results_df, best_lr_sentiment

results_df, best_lr_sentiment = gridsearch_sentiment()

# Display styled table
results_df.style.set_caption("Logistic Regression (TF-IDF Sentiment Dataset)").
↪set_table_styles([
    {'selector': 'caption', 'props': [('font-size', '16px'), ('font-weight',
↪'bold')]}
])

```

Starting Grid Search for Logistic Regression on TF-IDF Features...

Fitting 5 folds for each of 24 candidates, totalling 120 fits

Grid Search Results - Sentiment Dataset (TF-IDF)

Best Parameters: {'C': 0.01, 'max_iter': 200, 'penalty': 'l2', 'solver':
'lbfgs'}

Best Cross-Validation Accuracy: 0.5283

[77]: <pandas.io.formats.style.Styler at 0x7390354550d0>

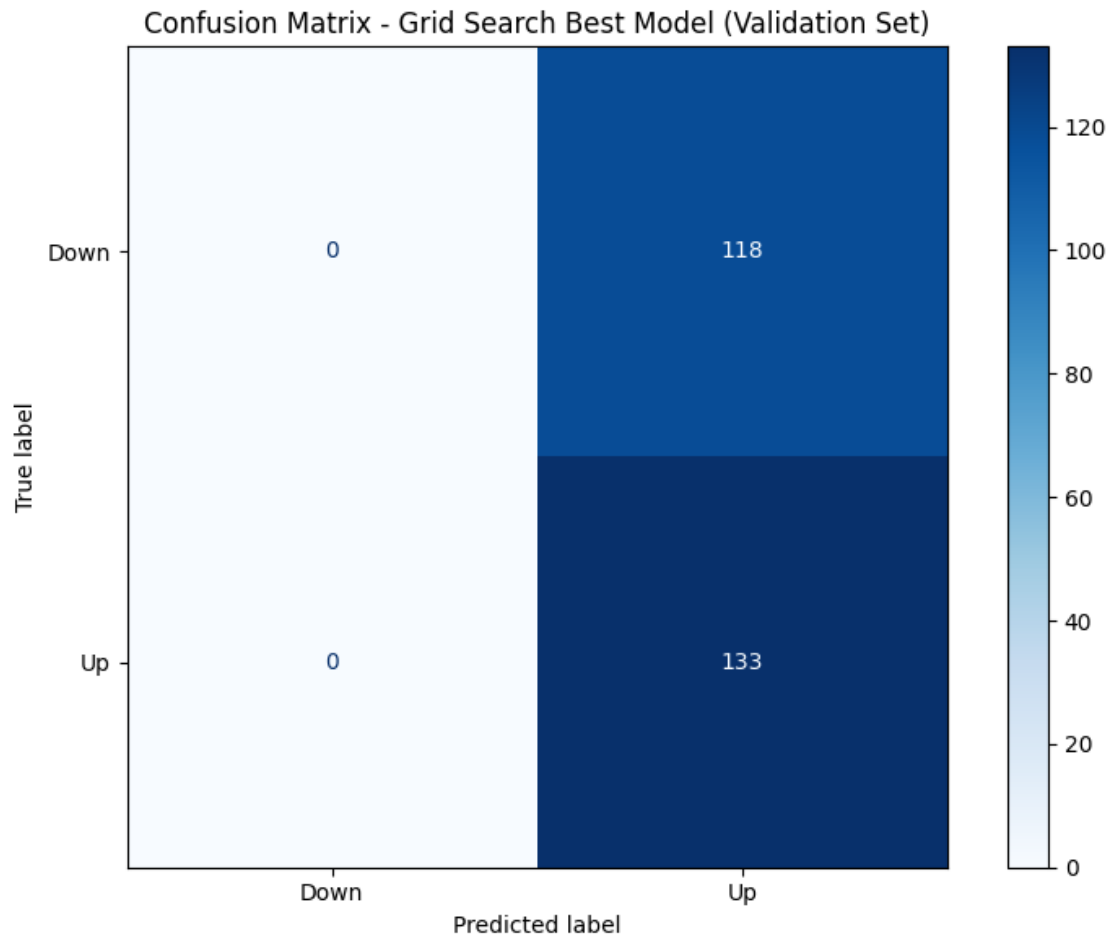
```

[78]: # Get the best model and make predictions on validation set
y_pred_val = best_lr_sentiment.predict(X_val_combined)

# Create and plot confusion matrix
cm = confusion_matrix(y_val_sentiment, y_pred_val)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Down', 'Up'])

fig, ax = plt.subplots(figsize=(8, 6))
disp.plot(ax=ax, cmap='Blues', values_format='d')
ax.set_title('Confusion Matrix - Grid Search Best Model (Validation Set)')
plt.tight_layout()
plt.show()

```



6.2 Price Dataset

First, a base logistic regression built with the training data from the price dataset.

```
[79]: def fit_model_price(model, train_dataset, val_dataset,
        target_var="Stock Price Direction", print_matrix=True):
        """Fit on train, evaluate on val."""

        train_X = train_dataset.drop(columns=[target_var], errors="ignore")
        train_y = train_dataset[target_var]
        val_X = val_dataset.drop(columns=[target_var], errors="ignore")
        val_y = val_dataset[target_var]

        #Drop datetime cols
        dt_cols = train_X.select_dtypes(include=["datetime64[ns]", "datetime64"]).
        ↪columns
        train_X = train_X.drop(columns=dt_cols)
```

```

val_X    = val_X.drop(columns=dt_cols, errors="ignore")

#Fit + predict
model.fit(train_X, train_y)
my_pred = model.predict(val_X)

print(f"Model: {model}")
print(CR(my_pred, val_y))

#Confusion matrix
cm = confusion_matrix(val_y, my_pred, labels=model.classes_)

if print_matrix:
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=model.
→classes_)
    fig, ax = plt.subplots(figsize=(12, 12))
    disp.plot(ax=ax, cmap="viridis", xticks_rotation=45)

    for row in disp.text_:
        for text in row:
            text.set_fontsize(20)

    plt.title("Confusion Matrix")
    plt.tight_layout()
    plt.show()

```

```

[80]: #Logistic Regression for the price dataset
price_lr=LR(max_iter=500, C=1, random_state=42) #Logistic Regression
fit_model_price(price_lr, cleaned_ts_train, cleaned_ts_val)

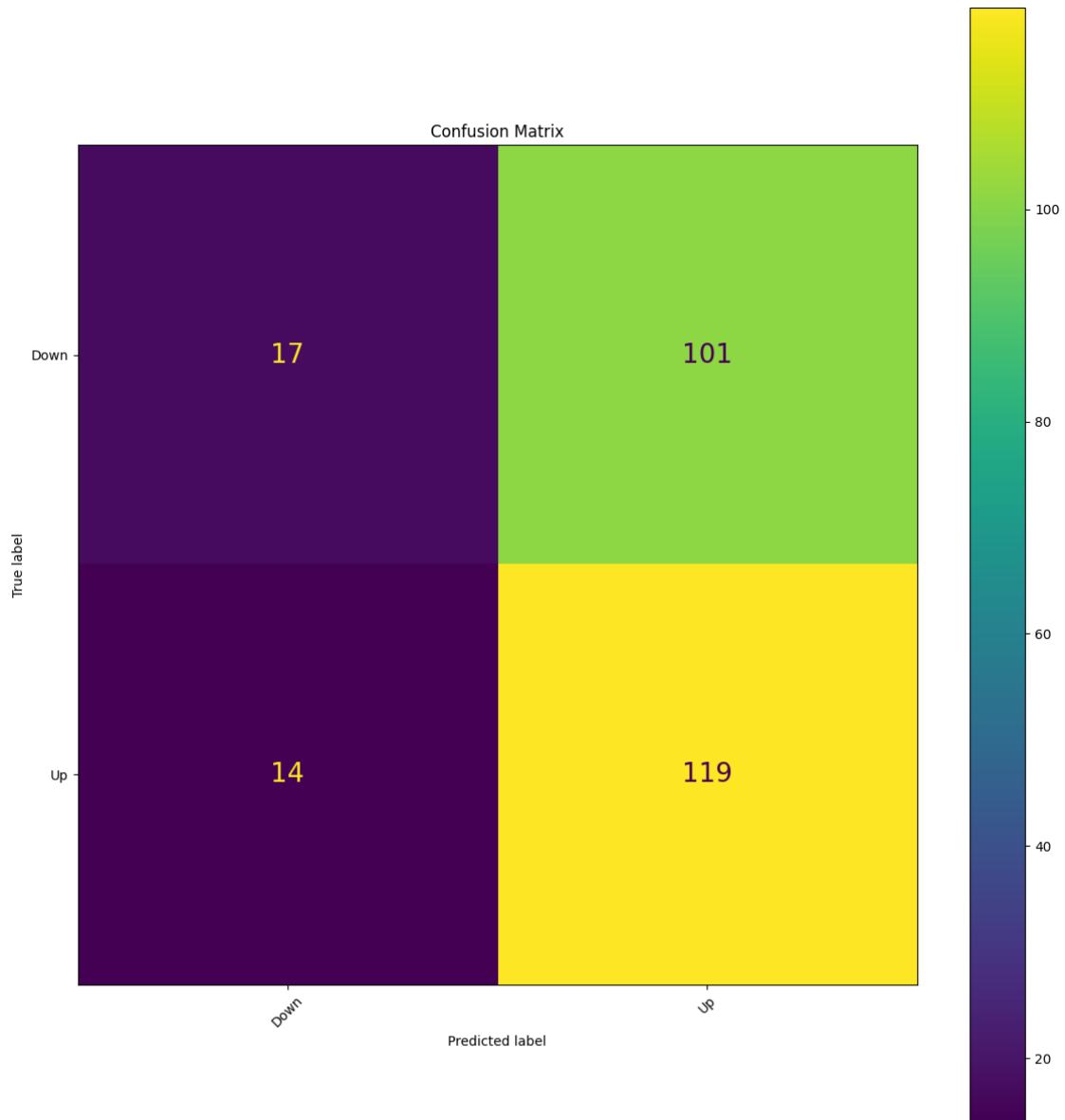
```

```

Model: LogisticRegression(C=1, max_iter=500, random_state=42)
      precision    recall  f1-score   support

```

Down	0.14	0.55	0.23	31
Up	0.89	0.54	0.67	220
accuracy			0.54	251
macro avg	0.52	0.54	0.45	251
weighted avg	0.80	0.54	0.62	251



Next, hyperparameters are specified to optimize the logistic regression's model performance.

```
[81]: #need to convert categorical target variable to numerical
if (cleaned_ts_train.loc[5, 'Stock Price Direction'] == 'Down') |
↳(cleaned_ts_train.loc[5, 'Stock Price Direction'] == 'Up'):
    cleaned_ts_train['Stock Price Direction'] = cleaned_ts_train['Stock Price_
↳Direction'].map({'Down': 0, 'Up': 1})
    cleaned_ts_val['Stock Price Direction'] = cleaned_ts_val['Stock Price_
↳Direction'].map({'Down': 0, 'Up': 1})
    cleaned_ts_test['Stock Price Direction'] = cleaned_ts_test['Stock Price_
↳Direction'].map({'Down': 0, 'Up': 1})
```

```

# Define the hyperparameter grid to search
param_grid = {
    'max_iter':[200, 500, 1000],    # number of iterations for model
    'C': [0.1, 1, 5]                # regularization strength
}

scoring = {
    'accuracy': make_scorer(accuracy_score),
    'f1': make_scorer(f1_score, pos_label=1)    # or pos_label=1 if encoded
}

# Create the GridSearchCV object
grid_search = GridSearchCV(
    estimator=price_lr,
    param_grid=param_grid,
    cv=5,                # 5-fold cross-validation
    scoring=scoring,    # print f1 and accuracy
    refit = 'f1',
    return_train_score=True,
    verbose=2
)

# Fit to training data
train_X = cleaned_com_train.drop(columns = ['Stock Price Direction'])
train_y = cleaned_com_train['Stock Price Direction']
grid_search.fit(train_X, train_y)

# Best hyperparameters
print("Best Hyperparameters:", grid_search.best_params_)

# Print performance metrics for cross-fold validation
results = pd.DataFrame(grid_search.cv_results_)
summary = results[['params', 'mean_test_accuracy', 'mean_test_f1',
    ↪ 'mean_train_accuracy', 'mean_train_f1']]
print(summary.sort_values(by='mean_test_accuracy', ascending=False))

# Best model
best_price_lr = grid_search.best_estimator_

price_lr_tuned = LR(max_iter = grid_search.best_params_['max_iter'],
    C = grid_search.best_params_['C'],
    random_state = 42
)

```



```
fit_model_price(price_lr_tuned, cleaned_ts_train, cleaned_ts_val)
```

Fitting 5 folds for each of 9 candidates, totalling 45 fits

```
[CV] END ...C=0.1, max_iter=200; total time= 0.1s
[CV] END ...C=0.1, max_iter=200; total time= 0.1s
[CV] END ...C=0.1, max_iter=200; total time= 0.0s
[CV] END ...C=0.1, max_iter=200; total time= 0.1s
[CV] END ...C=0.1, max_iter=200; total time= 0.0s
[CV] END ...C=0.1, max_iter=500; total time= 0.1s
[CV] END ...C=0.1, max_iter=500; total time= 0.0s
[CV] END ...C=0.1, max_iter=500; total time= 0.1s
[CV] END ...C=0.1, max_iter=500; total time= 0.0s
[CV] END ...C=0.1, max_iter=500; total time= 0.1s
[CV] END ...C=0.1, max_iter=1000; total time= 0.0s
[CV] END ...C=0.1, max_iter=1000; total time= 0.0s
[CV] END ...C=0.1, max_iter=1000; total time= 0.0s
[CV] END ...C=0.1, max_iter=1000; total time= 0.1s
[CV] END ...C=0.1, max_iter=1000; total time= 0.0s
[CV] END ...C=1, max_iter=200; total time= 0.1s
[CV] END ...C=1, max_iter=200; total time= 0.1s
[CV] END ...C=1, max_iter=200; total time= 0.0s
[CV] END ...C=1, max_iter=200; total time= 0.0s
[CV] END ...C=1, max_iter=200; total time= 0.0s
[CV] END ...C=1, max_iter=500; total time= 0.1s
[CV] END ...C=1, max_iter=500; total time= 0.0s
[CV] END ...C=1, max_iter=500; total time= 0.0s
[CV] END ...C=1, max_iter=500; total time= 0.1s
[CV] END ...C=1, max_iter=500; total time= 0.0s
[CV] END ...C=1, max_iter=1000; total time= 0.0s
[CV] END ...C=1, max_iter=1000; total time= 0.1s
[CV] END ...C=1, max_iter=1000; total time= 0.0s
[CV] END ...C=1, max_iter=1000; total time= 0.0s
[CV] END ...C=1, max_iter=1000; total time= 0.1s
[CV] END ...C=5, max_iter=200; total time= 0.0s
[CV] END ...C=5, max_iter=200; total time= 0.0s
[CV] END ...C=5, max_iter=200; total time= 0.1s
[CV] END ...C=5, max_iter=200; total time= 0.1s
[CV] END ...C=5, max_iter=200; total time= 0.1s
[CV] END ...C=5, max_iter=500; total time= 0.1s
[CV] END ...C=5, max_iter=500; total time= 0.0s
[CV] END ...C=5, max_iter=500; total time= 0.0s
[CV] END ...C=5, max_iter=500; total time= 0.1s
[CV] END ...C=5, max_iter=500; total time= 0.1s
[CV] END ...C=5, max_iter=1000; total time= 0.0s
[CV] END ...C=5, max_iter=1000; total time= 0.0s
[CV] END ...C=5, max_iter=1000; total time= 0.1s
[CV] END ...C=5, max_iter=1000; total time= 0.1s
```

[CV] END ...C=5, max_iter=1000; total time= 0.1s

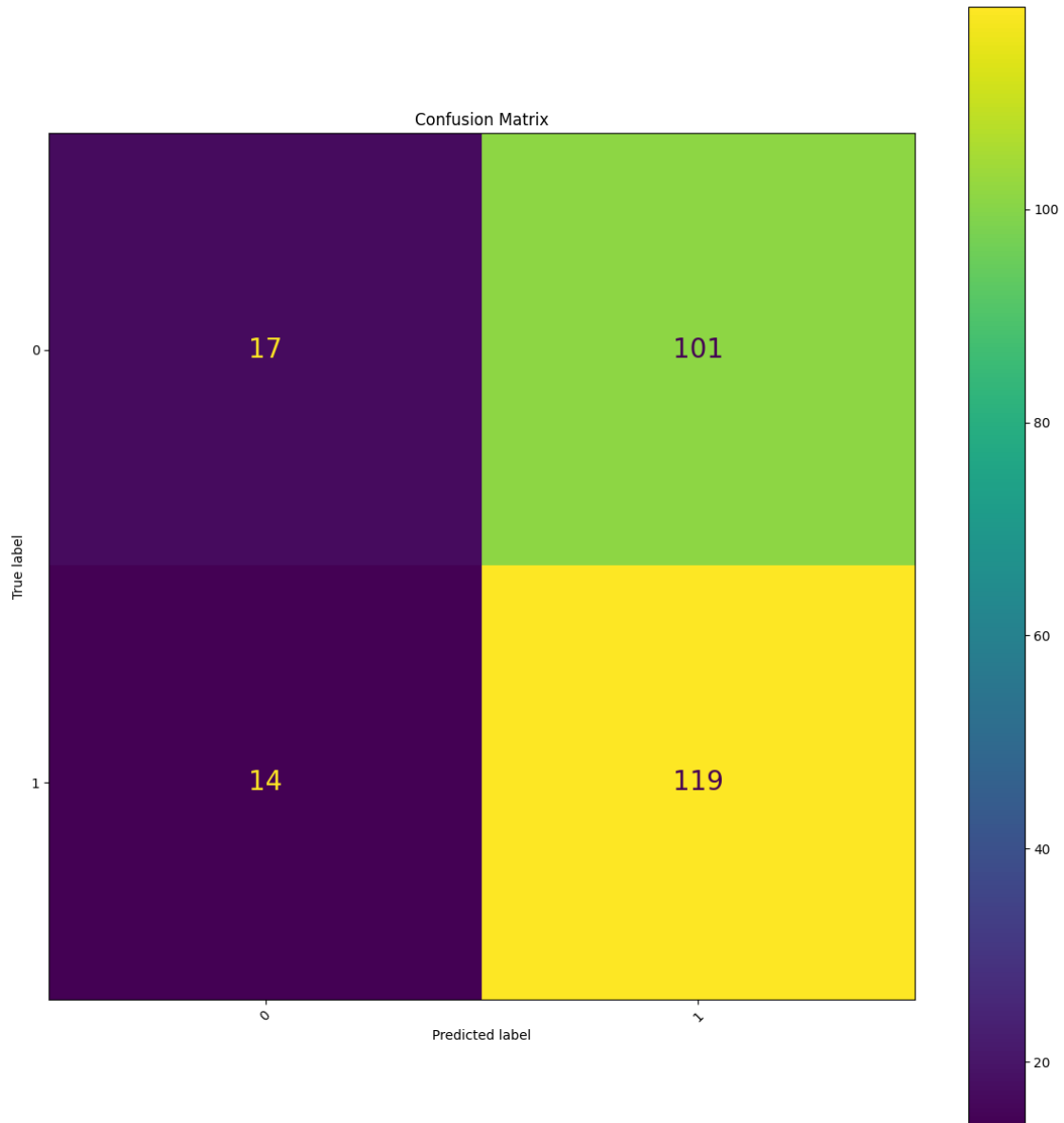
Best Hyperparameters: {'C': 0.1, 'max_iter': 200}

	params	mean_test_accuracy	mean_test_f1	\
7	{'C': 5, 'max_iter': 500}	0.509904	NaN	
6	{'C': 5, 'max_iter': 200}	0.509904	NaN	
8	{'C': 5, 'max_iter': 1000}	0.509904	NaN	
3	{'C': 1, 'max_iter': 200}	0.501636	NaN	
4	{'C': 1, 'max_iter': 500}	0.501636	NaN	
5	{'C': 1, 'max_iter': 1000}	0.501636	NaN	
0	{'C': 0.1, 'max_iter': 200}	0.501609	NaN	
1	{'C': 0.1, 'max_iter': 500}	0.501609	NaN	
2	{'C': 0.1, 'max_iter': 1000}	0.501609	NaN	

	mean_train_accuracy	mean_train_f1
7	0.551154	NaN
6	0.551154	NaN
8	0.551154	NaN
3	0.549709	NaN
4	0.549709	NaN
5	0.549709	NaN
0	0.541868	NaN
1	0.541868	NaN
2	0.541868	NaN

Model: LogisticRegression(C=0.1, max_iter=200, random_state=42)

	precision	recall	f1-score	support
0	0.14	0.55	0.23	31
1	0.89	0.54	0.67	220
accuracy			0.54	251
macro avg	0.52	0.54	0.45	251
weighted avg	0.80	0.54	0.62	251



According to the results, tuning does very little to improve the logistic regression's performance on the price dataset.

6.3 Combined Dataset

```
[82]: #Model Selection
def fit_model(model, train_dataset, val_dataset, target_var = 'Stock Price_
    ↳Direction', print_matrix = True):
    '''General function to fit model on training set, evaluate on validation_
    ↳set, and produce confusion matrix'''
```

```

train_X = train_dataset.drop(columns = [target_var])
train_y = train_dataset[target_var]
val_X = val_dataset.drop(columns = [target_var])
val_y = val_dataset[target_var]

#Fit model on training dataset
model.fit(train_X, train_y)

#Generate predictions and produce accuracy metrics
my_pred = model.predict(val_X)
print(f'Model: {model}')
print(CR(my_pred, val_y))

#generate confusion matrix
cm = confusion_matrix(val_y, my_pred, labels=model.classes_)

#print confusion matrix
if print_matrix:
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=model.
→classes_)

    fig, ax = plt.subplots(figsize=(12, 12)) # Adjust size for readability
    disp.plot(ax=ax, cmap='viridis', xticks_rotation=45)

    for row in disp.text_:
        for text in row:
            text.set_fontsize(20)

    plt.title("Confusion Matrix")
    plt.tight_layout()
    plt.show()

```

```

[83]: #Logistic Regression for the combined dataset
lr=LR(l1_ratio=0, max_iter=500, C=1, random_state=42) #Logistic Regression
fit_model(lr, cleaned_com_train, cleaned_com_val)

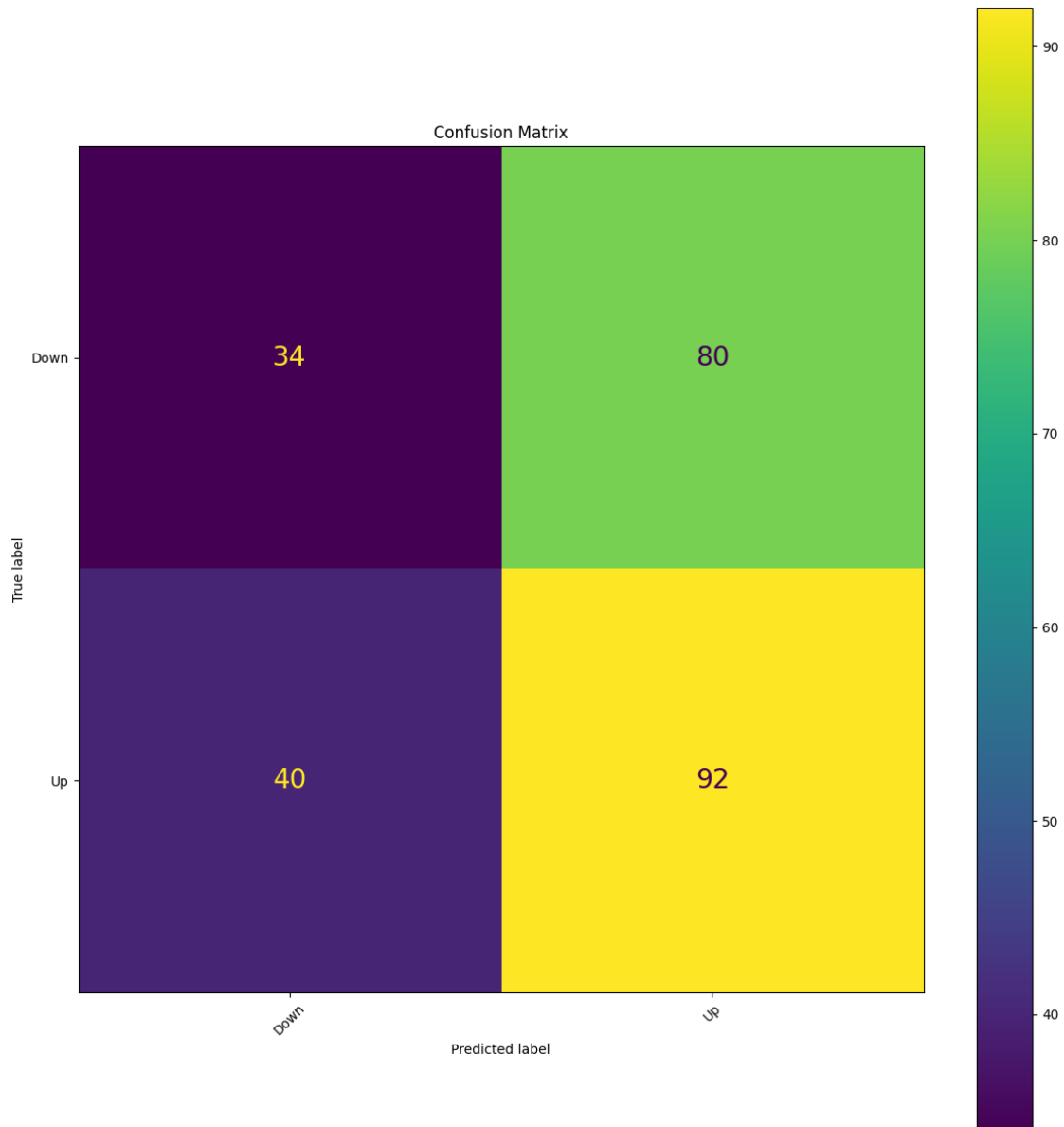
```

```

Model: LogisticRegression(C=1, l1_ratio=0, max_iter=500, random_state=42)
      precision    recall  f1-score   support

```

Down	0.30	0.46	0.36	74
Up	0.70	0.53	0.61	172
accuracy			0.51	246
macro avg	0.50	0.50	0.48	246
weighted avg	0.58	0.51	0.53	246



```
[84]: #need to convert categorical target variable to numerical
if (cleaned_com_train.loc[5, 'Stock Price Direction'] == 'Down') |
↳(cleaned_com_train.loc[5, 'Stock Price Direction'] == 'Up'):
    cleaned_com_train['Stock Price Direction'] = cleaned_com_train['Stock Price_
↳Direction'].map({'Down': 0, 'Up': 1})
    cleaned_com_val['Stock Price Direction'] = cleaned_com_val['Stock Price_
↳Direction'].map({'Down': 0, 'Up': 1})
    cleaned_com_test['Stock Price Direction'] = cleaned_com_test['Stock Price_
↳Direction'].map({'Down': 0, 'Up': 1})
```

```

# Define the hyperparameter grid to search
param_grid = {
    'max_iter':[200, 500, 1000],    # number of iterations for model
    'C': [0.1, 1, 5]                # regularization strength
}

scoring = {
    'accuracy': make_scorer(accuracy_score),
    'f1': make_scorer(f1_score, pos_label=1)    # or pos_label=1 if encoded
}

# Create the GridSearchCV object
grid_search = GridSearchCV(
    estimator=lr,
    param_grid=param_grid,
    cv=5,                # 5-fold cross-validation
    scoring=scoring,    # print f1 and accuracy
    refit = 'f1',
    return_train_score=True,
    verbose=2
)

# Fit to training data
train_X = cleaned_com_train.drop(columns = ['Stock Price Direction'])
train_y = cleaned_com_train['Stock Price Direction']
grid_search.fit(train_X, train_y)

# Best hyperparameters
print("Best Hyperparameters:", grid_search.best_params_)

# Print performance metrics for cross-fold validation
results = pd.DataFrame(grid_search.cv_results_)
summary = results[['params', 'mean_test_accuracy', 'mean_test_f1',
    ↪ 'mean_train_accuracy', 'mean_train_f1']]
print(summary.sort_values(by='mean_test_accuracy', ascending=False))

# Best model
best_lr = grid_search.best_estimator_

lr2 = LR(max_iter = grid_search.best_params_['max_iter'],
        C = grid_search.best_params_['C'],
        random_state = 42
    )

```

```
fit_model(lr2, cleaned_com_train, cleaned_com_val)
```

Fitting 5 folds for each of 9 candidates, totalling 45 fits

```
[CV] END ...C=0.1, max_iter=200; total time= 0.0s
[CV] END ...C=0.1, max_iter=200; total time= 0.0s
[CV] END ...C=0.1, max_iter=200; total time= 0.0s
[CV] END ...C=0.1, max_iter=200; total time= 0.0s
[CV] END ...C=0.1, max_iter=200; total time= 0.0s
[CV] END ...C=0.1, max_iter=500; total time= 0.0s
[CV] END ...C=0.1, max_iter=500; total time= 0.0s
[CV] END ...C=0.1, max_iter=500; total time= 0.0s
[CV] END ...C=0.1, max_iter=500; total time= 0.0s
[CV] END ...C=0.1, max_iter=500; total time= 0.0s
[CV] END ...C=0.1, max_iter=1000; total time= 0.1s
[CV] END ...C=0.1, max_iter=1000; total time= 0.0s
[CV] END ...C=0.1, max_iter=1000; total time= 0.1s
[CV] END ...C=0.1, max_iter=1000; total time= 0.0s
[CV] END ...C=0.1, max_iter=1000; total time= 0.0s
[CV] END ...C=1, max_iter=200; total time= 0.1s
[CV] END ...C=1, max_iter=200; total time= 0.0s
[CV] END ...C=1, max_iter=200; total time= 0.1s
[CV] END ...C=1, max_iter=200; total time= 0.1s
[CV] END ...C=1, max_iter=200; total time= 0.0s
[CV] END ...C=1, max_iter=500; total time= 0.0s
[CV] END ...C=1, max_iter=500; total time= 0.1s
[CV] END ...C=1, max_iter=500; total time= 0.1s
[CV] END ...C=1, max_iter=500; total time= 0.0s
[CV] END ...C=1, max_iter=500; total time= 0.1s
[CV] END ...C=1, max_iter=1000; total time= 0.0s
[CV] END ...C=1, max_iter=1000; total time= 0.0s
[CV] END ...C=1, max_iter=1000; total time= 0.0s
[CV] END ...C=1, max_iter=1000; total time= 0.0s
[CV] END ...C=1, max_iter=1000; total time= 0.1s
[CV] END ...C=5, max_iter=200; total time= 0.0s
[CV] END ...C=5, max_iter=200; total time= 0.0s
[CV] END ...C=5, max_iter=200; total time= 0.1s
[CV] END ...C=5, max_iter=200; total time= 0.1s
[CV] END ...C=5, max_iter=200; total time= 0.0s
[CV] END ...C=5, max_iter=500; total time= 0.0s
[CV] END ...C=5, max_iter=500; total time= 0.1s
[CV] END ...C=5, max_iter=500; total time= 0.0s
[CV] END ...C=5, max_iter=500; total time= 0.1s
[CV] END ...C=5, max_iter=500; total time= 0.0s
[CV] END ...C=5, max_iter=1000; total time= 0.0s
[CV] END ...C=5, max_iter=1000; total time= 0.1s
[CV] END ...C=5, max_iter=1000; total time= 0.0s
[CV] END ...C=5, max_iter=1000; total time= 0.0s
[CV] END ...C=5, max_iter=1000; total time= 0.1s
```

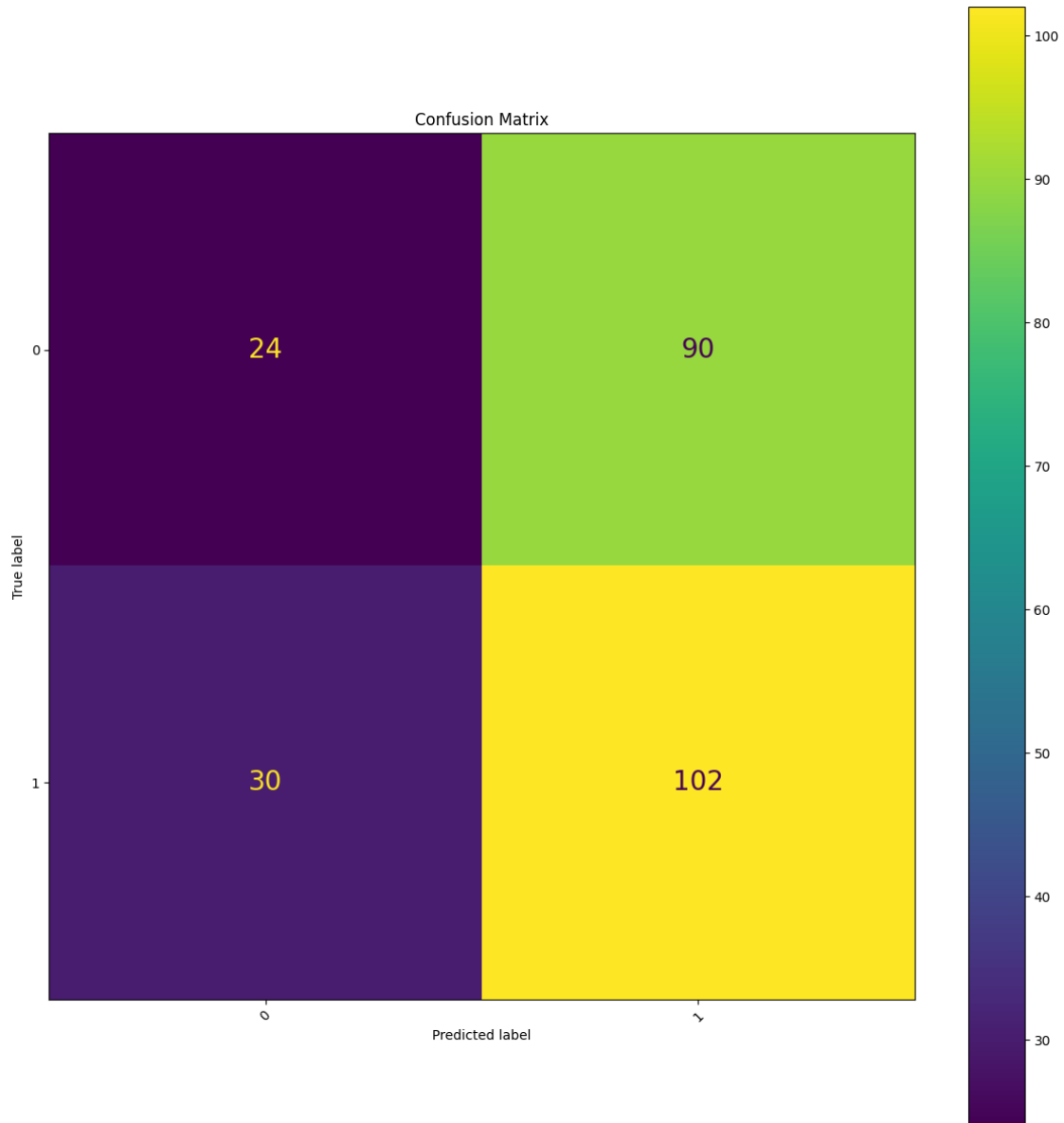
Best Hyperparameters: {'C': 0.1, 'max_iter': 200}

	params	mean_test_accuracy	mean_test_f1	\
7	{'C': 5, 'max_iter': 500}	0.509904	0.602367	
6	{'C': 5, 'max_iter': 200}	0.509904	0.602367	
8	{'C': 5, 'max_iter': 1000}	0.509904	0.602367	
3	{'C': 1, 'max_iter': 200}	0.501636	0.607691	
4	{'C': 1, 'max_iter': 500}	0.501636	0.607691	
5	{'C': 1, 'max_iter': 1000}	0.501636	0.607691	
0	{'C': 0.1, 'max_iter': 200}	0.501609	0.622883	
1	{'C': 0.1, 'max_iter': 500}	0.501609	0.622883	
2	{'C': 0.1, 'max_iter': 1000}	0.501609	0.622883	

	mean_train_accuracy	mean_train_f1
7	0.551154	0.642432
6	0.551154	0.642432
8	0.551154	0.642432
3	0.549709	0.644149
4	0.549709	0.644149
5	0.549709	0.644149
0	0.541868	0.653784
1	0.541868	0.653784
2	0.541868	0.653784

Model: LogisticRegression(C=0.1, max_iter=200, random_state=42)

	precision	recall	f1-score	support
0	0.21	0.44	0.29	54
1	0.77	0.53	0.63	192
accuracy			0.51	246
macro avg	0.49	0.49	0.46	246
weighted avg	0.65	0.51	0.55	246



This model is performing slightly better than 50/50 with 51% accuracy. Our goal at the beginning of the project is 52% accuracy, so this is slightly below. F1 score on the “up” stock price direction is better than “down”, this is likely because there were more up days in the training dataset. Precision was much better on the “up” direction than down, and recall was slightly higher for “up”.

```
[85]: cleaned_com_train.head(5)
```

```
[85]:   Stock Price Direction  sentiment_polarity_lag1  sentiment_polarity_lag2  \
5                0          0.0          0.0
6                0          0.0          0.0
7                1          0.0          0.0
```

8	1	0.0	0.0
9	0	0.0	0.0

	sentiment_polarity_lag3	sentiment_polarity_lag4	sentiment_polarity_lag5	\
5	0.0	0.0	0.0	
6	0.0	0.0	0.0	
7	0.0	0.0	0.0	
8	0.0	0.0	0.0	
9	0.0	0.0	0.0	

	sentiment_pos_lag1	sentiment_pos_lag2	sentiment_pos_lag3	\
5	0.0	0.0	0.0	
6	0.0	0.0	0.0	
7	0.0	0.0	0.0	
8	0.0	0.0	0.0	
9	0.0	0.0	0.0	

	sentiment_pos_lag4	...	sentiment_neu_lag5	sentiment_neg_lag1	\
5	0.0	...	0.0	0.0	
6	0.0	...	0.0	0.0	
7	0.0	...	0.0	0.0	
8	0.0	...	0.0	0.0	
9	0.0	...	0.0	0.0	

	sentiment_neg_lag2	sentiment_neg_lag3	sentiment_neg_lag4	\
5	0.0	0.0	0.0	
6	0.0	0.0	0.0	
7	0.0	0.0	0.0	
8	0.0	0.0	0.0	
9	0.0	0.0	0.0	

	sentiment_neg_lag5	aapl_price_direction_lag1	aapl_price_direction_lag2	\
5	0.0	0	1	
6	0.0	0	0	
7	0.0	0	0	
8	0.0	1	0	
9	0.0	1	1	

	aapl_price_direction_lag3	aapl_price_direction_lag4
5	1	0
6	1	1
7	0	1
8	0	0
9	0	0

[5 rows x 25 columns]

7 Week 7 - Support Vector Machine (SVM)

7.1 Sentiment Dataset

```
[86]: # Base SVM for Sentiment Dataset
svm_sentiment = SVC(kernel='linear', C=1, random_state=42)
svm_sentiment.fit(X_train_combined, y_train_sentiment)

# Make predictions on validation set
y_pred_svm = svm_sentiment.predict(X_val_combined)

# Accuracy
accuracy = accuracy_score(y_val_sentiment, y_pred_svm)
print(f"\nAccuracy: {accuracy:.4f}")

# Classification Report (Precision, Recall, F1-Score, Support)
print("\nClassification Report:")
print(CR(y_val_sentiment, y_pred_svm, target_names=['Down (0)', 'Up (1)']))

# Confusion Matrix
print("\nConfusion Matrix:")
cm = confusion_matrix(y_val_sentiment, y_pred_svm, labels=svm_sentiment.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Down', 'Up'])

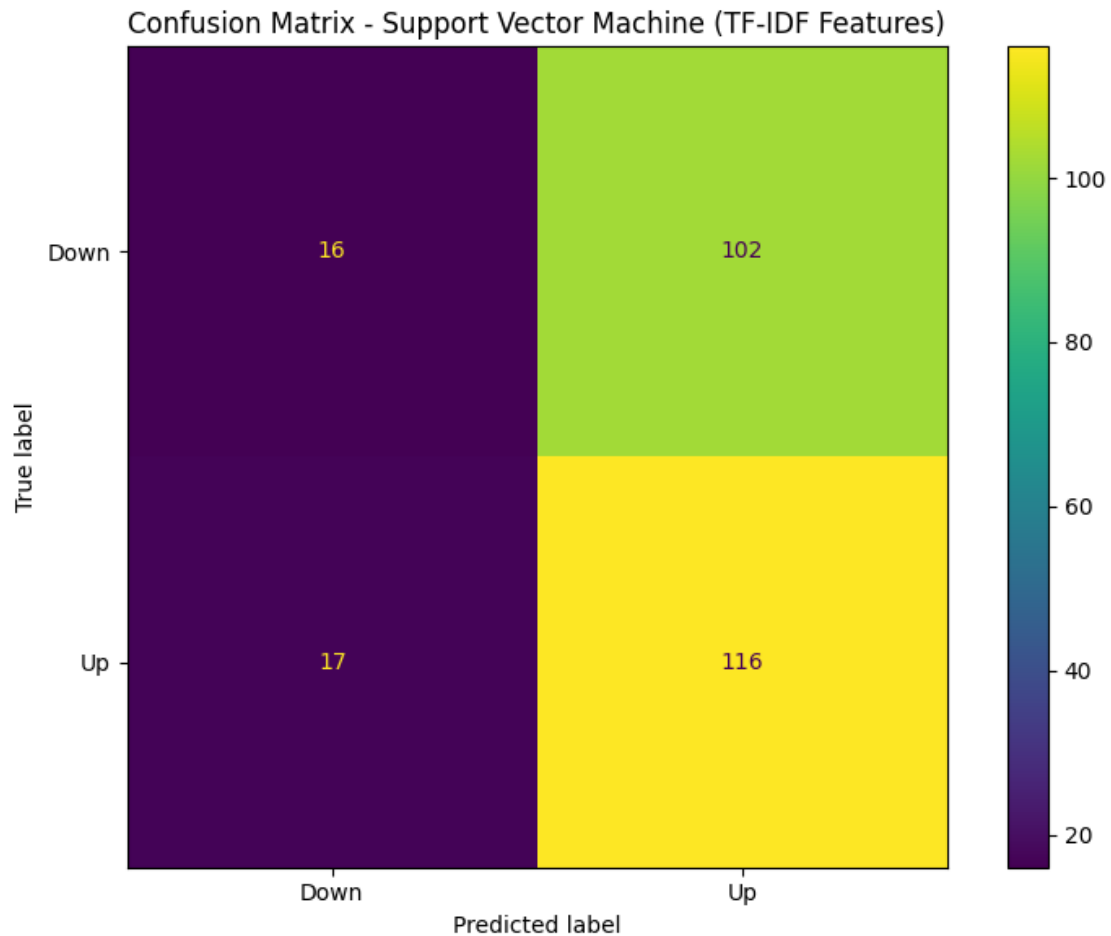
fig, ax = plt.subplots(figsize=(8, 6))
disp.plot(ax=ax, cmap='viridis')
plt.title("Confusion Matrix - Support Vector Machine (TF-IDF Features)")
plt.tight_layout()
plt.show()
```

Accuracy: 0.5259

Classification Report:

	precision	recall	f1-score	support
Down (0)	0.48	0.14	0.21	118
Up (1)	0.53	0.87	0.66	133
accuracy			0.53	251
macro avg	0.51	0.50	0.44	251
weighted avg	0.51	0.53	0.45	251

Confusion Matrix:



```
[87]: # Define the hyperparameter grid for SVM
param_grid_svm = {
    'C': [0.01, 0.1, 1, 10],          # Regularization strength
    'kernel': ['linear', 'rbf'],      # Kernel type
    'gamma': ['scale', 'auto']        # Kernel coefficient for 'rbf'
}

# Define scoring metrics for GridSearchCV
scoring_svm = {
    'accuracy': make_scorer(accuracy_score),
    'f1':       make_scorer(f1_score, pos_label=1)
}

# Initialize the SVM model
svm_model = SVC(max_iter=5000, random_state=42)

# Set up GridSearchCV with cross-validation
```

```

grid_search_svm = GridSearchCV(
    estimator=svm_model,
    param_grid=param_grid_svm,
    cv=5,                                # 5-fold cross-validation
    scoring=scoring_svm,
    refit='f1',                          # Refit using F1 score
)

# Fit grid search on training data
print("Starting Grid Search for Support Vector Machine on Sentiment TF-IDF_
↳Features...")
grid_search_svm.fit(X_train_combined, y_train_sentiment)

```

Starting Grid Search for Support Vector Machine on Sentiment TF-IDF Features...

```

[87]: GridSearchCV(cv=5, estimator=SVC(max_iter=5000, random_state=42),
    param_grid={'C': [0.01, 0.1, 1, 10], 'gamma': ['scale', 'auto'],
    'kernel': ['linear', 'rbf']},
    refit='f1',
    scoring={'accuracy': make_scorer(accuracy_score,
    response_method='predict'),
    'f1': make_scorer(f1_score, response_method='predict',
    pos_label=1)})

```

```

[88]: # Display best hyperparameters and cross-validation results
print("Best Hyperparameters:", grid_search_svm.best_params_)

# Print performance metrics for cross-fold validation
results_df = pd.DataFrame(grid_search_svm.cv_results_)
summary = results_df[['params', 'mean_test_accuracy', 'mean_test_f1']]
print(summary.sort_values(by='mean_test_f1', ascending=False))

# Get the best estimator from grid search
best_sentiment_svm = grid_search_svm.best_estimator_

# Evaluate best model on validation set
y_pred_val = best_sentiment_svm.predict(X_val_combined)

print("\n--- Validation Set Performance ---")
print(f"Accuracy: {accuracy_score(y_val_sentiment, y_pred_val):.4f}")
print("\nClassification Report:")
print(CR(y_val_sentiment, y_pred_val, target_names=['Down (0)', 'Up (1)']))

# Confusion Matrix
cm = confusion_matrix(y_val_sentiment, y_pred_val, labels=best_sentiment_svm.
↳classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Down', 'Up'])

```

```
fig, ax = plt.subplots(figsize=(8, 6))
disp.plot(ax=ax, cmap='viridis')
plt.title("Confusion Matrix - Tuned SVM (Sentiment TF-IDF)")
plt.tight_layout()
plt.show()
```

Best Hyperparameters: {'C': 0.01, 'gamma': 'scale', 'kernel': 'linear'}

	params	mean_test_accuracy \
0	{'C': 0.01, 'gamma': 'scale', 'kernel': 'linear'}	0.528348
1	{'C': 0.01, 'gamma': 'scale', 'kernel': 'rbf'}	0.528348
2	{'C': 0.01, 'gamma': 'auto', 'kernel': 'linear'}	0.528348
3	{'C': 0.01, 'gamma': 'auto', 'kernel': 'rbf'}	0.528348
4	{'C': 0.1, 'gamma': 'scale', 'kernel': 'linear'}	0.528348
5	{'C': 0.1, 'gamma': 'scale', 'kernel': 'rbf'}	0.528348
6	{'C': 0.1, 'gamma': 'auto', 'kernel': 'linear'}	0.528348
7	{'C': 0.1, 'gamma': 'auto', 'kernel': 'rbf'}	0.528348
11	{'C': 1, 'gamma': 'auto', 'kernel': 'rbf'}	0.528348
15	{'C': 10, 'gamma': 'auto', 'kernel': 'rbf'}	0.528348
9	{'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}	0.511064
12	{'C': 10, 'gamma': 'scale', 'kernel': 'linear'}	0.511064
14	{'C': 10, 'gamma': 'auto', 'kernel': 'linear'}	0.511064
8	{'C': 1, 'gamma': 'scale', 'kernel': 'linear'}	0.508595
10	{'C': 1, 'gamma': 'auto', 'kernel': 'linear'}	0.508595
13	{'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}	0.508581

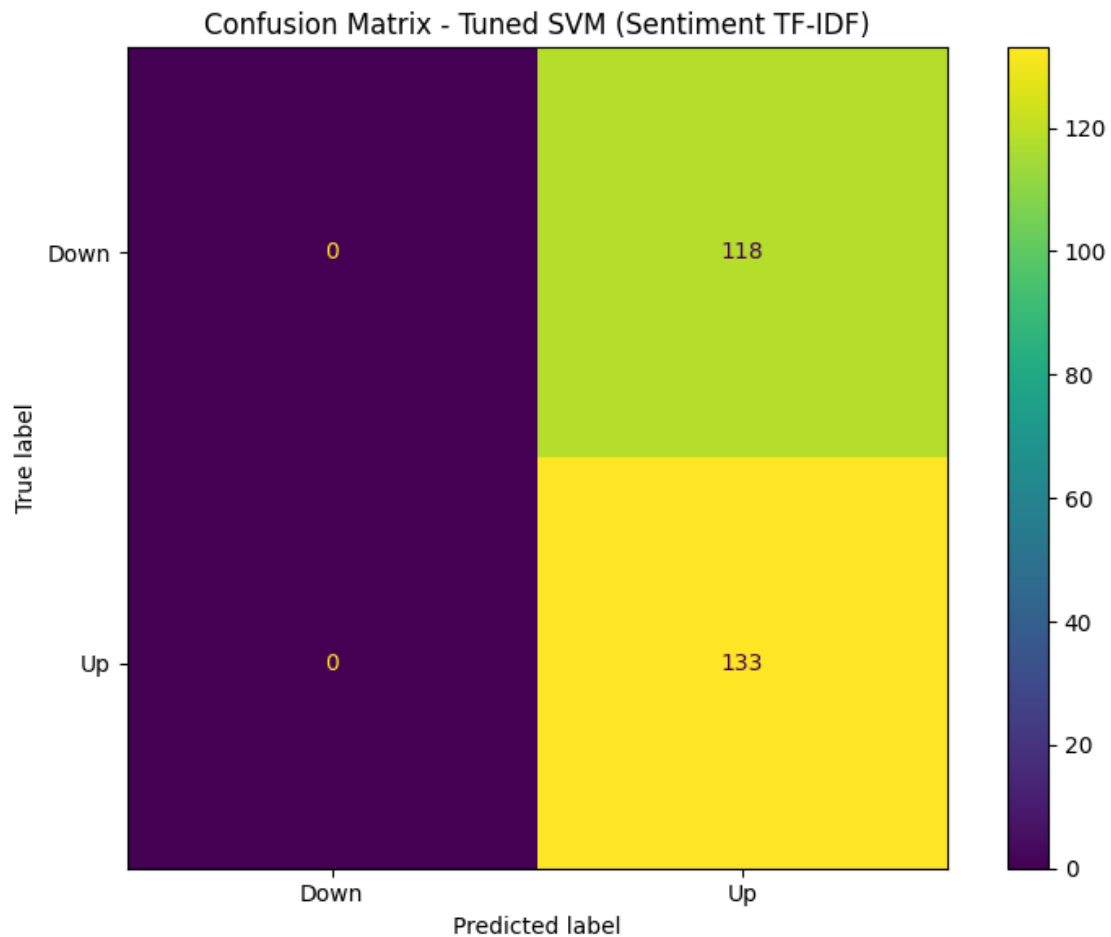
	mean_test_f1
0	0.691396
1	0.691396
2	0.691396
3	0.691396
4	0.691396
5	0.691396
6	0.691396
7	0.691396
11	0.691396
15	0.691396
9	0.562857
12	0.556281
14	0.556281
8	0.552021
10	0.552021
13	0.550541

--- Validation Set Performance ---
Accuracy: 0.5299

Classification Report:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Down (0)	0.00	0.00	0.00	118
Up (1)	0.53	1.00	0.69	133
accuracy			0.53	251
macro avg	0.26	0.50	0.35	251
weighted avg	0.28	0.53	0.37	251



7.2 Price Dataset

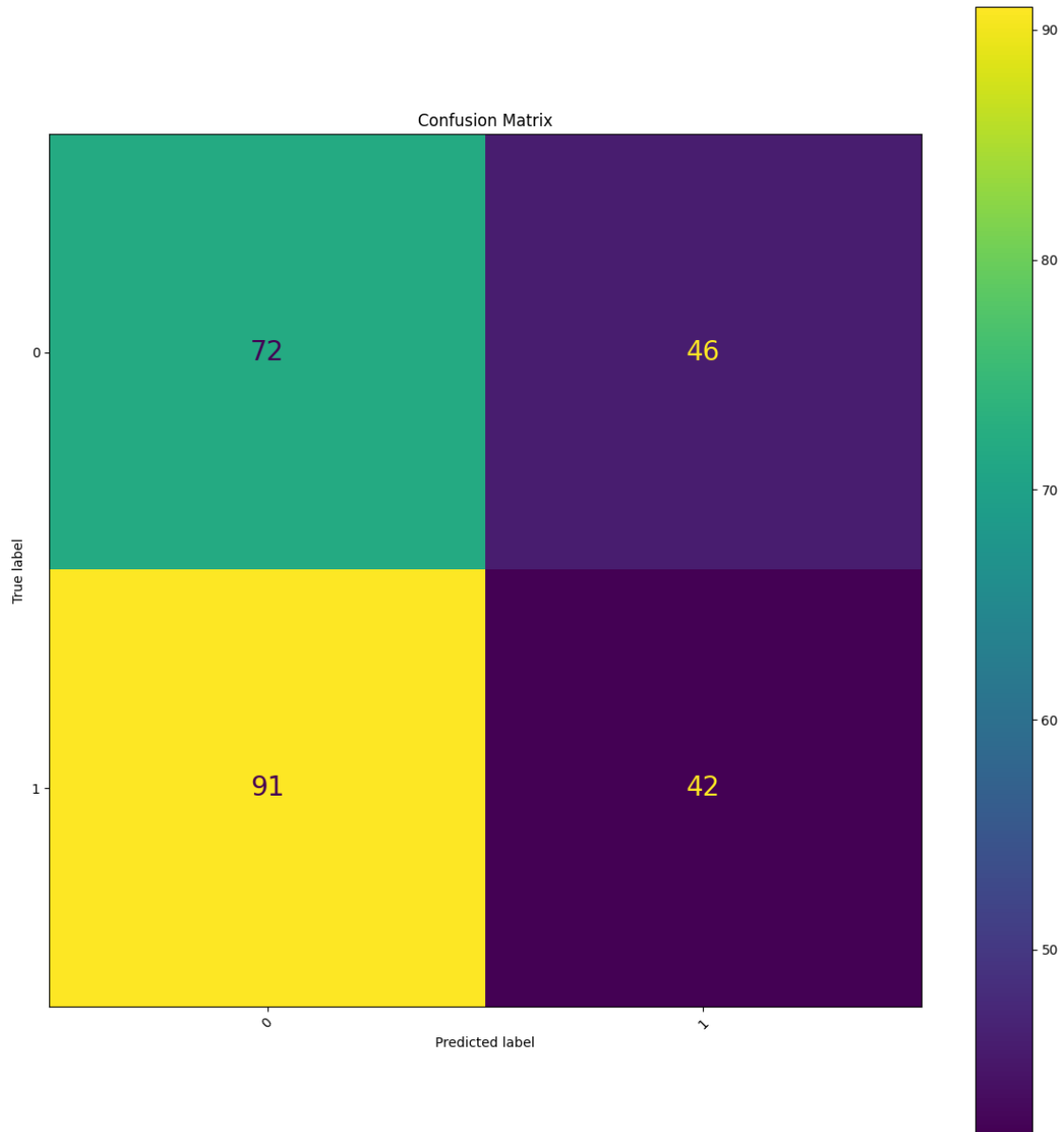
Baseline SVM model with RBF kernel.

```
[89]: # Support Vector Machine for Price Prediction
price_svm = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(kernel='rbf', class_weight='balanced', random_state=42))
])
```

```
fit_model_price(price_svm, cleaned_ts_train, cleaned_ts_val)
```

```
Model: Pipeline(steps=[('scaler', StandardScaler()),  
                        ('svm', SVC(class_weight='balanced', random_state=42))])
```

	precision	recall	f1-score	support
0	0.61	0.44	0.51	163
1	0.32	0.48	0.38	88
accuracy			0.45	251
macro avg	0.46	0.46	0.45	251
weighted avg	0.51	0.45	0.47	251



Grid search to tune hyperparameters and find optimal combination.

```
[90]: # Convert categorical target variable to numerical
if (cleaned_ts_train.loc[5, 'Stock Price Direction'] == 'Down') |
    (cleaned_ts_train.loc[5, 'Stock Price Direction'] == 'Up'):
    cleaned_ts_train['Stock Price Direction'] = cleaned_ts_train['Stock Price_
    Direction'].map({'Down': 0, 'Up': 1})
    cleaned_ts_val['Stock Price Direction'] = cleaned_ts_val['Stock Price_
    Direction'].map({'Down': 0, 'Up': 1})
```

```

        cleaned_ts_test['Stock Price Direction'] = cleaned_ts_test['Stock Price_
↪Direction'].map({'Down': 0, 'Up': 1})

# Define the hyperparameter grid for SVM
param_grid = {
    'svm__C':      [0.1, 1, 10, 100],
    'svm__kernel': ['linear', 'rbf'],
    'svm__gamma':  ['scale', 'auto']
}

# Define scoring metrics for GridSearchCV
scoring = {
    'accuracy': make_scorer(accuracy_score),
    'f1':       make_scorer(f1_score, pos_label=1)
}

# Create a pipeline for SVM with scaling
price_svm = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(class_weight='balanced', random_state=42))
])

# Set up GridSearchCV for hyperparameter tuning
grid_search = GridSearchCV(
    estimator=price_svm,
    param_grid=param_grid,
    cv=5,
    scoring=scoring,
    refit='f1',
    return_train_score=True,
    verbose=2
)

# Fit the grid search on the training data
price_train_X = cleaned_ts_train.drop(columns=['Stock Price Direction', 'Dates'])
price_train_y = cleaned_ts_train['Stock Price Direction']
grid_search.fit(price_train_X, price_train_y)

# Display best hyperparameters and cross-validation results
print("Best Hyperparameters:", grid_search.best_params_)

# Print performance metrics for cross-fold validation
results = pd.DataFrame(grid_search.cv_results_)
summary = results[['params', 'mean_test_accuracy', 'mean_test_f1',
↪'mean_train_accuracy', 'mean_train_f1']]
print(summary.sort_values(by='mean_test_f1', ascending=False))

```

```

best_price_svm = grid_search.best_estimator_

price_svm_tuned = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(
        C=grid_search.best_params_['svm__C'],
        kernel=grid_search.best_params_['svm__kernel'],
        gamma=grid_search.best_params_['svm__gamma'],
        class_weight='balanced',
        random_state=42
    ))
])

fit_model_price(price_svm_tuned, cleaned_ts_train, cleaned_ts_val)

```

Fitting 5 folds for each of 16 candidates, totalling 80 fits

```

[CV] END ...svm__C=0.1, svm__gamma=scale, svm__kernel=linear; total time= 0.0s
[CV] END ...svm__C=0.1, svm__gamma=scale, svm__kernel=linear; total time= 0.0s
[CV] END ...svm__C=0.1, svm__gamma=scale, svm__kernel=linear; total time= 0.0s
[CV] END ...svm__C=0.1, svm__gamma=scale, svm__kernel=linear; total time= 0.0s
[CV] END ...svm__C=0.1, svm__gamma=scale, svm__kernel=linear; total time= 0.0s
[CV] END ...svm__C=0.1, svm__gamma=scale, svm__kernel=rbf; total time= 0.1s
[CV] END ...svm__C=0.1, svm__gamma=scale, svm__kernel=rbf; total time= 0.1s
[CV] END ...svm__C=0.1, svm__gamma=scale, svm__kernel=rbf; total time= 0.1s
[CV] END ...svm__C=0.1, svm__gamma=scale, svm__kernel=rbf; total time= 0.1s
[CV] END ...svm__C=0.1, svm__gamma=scale, svm__kernel=rbf; total time= 0.1s
[CV] END ...svm__C=0.1, svm__gamma=auto, svm__kernel=linear; total time= 0.1s
[CV] END ...svm__C=0.1, svm__gamma=auto, svm__kernel=linear; total time= 0.0s
[CV] END ...svm__C=0.1, svm__gamma=auto, svm__kernel=linear; total time= 0.0s
[CV] END ...svm__C=0.1, svm__gamma=auto, svm__kernel=linear; total time= 0.0s
[CV] END ...svm__C=0.1, svm__gamma=auto, svm__kernel=linear; total time= 0.0s
[CV] END ...svm__C=0.1, svm__gamma=auto, svm__kernel=rbf; total time= 0.1s
[CV] END ...svm__C=0.1, svm__gamma=auto, svm__kernel=rbf; total time= 0.1s
[CV] END ...svm__C=0.1, svm__gamma=auto, svm__kernel=rbf; total time= 0.1s
[CV] END ...svm__C=0.1, svm__gamma=auto, svm__kernel=rbf; total time= 0.1s
[CV] END ...svm__C=0.1, svm__gamma=auto, svm__kernel=rbf; total time= 0.1s
[CV] END ...svm__C=1, svm__gamma=scale, svm__kernel=linear; total time= 0.0s
[CV] END ...svm__C=1, svm__gamma=scale, svm__kernel=linear; total time= 0.0s
[CV] END ...svm__C=1, svm__gamma=scale, svm__kernel=linear; total time= 0.0s
[CV] END ...svm__C=1, svm__gamma=scale, svm__kernel=linear; total time= 0.1s
[CV] END ...svm__C=1, svm__gamma=scale, svm__kernel=linear; total time= 0.0s
[CV] END ...svm__C=1, svm__gamma=scale, svm__kernel=rbf; total time= 0.1s
[CV] END ...svm__C=1, svm__gamma=scale, svm__kernel=rbf; total time= 0.1s
[CV] END ...svm__C=1, svm__gamma=scale, svm__kernel=rbf; total time= 0.1s
[CV] END ...svm__C=1, svm__gamma=scale, svm__kernel=rbf; total time= 0.1s
[CV] END ...svm__C=1, svm__gamma=scale, svm__kernel=rbf; total time= 0.1s
[CV] END ...svm__C=1, svm__gamma=auto, svm__kernel=linear; total time= 0.0s
[CV] END ...svm__C=1, svm__gamma=auto, svm__kernel=linear; total time= 0.0s

```


Best Hyperparameters: {'svm__C': 0.1, 'svm__gamma': 'scale', 'svm__kernel': 'linear'}

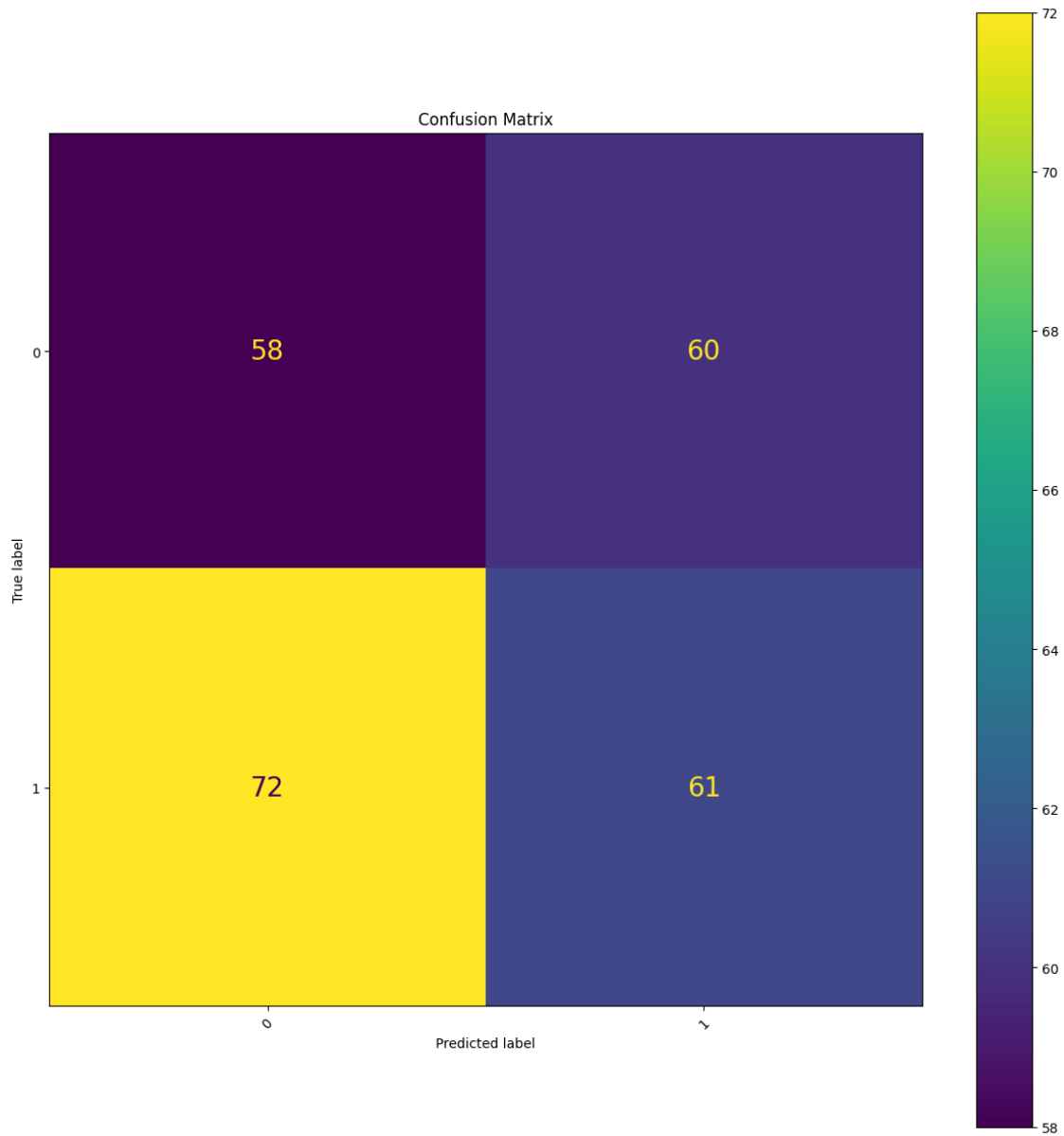
	params	mean_test_accuracy \
0	{'svm__C': 0.1, 'svm__gamma': 'scale', 'svm__k...	0.485620
2	{'svm__C': 0.1, 'svm__gamma': 'auto', 'svm__ke...	0.485620
6	{'svm__C': 1, 'svm__gamma': 'auto', 'svm__kern...	0.485620
4	{'svm__C': 1, 'svm__gamma': 'scale', 'svm__ker...	0.485620
12	{'svm__C': 100, 'svm__gamma': 'scale', 'svm__k...	0.485620
14	{'svm__C': 100, 'svm__gamma': 'auto', 'svm__ke...	0.485620
10	{'svm__C': 10, 'svm__gamma': 'auto', 'svm__ker...	0.485620
8	{'svm__C': 10, 'svm__gamma': 'scale', 'svm__ke...	0.485620
11	{'svm__C': 10, 'svm__gamma': 'auto', 'svm__ker...	0.512737
9	{'svm__C': 10, 'svm__gamma': 'scale', 'svm__ke...	0.512737
13	{'svm__C': 100, 'svm__gamma': 'scale', 'svm__k...	0.512737
15	{'svm__C': 100, 'svm__gamma': 'auto', 'svm__ke...	0.512737
7	{'svm__C': 1, 'svm__gamma': 'auto', 'svm__kern...	0.510248
5	{'svm__C': 1, 'svm__gamma': 'scale', 'svm__ker...	0.510248
1	{'svm__C': 0.1, 'svm__gamma': 'scale', 'svm__k...	0.488056
3	{'svm__C': 0.1, 'svm__gamma': 'auto', 'svm__ke...	0.488056

	mean_test_f1	mean_train_accuracy	mean_train_f1
0	0.485975	0.520747	0.520842
2	0.485975	0.520747	0.520842
6	0.485975	0.520747	0.520842
4	0.485975	0.520747	0.520842
12	0.485975	0.520747	0.520842
14	0.485975	0.520747	0.520842
10	0.485975	0.520747	0.520842
8	0.485975	0.520747	0.520842
11	0.480362	0.549710	0.521211
9	0.480362	0.549710	0.521211
13	0.480362	0.549710	0.521211
15	0.480362	0.549710	0.521211
7	0.431830	0.544574	0.472517
5	0.431830	0.544574	0.472517
1	0.348732	0.530606	0.401055
3	0.348732	0.530606	0.401055

Model: Pipeline(steps=[('scaler', StandardScaler()),
('svm',
SVC(C=0.1, class_weight='balanced', kernel='linear',
random_state=42))])

	precision	recall	f1-score	support
0	0.49	0.45	0.47	130
1	0.46	0.50	0.48	121
accuracy			0.47	251
macro avg	0.48	0.48	0.47	251

weighted avg 0.48 0.47 0.47 251



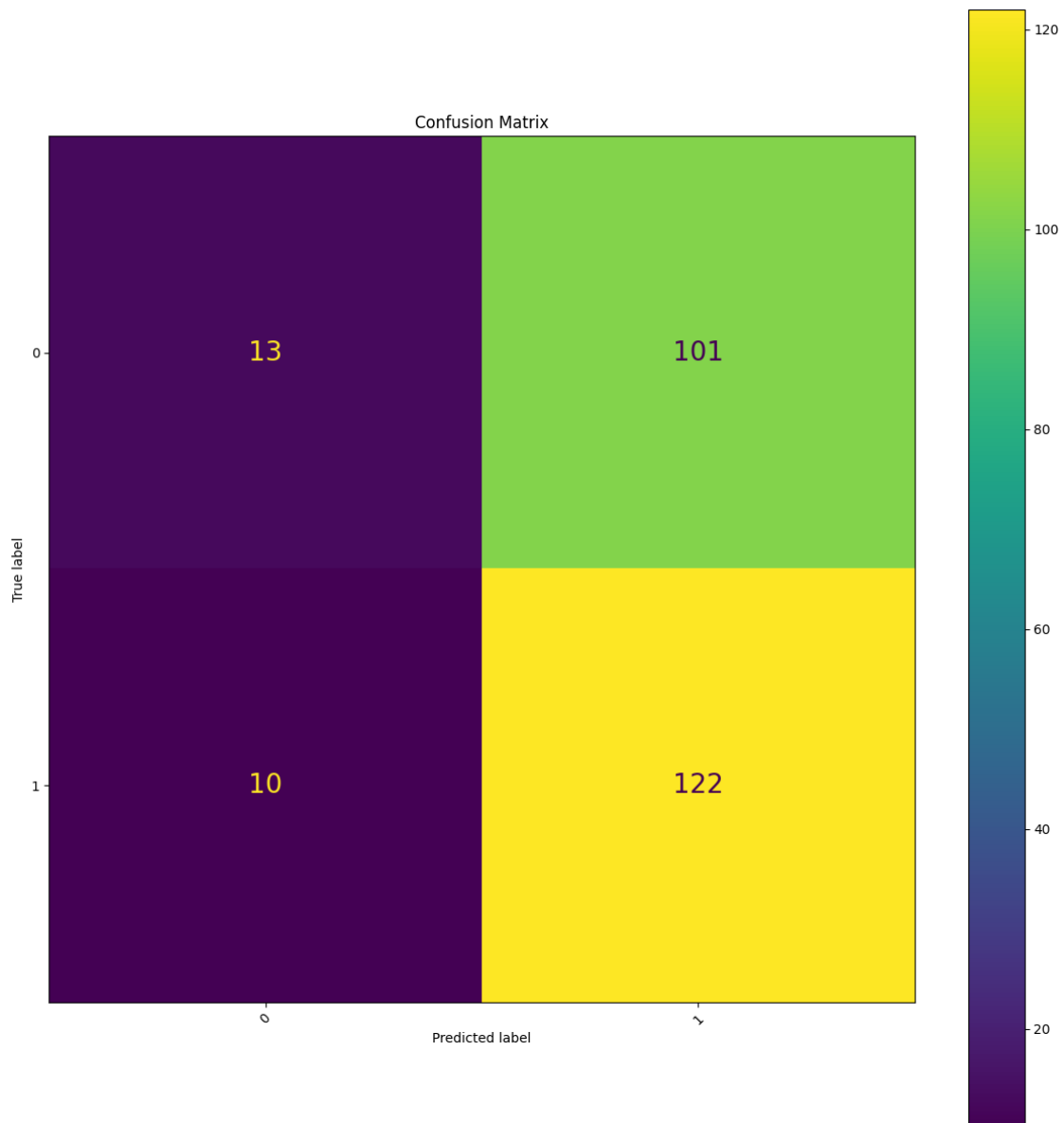
7.3 Combined Dataset

```
[91]: # Support Vector Machine for Combined Dataset
svm = SVC(random_state=42)
svm = Pipeline([
    ('scaler', StandardScaler()),
    ('svc', SVC(random_state=42))
])
```

```
fit_model(svm, cleaned_com_train, cleaned_com_val)
```

```
Model: Pipeline(steps=[('scaler', StandardScaler()), ('svc',  
SVC(random_state=42))])
```

	precision	recall	f1-score	support
0	0.11	0.57	0.19	23
1	0.92	0.55	0.69	223
accuracy			0.55	246
macro avg	0.52	0.56	0.44	246
weighted avg	0.85	0.55	0.64	246



```

[92]: # Define the hyperparameter grid to search
param_grid = {
    'svc__kernel': ['linear', 'rbf', 'poly', 'sigmoid'],
    'svc__C': [0.1, 1, 5],
    'svc__gamma': ['scale', 'auto'],
    'svc__max_iter': [1000, 2000]
}

# Create the GridSearchCV object
grid_search = GridSearchCV(
    estimator=svm,
    param_grid=param_grid,
    cv=5, # 5-fold cross-validation
    scoring=scoring, # print f1 and accuracy
    refit = 'accuracy',
    return_train_score=True,
    verbose=2
)

# Fit to training data
cleaned_com_train_df = pd.DataFrame(cleaned_com_train)
grid_search.fit(cleaned_com_train_df, train_y)

# Best hyperparameters
print("Best Hyperparameters:", grid_search.best_params_)

# Print performance metrics for cross-fold validation
pd.set_option('display.max_colwidth', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 2000)
results = pd.DataFrame(grid_search.cv_results_)
summary = results[['params', 'mean_test_accuracy', 'mean_test_f1',
    ↪ 'mean_train_accuracy', 'mean_train_f1']]
print(summary.sort_values(by='mean_test_accuracy', ascending=False))

# Best model
best_svm = grid_search.best_estimator_

fit_model(best_svm, cleaned_com_train, cleaned_com_val)

```

Fitting 5 folds for each of 48 candidates, totalling 240 fits
 [CV] END svc__C=0.1, svc__gamma=scale, svc__kernel=linear, svc__max_iter=1000;
 total time= 0.0s

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]


```
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=linear, svc__max_iter=2000;
total time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=rbf, svc__max_iter=1000; total
time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=rbf, svc__max_iter=1000; total
time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=rbf, svc__max_iter=1000; total
time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=rbf, svc__max_iter=1000; total
time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=rbf, svc__max_iter=2000; total
time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=rbf, svc__max_iter=2000; total
time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=rbf, svc__max_iter=2000; total
time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=poly, svc__max_iter=1000; total
time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=poly, svc__max_iter=1000; total
time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=poly, svc__max_iter=1000; total
time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=poly, svc__max_iter=1000; total
time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=poly, svc__max_iter=2000; total
time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=poly, svc__max_iter=2000; total
time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=poly, svc__max_iter=2000; total
time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=poly, svc__max_iter=2000; total
time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=sigmoid, svc__max_iter=1000;
total time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=sigmoid, svc__max_iter=1000;
total time=    0.0s
[CV] END svc__C=5, svc__gamma=scale, svc__kernel=sigmoid, svc__max_iter=1000;
total time=    0.0s
```



```

[CV] END svc__C=5, svc__gamma=auto, svc__kernel=rbf, svc__max_iter=2000; total
time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=rbf, svc__max_iter=2000; total
time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=rbf, svc__max_iter=2000; total
time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=poly, svc__max_iter=1000; total
time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=poly, svc__max_iter=1000; total
time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=poly, svc__max_iter=1000; total
time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=poly, svc__max_iter=1000; total
time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=poly, svc__max_iter=1000; total
time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=poly, svc__max_iter=2000; total
time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=poly, svc__max_iter=2000; total
time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=poly, svc__max_iter=2000; total
time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=poly, svc__max_iter=2000; total
time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=poly, svc__max_iter=2000; total
time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=sigmoid, svc__max_iter=1000;
total time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=sigmoid, svc__max_iter=1000;
total time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=sigmoid, svc__max_iter=1000;
total time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=sigmoid, svc__max_iter=1000;
total time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=sigmoid, svc__max_iter=1000;
total time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=sigmoid, svc__max_iter=2000;
total time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=sigmoid, svc__max_iter=2000;
total time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=sigmoid, svc__max_iter=2000;
total time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=sigmoid, svc__max_iter=2000;
total time= 0.0s
[CV] END svc__C=5, svc__gamma=auto, svc__kernel=sigmoid, svc__max_iter=2000;
total time= 0.0s
Best Hyperparameters: {'svc__C': 0.1, 'svc__gamma': 'scale', 'svc__kernel':
'linear', 'svc__max_iter': 1000}

```

	params	mean_test_accuracy	mean_test_f1	mean_train_accuracy	mean_train_f1
0	{'svc__C': 0.1, 'svc__gamma': 'scale', 'svc__kernel': 'linear', 'svc__max_iter': 1000}	1.000000	1.000000	1.000000	1.000000
1	{'svc__C': 0.1, 'svc__gamma': 'scale', 'svc__kernel': 'linear', 'svc__max_iter': 2000}	1.000000	1.000000	1.000000	1.000000
9	{'svc__C': 0.1, 'svc__gamma': 'auto', 'svc__kernel': 'linear', 'svc__max_iter': 2000}	1.000000	1.000000	1.000000	1.000000
8	{'svc__C': 0.1, 'svc__gamma': 'auto', 'svc__kernel': 'linear', 'svc__max_iter': 1000}	1.000000	1.000000	1.000000	1.000000
44	{'svc__C': 5, 'svc__gamma': 'auto', 'svc__kernel': 'poly', 'svc__max_iter': 1000}	1.000000	1.000000	1.000000	1.000000
45	{'svc__C': 5, 'svc__gamma': 'auto', 'svc__kernel': 'poly', 'svc__max_iter': 2000}	1.000000	1.000000	1.000000	1.000000
41	{'svc__C': 5, 'svc__gamma': 'auto', 'svc__kernel': 'linear', 'svc__max_iter': 2000}	1.000000	1.000000	1.000000	1.000000
40	{'svc__C': 5, 'svc__gamma': 'auto', 'svc__kernel': 'linear', 'svc__max_iter': 1000}	1.000000	1.000000	1.000000	1.000000
24	{'svc__C': 1, 'svc__gamma': 'auto', 'svc__kernel': 'linear', 'svc__max_iter': 1000}	1.000000	1.000000	1.000000	1.000000
25	{'svc__C': 1, 'svc__gamma': 'auto', 'svc__kernel': 'linear', 'svc__max_iter': 2000}	1.000000	1.000000	1.000000	1.000000
29	{'svc__C': 1, 'svc__gamma': 'auto', 'svc__kernel': 'poly', 'svc__max_iter': 2000}	1.000000	1.000000	1.000000	1.000000
28	{'svc__C': 1, 'svc__gamma': 'auto', 'svc__kernel': 'poly', 'svc__max_iter': 1000}	1.000000	1.000000	1.000000	1.000000
20	{'svc__C': 1, 'svc__gamma': 'scale', 'svc__kernel': 'poly', 'svc__max_iter': 1000}	1.000000	1.000000	1.000000	1.000000
21	{'svc__C': 1, 'svc__gamma': 'scale', 'svc__kernel': 'poly', 'svc__max_iter': 2000}	1.000000	1.000000	1.000000	1.000000
17	{'svc__C': 1, 'svc__gamma': 'scale', 'svc__kernel': 'linear', 'svc__max_iter': 2000}	1.000000	1.000000	1.000000	1.000000
16	{'svc__C': 1, 'svc__gamma': 'scale', 'svc__kernel': 'linear', 'svc__max_iter': 1000}	1.000000	1.000000	1.000000	1.000000

	'svc__max_iter': 1000}	1.000000	1.000000	1.000000
1.000000				
36	{'svc__C': 5, 'svc__gamma': 'scale', 'svc__kernel': 'poly', 'svc__max_iter': 1000}	1.000000	1.000000	1.000000
1.000000				
37	{'svc__C': 5, 'svc__gamma': 'scale', 'svc__kernel': 'poly', 'svc__max_iter': 2000}	1.000000	1.000000	1.000000
1.000000				
33	{'svc__C': 5, 'svc__gamma': 'scale', 'svc__kernel': 'linear', 'svc__max_iter': 2000}	1.000000	1.000000	1.000000
1.000000				
32	{'svc__C': 5, 'svc__gamma': 'scale', 'svc__kernel': 'linear', 'svc__max_iter': 1000}	1.000000	1.000000	1.000000
1.000000				
7	{'svc__C': 0.1, 'svc__gamma': 'scale', 'svc__kernel': 'sigmoid', 'svc__max_iter': 2000}	0.993416	0.993798	0.994017
0.994389				
6	{'svc__C': 0.1, 'svc__gamma': 'scale', 'svc__kernel': 'sigmoid', 'svc__max_iter': 1000}	0.993416	0.993798	0.994017
0.994389				
14	{'svc__C': 0.1, 'svc__gamma': 'auto', 'svc__kernel': 'sigmoid', 'svc__max_iter': 1000}	0.993416	0.993798	0.994017
0.994389				
15	{'svc__C': 0.1, 'svc__gamma': 'auto', 'svc__kernel': 'sigmoid', 'svc__max_iter': 2000}	0.993416	0.993798	0.994017
0.994389				
43	{'svc__C': 5, 'svc__gamma': 'auto', 'svc__kernel': 'rbf', 'svc__max_iter': 2000}	0.992593	0.993102	1.000000
1.000000				
42	{'svc__C': 5, 'svc__gamma': 'auto', 'svc__kernel': 'rbf', 'svc__max_iter': 1000}	0.992593	0.993102	1.000000
1.000000				
34	{'svc__C': 5, 'svc__gamma': 'scale', 'svc__kernel': 'rbf', 'svc__max_iter': 1000}	0.992593	0.993102	1.000000
1.000000				
35	{'svc__C': 5, 'svc__gamma': 'scale', 'svc__kernel': 'rbf', 'svc__max_iter': 2000}	0.992593	0.993102	1.000000
1.000000				
19	{'svc__C': 1, 'svc__gamma': 'scale', 'svc__kernel': 'rbf', 'svc__max_iter': 2000}	0.988477	0.989306	1.000000
1.000000				
18	{'svc__C': 1, 'svc__gamma': 'scale', 'svc__kernel': 'rbf', 'svc__max_iter': 1000}	0.988477	0.989306	1.000000
1.000000				
26	{'svc__C': 1, 'svc__gamma': 'auto', 'svc__kernel': 'rbf', 'svc__max_iter': 1000}	0.988477	0.989306	1.000000
1.000000				
27	{'svc__C': 1, 'svc__gamma': 'auto', 'svc__kernel': 'rbf',			

	'svc__max_iter': 2000}	0.988477	0.989306	1.000000
1.000000				
3	{'svc__C': 0.1, 'svc__gamma': 'scale', 'svc__kernel': 'rbf', 'svc__max_iter': 2000}	0.983532	0.985089	0.997730
0.997863				
2	{'svc__C': 0.1, 'svc__gamma': 'scale', 'svc__kernel': 'rbf', 'svc__max_iter': 1000}	0.983532	0.985089	0.997730
0.997863				
10	{'svc__C': 0.1, 'svc__gamma': 'auto', 'svc__kernel': 'rbf', 'svc__max_iter': 1000}	0.983532	0.985089	0.997730
0.997863				
11	{'svc__C': 0.1, 'svc__gamma': 'auto', 'svc__kernel': 'rbf', 'svc__max_iter': 2000}	0.983532	0.985089	0.997730
0.997863				
12	{'svc__C': 0.1, 'svc__gamma': 'auto', 'svc__kernel': 'poly', 'svc__max_iter': 1000}	0.979420	0.981601	0.920331
0.936680				
13	{'svc__C': 0.1, 'svc__gamma': 'auto', 'svc__kernel': 'poly', 'svc__max_iter': 2000}	0.979420	0.981601	0.920331
0.936680				
5	{'svc__C': 0.1, 'svc__gamma': 'scale', 'svc__kernel': 'poly', 'svc__max_iter': 2000}	0.979420	0.981601	0.920331
0.936680				
4	{'svc__C': 0.1, 'svc__gamma': 'scale', 'svc__kernel': 'poly', 'svc__max_iter': 1000}	0.979420	0.981601	0.920331
0.936680				
31	{'svc__C': 1, 'svc__gamma': 'auto', 'svc__kernel': 'sigmoid', 'svc__max_iter': 2000}	0.961283	0.961440	0.977310
0.978153				
30	{'svc__C': 1, 'svc__gamma': 'auto', 'svc__kernel': 'sigmoid', 'svc__max_iter': 1000}	0.961283	0.961440	0.977310
0.978153				
22	{'svc__C': 1, 'svc__gamma': 'scale', 'svc__kernel': 'sigmoid', 'svc__max_iter': 1000}	0.961283	0.961440	0.977310
0.978153				
23	{'svc__C': 1, 'svc__gamma': 'scale', 'svc__kernel': 'sigmoid', 'svc__max_iter': 2000}	0.961283	0.961440	0.977310
0.978153				
39	{'svc__C': 5, 'svc__gamma': 'scale', 'svc__kernel': 'sigmoid', 'svc__max_iter': 2000}	0.920760	0.927939	0.952566
0.955109				
38	{'svc__C': 5, 'svc__gamma': 'scale', 'svc__kernel': 'sigmoid', 'svc__max_iter': 1000}	0.920760	0.927939	0.952566
0.955109				
46	{'svc__C': 5, 'svc__gamma': 'auto', 'svc__kernel': 'sigmoid', 'svc__max_iter': 1000}	0.920760	0.927939	0.952566
0.955109				
47	{'svc__C': 5, 'svc__gamma': 'auto', 'svc__kernel': 'sigmoid',			

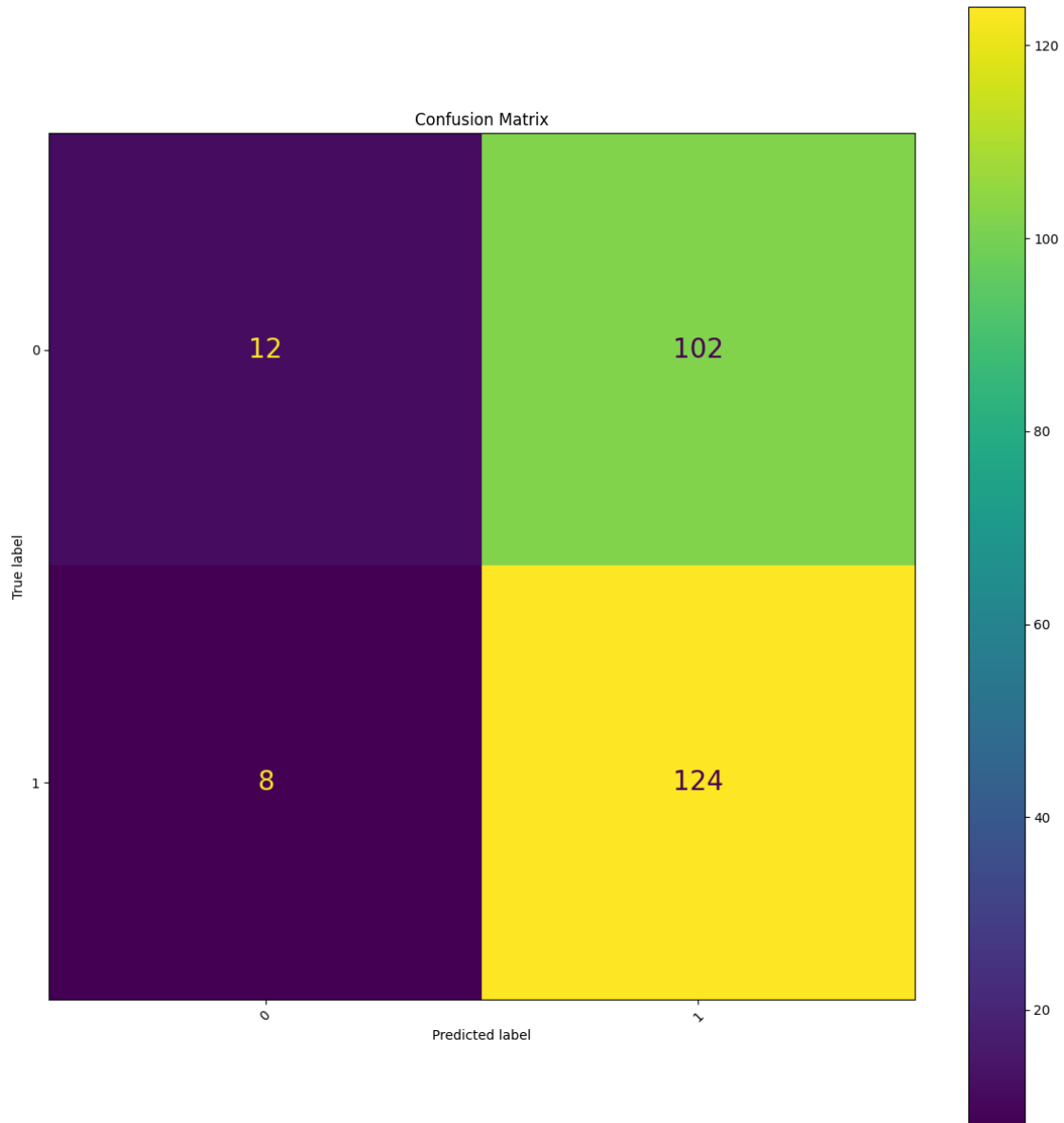
```

'svc__max_iter': 2000}          0.920760      0.927939          0.952566
0.955109
Model: Pipeline(steps=[('scaler', StandardScaler()),
                        ('svc',
                         SVC(C=0.1, kernel='linear', max_iter=1000, random_state=42))])
precision    recall  f1-score   support

     0         0.11     0.60     0.18         20
     1         0.94     0.55     0.69        226

 accuracy          0.55         246
 macro avg         0.52     0.57     0.44         246
weighted avg         0.87     0.55     0.65         246

```



8 Week 8 - Third Model

8.1 Sentiment Dataset

```
[93]: # Define the model name for FinBERT
MODEL_NAME = "ProsusAI/finbert"

# Load the tokenizer and model from Hugging Face Transformers
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
finbert_model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME)
```

```
finbert_model.eval()
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Loading weights: 0%| | 0/201 [00:00<?, ?it/s]

BertForSequenceClassification LOAD REPORT from: ProsusAI/finbert

Key	Status	
-----+-----+--		
bert.embeddings.position_ids	UNEXPECTED	

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

```
[93]: BertForSequenceClassification(
  (bert): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): BertEncoder(
      (layer): ModuleList(
        (0-11): 12 x BertLayer(
          (attention): BertAttention(
            (self): BertSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): BertSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
        )
      )
      (intermediate): BertIntermediate(
        (dense): Linear(in_features=768, out_features=3072, bias=True)
        (intermediate_act_fn): GELUActivation()
      )
      (output): BertOutput(
        (dense): Linear(in_features=3072, out_features=768, bias=True)
        (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      )
    )
  )
)
```



```

        (dropout): Dropout(p=0.1, inplace=False)
    )
)
)
)
(pooler): BertPooler(
  (dense): Linear(in_features=768, out_features=768, bias=True)
  (activation): Tanh()
)
)
(dropout): Dropout(p=0.1, inplace=False)
(classifier): Linear(in_features=768, out_features=3, bias=True)
)

```

[94]: *# Function to get sentiment probabilities from FinBERT for a given text input*

```

def finbert_sentiment_probs(text, max_length=512):
    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=max_length
    )
    with torch.no_grad():
        outputs = finbert_model(**inputs)
        probs = torch.softmax(outputs.logits, dim=1).numpy()[0]
        labels = [finbert_model.config.id2label[i] for i in range(len(probs))]
    return dict(zip(labels, probs))

```

[115]: *# Apply the function to the 'full_text' column of the training dataset to get*

```

    ↪ sentiment probabilities
scores = Final_Raw_X_train["full_text"].fillna("").apply(finbert_sentiment_probs)

# Convert the list of dictionaries into a DataFrame and rename columns to
    ↪ indicate they are from FinBERT
score_sentiment = scores.apply(pd.Series)
score_sentiment.columns = [f"finbert_{c.lower()}" for c in score_sentiment.
    ↪ columns]

# Combine the original training dataset with the new FinBERT sentiment scores
finbert_df = pd.concat([Final_Raw_X_train, score_sentiment], axis=1)
finbert_df["finbert_polarity"] = finbert_df["finbert_positive"] -
    ↪ finbert_df["finbert_negative"]
finbert_df["finbert_confidence"] = finbert_df[[
    "finbert_positive", "finbert_negative", "finbert_neutral"
]].max(axis=1)

```

```
[116]: finbert_df.head(10)
```

```
[116]:      Dates AAPL Share Price full_text sentiment_neg sentiment_neu
sentiment_pos sentiment_polarity tokens Stock Price Direction finbert_positive
finbert_negative finbert_neutral finbert_polarity finbert_confidence
0 2018-01-02      43.07      0.0      0.0
0.0      0.0 NaN      down      0.359163
0.216652      0.424185      0.142512      0.424185
1 2018-01-03      43.06      0.0      0.0
0.0      0.0 NaN      down      0.359163
0.216652      0.424185      0.142512      0.424185
2 2018-01-04      43.26      0.0      0.0
0.0      0.0 NaN      up      0.359163
0.216652      0.424185      0.142512      0.424185
3 2018-01-05      43.75      0.0      0.0
0.0      0.0 NaN      up      0.359163
0.216652      0.424185      0.142512      0.424185
4 2018-01-08      43.59      0.0      0.0
0.0      0.0 NaN      down      0.359163
0.216652      0.424185      0.142512      0.424185
5 2018-01-09      43.58      0.0      0.0
0.0      0.0 NaN      down      0.359163
0.216652      0.424185      0.142512      0.424185
6 2018-01-10      43.57      0.0      0.0
0.0      0.0 NaN      down      0.359163
0.216652      0.424185      0.142512      0.424185
7 2018-01-11      43.82      0.0      0.0
0.0      0.0 NaN      up      0.359163
0.216652      0.424185      0.142512      0.424185
8 2018-01-12      44.27      0.0      0.0
0.0      0.0 NaN      up      0.359163
0.216652      0.424185      0.142512      0.424185
9 2018-01-16      44.05      0.0      0.0
0.0      0.0 NaN      down      0.359163
0.216652      0.424185      0.142512      0.424185
```

```
[117]: # Repeat the same process for the validation set
# Apply the function to the 'full_text' column of the validation dataset to get
    ↳ sentiment probabilities
scores_val = Final_Raw_X_val["full_text"].fillna("").
    ↳ apply(finbert_sentiment_probs)

# Convert the list of dictionaries into a DataFrame and rename columns to
    ↳ indicate they are from FinBERT
score_sentiment_val = scores_val.apply(pd.Series)
score_sentiment_val.columns = [f"finbert_{c.lower()}" for c in
    ↳ score_sentiment_val.columns]
```

```
# Combine the original validation dataset with the new FinBERT sentiment scores
finbert_val_df = pd.concat([Final_Raw_X_val, score_sentiment_val], axis=1)
finbert_val_df["finbert_polarity"] = finbert_val_df["finbert_positive"] -
    ↪finbert_val_df["finbert_negative"]
finbert_val_df["finbert_confidence"] = finbert_val_df[["finbert_positive",
    ↪"finbert_negative", "finbert_neutral"]].max(axis=1)
```

```
[118]: # Define the list of FinBERT feature columns for later use in modeling
full_finbert_feature_cols = [
    "finbert_positive", "finbert_negative", "finbert_neutral",
    "finbert_polarity", "finbert_confidence",
    "sentiment_pos", "sentiment_neg", "sentiment_neu", "sentiment_polarity"
]
```

```
[119]: # Prepare the training data for modeling
X_train_finbert1 = finbert_df[full_finbert_feature_cols]
y_train_finbert1 = finbert_df["Stock Price Direction"].map({"up": 1, "down": 0})
# Prepare the validation data for modeling
X_val_finbert1 = finbert_val_df[full_finbert_feature_cols]
y_val_finbert1 = finbert_val_df["Stock Price Direction"].map({"up": 1, "down":
    ↪0})
```

```
[120]: # Initialize and fit the logistic regression model
finbert_lr = LR(max_iter=500, C=1, random_state=42)
finbert_lr.fit(X_train_finbert1, y_train_finbert1)

# Make predictions on the validation set
y_pred_finbert = finbert_lr.predict(X_val_finbert1)
# Evaluate the model
print("Logistic Regression - FinBERT Sentiment Features Results")
# Accuracy
finbert_accuracy = accuracy_score(y_val_finbert1, y_pred_finbert)
print(f"\nAccuracy: {finbert_accuracy:.4f}")
# Classification Report (Precision, Recall, F1-Score, Support)
print("\nClassification Report:")
print(CR(y_val_finbert1, y_pred_finbert, target_names=['Down (0)', 'Up (1)']))
```

Logistic Regression - FinBERT Sentiment Features Results

Accuracy: 0.5578

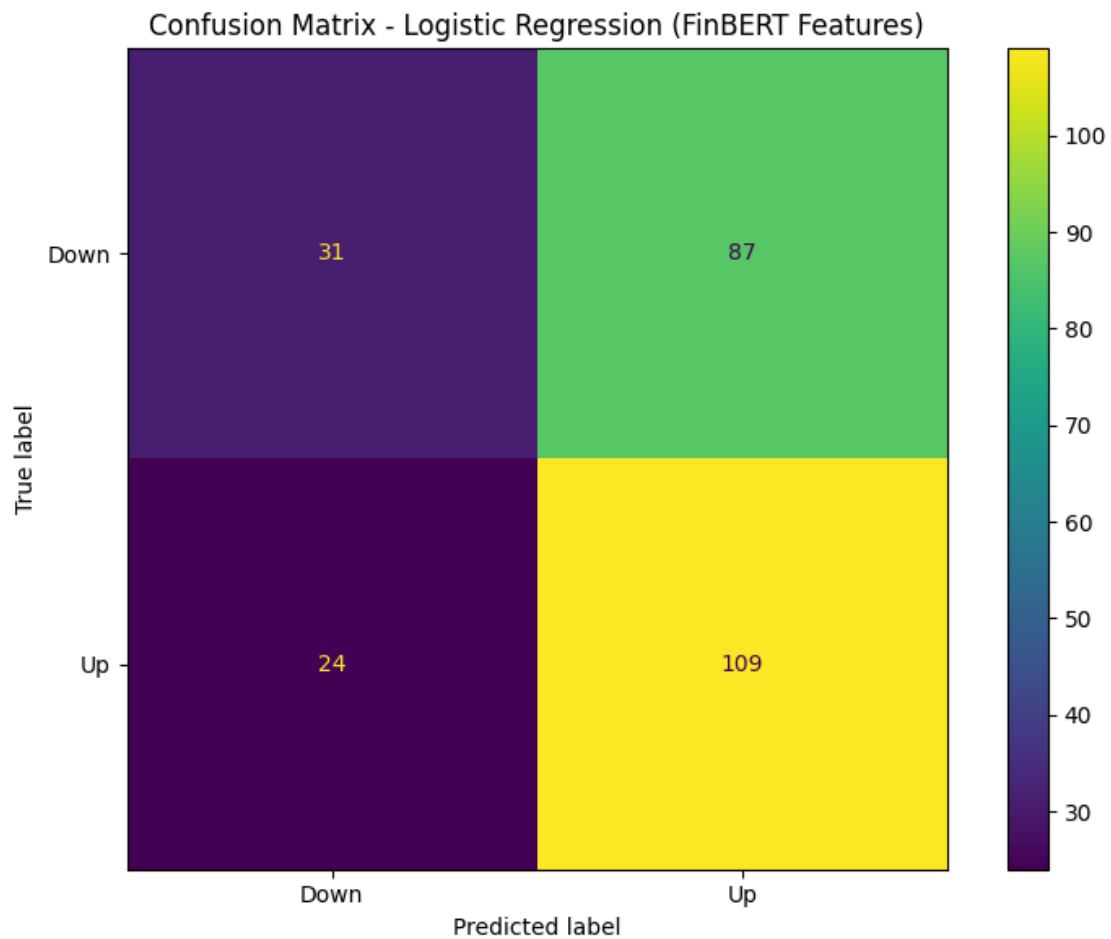
Classification Report:

	precision	recall	f1-score	support
Down (0)	0.56	0.26	0.36	118
Up (1)	0.56	0.82	0.66	133

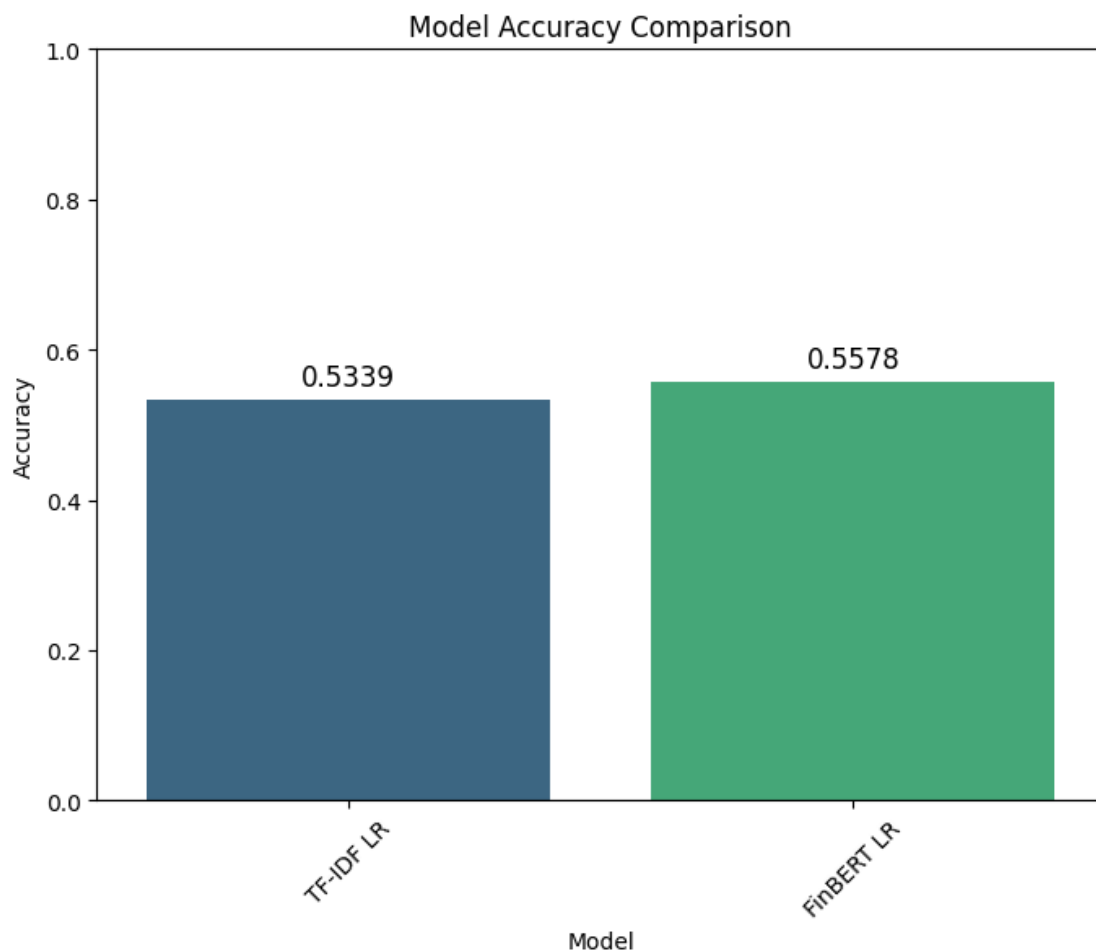
accuracy			0.56	251
macro avg	0.56	0.54	0.51	251
weighted avg	0.56	0.56	0.52	251

```
[121]: # Confusion Matrix
print("\nConfusion Matrix:")
cm = confusion_matrix(y_val_finbert1, y_pred_finbert, labels=finbert_lr.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Down', 'Up'])
fig, ax = plt.subplots(figsize=(8, 6))
disp.plot(ax=ax, cmap='viridis')
plt.title("Confusion Matrix - Logistic Regression (FinBERT Features)")
plt.tight_layout()
plt.show()
```

Confusion Matrix:



```
[122]: # Compare the performance of the FinBERT-based model to the previous models,
        ↳ using a bar chart
model_names = ['TF-IDF LR', 'FinBERT LR']
accuracies = [accuracy_score(y_val_sentiment, y_pred_combined),
               ↳ accuracy_score(y_val_finbert1, y_pred_finbert)]
plt.figure(figsize=(8, 6))
sns.barplot(x=model_names, y=accuracies, palette='viridis')
plt.ylim(0, 1)
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.xlabel("Model")
plt.xticks(rotation=45)
for index, value in enumerate(accuracies):
    plt.text(index, value + 0.01, f"{value:.4f}", ha='center', va='bottom',
             ↳ fontsize=12)
```



```
[123]: # Grid search on logistic regression for FinBERT + sentiment features

X_train_finbert_lr = X_train_finbert1.copy()
y_train_finbert_lr = y_train_finbert1.copy()
X_val_finbert_lr = X_val_finbert1.copy()
y_val_finbert_lr = y_val_finbert1.copy()

param_grid_lr = [
    {
        'C': [0.01, 0.1, 1.0, 10.0],
        'penalty': ['l2'],
        'solver': ['lbfgs', 'saga'],
        'class_weight': [None, 'balanced']
    },
    {
        'C': [0.01, 0.1, 1.0, 10.0],
        'penalty': ['l1'],
        'solver': ['saga'],
        'class_weight': [None, 'balanced']
    },
    {
        'C': [0.01, 0.1, 1.0, 10.0],
        'penalty': ['elasticnet'],
        'solver': ['saga'],
        'l1_ratio': [0.0, 0.25, 0.5, 0.75, 1.0],
        'class_weight': [None, 'balanced']
    }
]

grid_search_lr = GridSearchCV(
    LR(max_iter=1000, random_state=42),
    param_grid_lr,
    cv=5,
    scoring='f1',
    return_train_score=True,
    verbose=2
)

grid_search_lr.fit(X_train_finbert_lr, y_train_finbert_lr)

print("Best Hyperparameters (Logistic Regression):", grid_search_lr.best_params_)

results_lr = pd.DataFrame(grid_search_lr.cv_results_)
summary_lr = results_lr[['params', 'mean_test_score', 'mean_train_score']]
print(summary_lr.sort_values(by='mean_test_score', ascending=False).head(10))

best_lr = grid_search_lr.best_estimator_
```

Fitting 5 folds for each of 64 candidates, totalling 320 fits

[illegible]

[illegible]

```

0.0s
[CV] END C=0.1, class_weight=balanced, penalty=l1, solver=saga; total time=
0.0s
[CV] END C=0.1, class_weight=balanced, penalty=l1, solver=saga; total time=
0.0s
[CV] END C=0.1, class_weight=balanced, penalty=l1, solver=saga; total time=
0.0s
[CV] END ..C=1.0, class_weight=None, penalty=l1, solver=saga; total time= 0.0s
[CV] END ..C=1.0, class_weight=None, penalty=l1, solver=saga; total time= 0.0s
[CV] END ..C=1.0, class_weight=None, penalty=l1, solver=saga; total time= 0.0s
[CV] END ..C=1.0, class_weight=None, penalty=l1, solver=saga; total time= 0.0s
[CV] END ..C=1.0, class_weight=None, penalty=l1, solver=saga; total time= 0.0s
[CV] END C=1.0, class_weight=balanced, penalty=l1, solver=saga; total time=
0.0s
[CV] END C=1.0, class_weight=balanced, penalty=l1, solver=saga; total time=
0.0s
[CV] END C=1.0, class_weight=balanced, penalty=l1, solver=saga; total time=
0.0s
[CV] END C=1.0, class_weight=balanced, penalty=l1, solver=saga; total time=
0.0s
[CV] END C=1.0, class_weight=balanced, penalty=l1, solver=saga; total time=
0.0s
[CV] END .C=10.0, class_weight=None, penalty=l1, solver=saga; total time= 0.1s
[CV] END .C=10.0, class_weight=None, penalty=l1, solver=saga; total time= 0.0s
[CV] END .C=10.0, class_weight=None, penalty=l1, solver=saga; total time= 0.1s
[CV] END .C=10.0, class_weight=None, penalty=l1, solver=saga; total time= 0.0s
[CV] END .C=10.0, class_weight=None, penalty=l1, solver=saga; total time= 0.1s
[CV] END C=10.0, class_weight=balanced, penalty=l1, solver=saga; total time=
0.1s
[CV] END C=10.0, class_weight=balanced, penalty=l1, solver=saga; total time=
0.0s
[CV] END C=10.0, class_weight=balanced, penalty=l1, solver=saga; total time=
0.1s
[CV] END C=10.0, class_weight=balanced, penalty=l1, solver=saga; total time=
0.1s
[CV] END C=10.0, class_weight=balanced, penalty=l1, solver=saga; total time=
0.1s
[CV] END C=0.01, class_weight=None, l1_ratio=0.0, penalty=elasticnet,
solver=saga; total time= 0.0s
[CV] END C=0.01, class_weight=None, l1_ratio=0.0, penalty=elasticnet,
solver=saga; total time= 0.0s
[CV] END C=0.01, class_weight=None, l1_ratio=0.0, penalty=elasticnet,
solver=saga; total time= 0.0s
[CV] END C=0.01, class_weight=None, l1_ratio=0.0, penalty=elasticnet,
solver=saga; total time= 0.0s
[CV] END C=0.01, class_weight=None, l1_ratio=0.0, penalty=elasticnet,
solver=saga; total time= 0.0s
[CV] END C=0.01, class_weight=None, l1_ratio=0.25, penalty=elasticnet,

```


[illegible]

[illegible]

[illegible]


```

solver=saga; total time= 0.1s
[CV] END C=10.0, class_weight=balanced, l1_ratio=1.0, penalty=elasticnet,
solver=saga; total time= 0.1s
[CV] END C=10.0, class_weight=balanced, l1_ratio=1.0, penalty=elasticnet,
solver=saga; total time= 0.1s
Best Hyperparameters (Logistic Regression): {'C': 0.1, 'class_weight': None,
'l1_ratio': 0.5, 'penalty': 'elasticnet', 'solver': 'saga'}
      params mean_test_score mean_train_score
36  {'C': 0.1, 'class_weight': None, 'l1_ratio': 0.5, 'penalty': 'elasticnet',
'solver': 'saga'}          0.692015          0.690219
25  {'C': 0.01, 'class_weight': None, 'l1_ratio': 0.25, 'penalty': 'elasticnet',
'solver': 'saga'}          0.691396          0.691398
27  {'C': 0.01, 'class_weight': None, 'l1_ratio': 0.75, 'penalty': 'elasticnet',
'solver': 'saga'}          0.691396          0.691398
26  {'C': 0.01, 'class_weight': None, 'l1_ratio': 0.5, 'penalty': 'elasticnet',
'solver': 'saga'}          0.691396          0.691398
18                                {'C': 0.1, 'class_weight': None, 'penalty': 'l1',
'solver': 'saga'}          0.691396          0.691398
16                                {'C': 0.01, 'class_weight': None, 'penalty': 'l1',
'solver': 'saga'}          0.691396          0.691398
37  {'C': 0.1, 'class_weight': None, 'l1_ratio': 0.75, 'penalty': 'elasticnet',
'solver': 'saga'}          0.691396          0.691398
28  {'C': 0.01, 'class_weight': None, 'l1_ratio': 1.0, 'penalty': 'elasticnet',
'solver': 'saga'}          0.691396          0.691398
38  {'C': 0.1, 'class_weight': None, 'l1_ratio': 1.0, 'penalty': 'elasticnet',
'solver': 'saga'}          0.691396          0.691398
24  {'C': 0.01, 'class_weight': None, 'l1_ratio': 0.0, 'penalty': 'elasticnet',
'solver': 'saga'}          0.688716          0.691459

```

```

[124]: # Validation performance with best LR model
y_pred_val_lr = best_lr.predict(X_val_finbert_lr)

print("\nLogistic Regression GridSearch Results on Validation Set")
print(f"Accuracy: {accuracy_score(y_val_finbert_lr, y_pred_val_lr):.4f}")
print("Classification Report:\n", CR(y_val_finbert_lr, y_pred_val_lr,
    ↳target_names=['Down (0)', 'Up (1)']))

cm_lr = confusion_matrix(y_val_finbert_lr, y_pred_val_lr, labels=best_lr.
    ↳classes_)

disp_lr = ConfusionMatrixDisplay(confusion_matrix=cm_lr, display_labels=['Down',
    ↳'Up'])
fig, ax = plt.subplots(figsize=(8, 6))
disp_lr.plot(ax=ax, cmap='viridis')
plt.title("Confusion Matrix - Logistic Regression (FinBERT Features)")
plt.tight_layout()
plt.show()

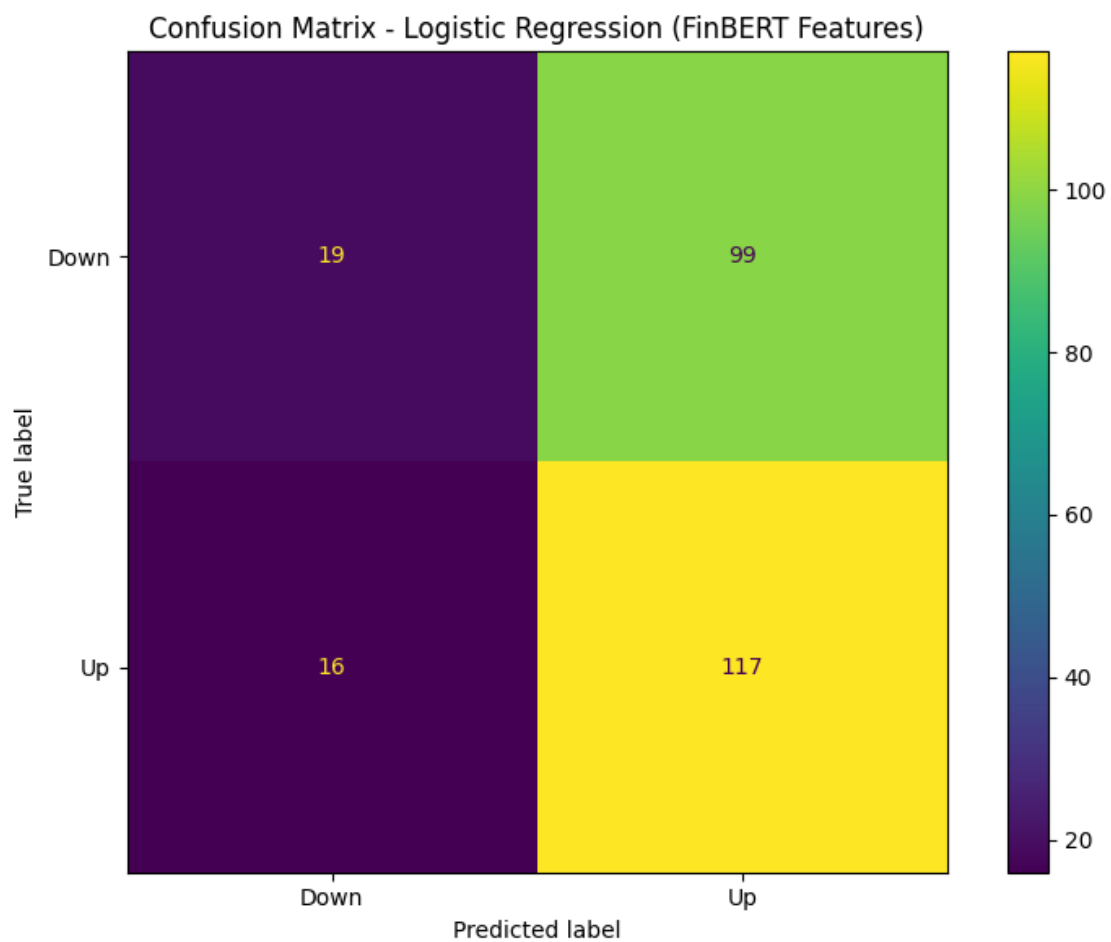
```

Logistic Regression GridSearch Results on Validation Set

Accuracy: 0.5418

Classification Report:

	precision	recall	f1-score	support
Down (0)	0.54	0.16	0.25	118
Up (1)	0.54	0.88	0.67	133
accuracy			0.54	251
macro avg	0.54	0.52	0.46	251
weighted avg	0.54	0.54	0.47	251



8.2 Historical Price Dataset

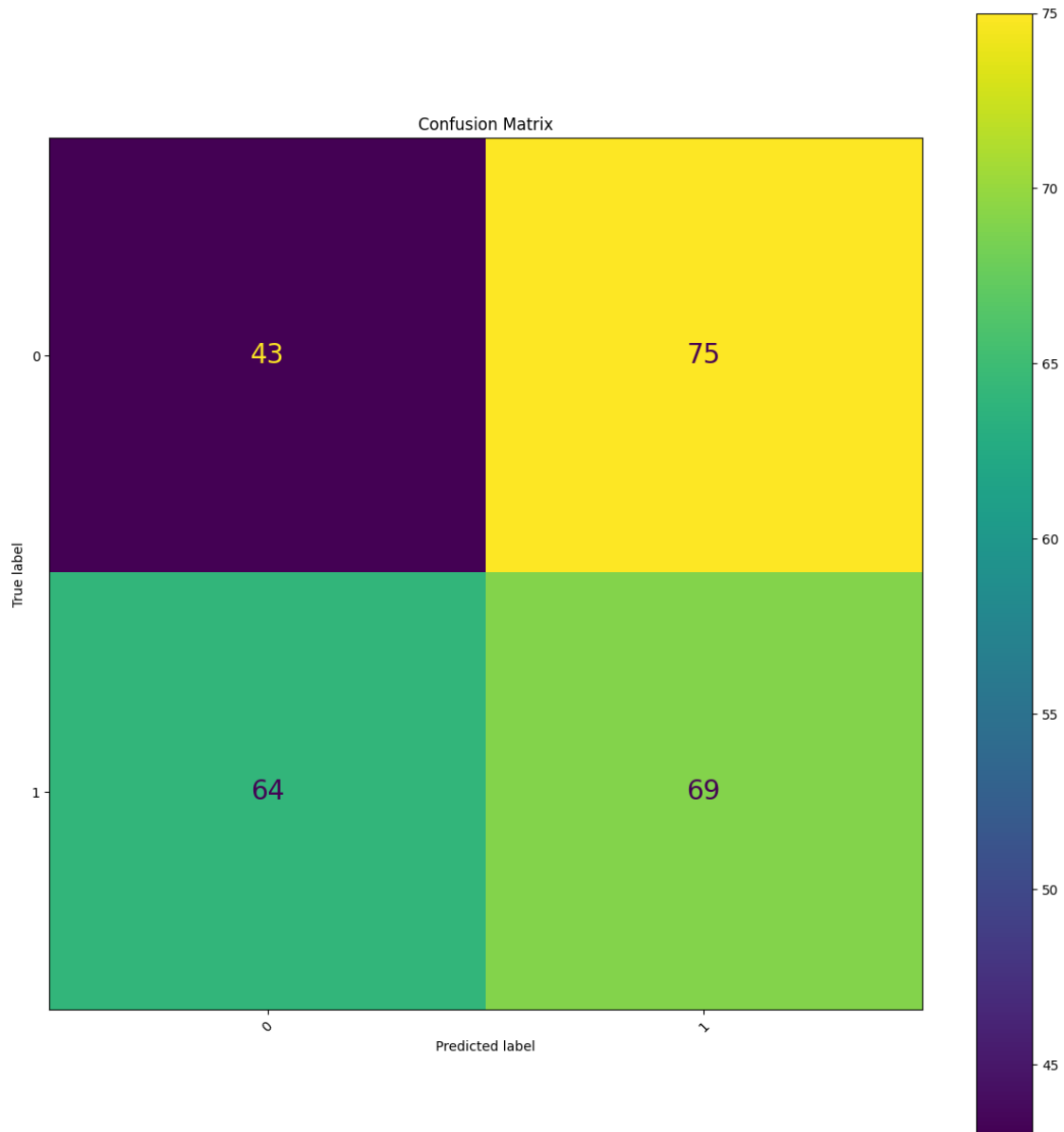
```
[125]: # Initial XGBoost model
xgb_model = XGBClassifier(random_state=42)

fit_model_price(xgb_model, cleaned_ts_train, cleaned_ts_val)
```

```
Model: XGBClassifier(base_score=None, booster=None, callbacks=None,
                     colsample_bylevel=None, colsample_bynode=None,
                     colsample_bytree=None, device=None, early_stopping_rounds=None,
                     enable_categorical=False, eval_metric=None, feature_types=None,
                     feature_weights=None, gamma=None, grow_policy=None,
                     importance_type=None, interaction_constraints=None,
                     learning_rate=None, max_bin=None, max_cat_threshold=None,
                     max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
                     max_leaves=None, min_child_weight=None, missing=nan,
                     monotone_constraints=None, multi_strategy=None, n_estimators=None,
                     n_jobs=None, num_parallel_tree=None, ...)
precision    recall  f1-score   support

           0       0.36    0.40    0.38         107
           1       0.52    0.48    0.50         144

 accuracy          0.45         251
 macro avg          0.44    0.44    0.44         251
weighted avg          0.45    0.45    0.45         251
```



```
[126]: # Grid Search for hyperparameter tuning
if (cleaned_ts_train.loc[5, 'Stock Price Direction'] == 'Down') |
↳(cleaned_ts_train.loc[5, 'Stock Price Direction'] == 'Up'):
    cleaned_ts_train['Stock Price Direction'] = cleaned_ts_train['Stock Price_
↳Direction'].map({'Down': 0, 'Up': 1})
    cleaned_ts_val['Stock Price Direction'] = cleaned_ts_val['Stock Price_
↳Direction'].map({'Down': 0, 'Up': 1})
    cleaned_ts_test['Stock Price Direction'] = cleaned_ts_test['Stock Price_
↳Direction'].map({'Down': 0, 'Up': 1})
```

```

param_grid = {
    'xgb__n_estimators': [100, 400, 750],
    'xgb__max_depth': [3, 4, 5],
    'xgb__learning_rate': [0.01, 0.02, 0.05, 0.10]
}

scoring = {
    'accuracy': make_scorer(accuracy_score),
    'f1':      make_scorer(f1_score, pos_label=1)
}

price_xgb = Pipeline([
    ('xgb', XGBClassifier(
        scale_pos_weight=len(train_y[train_y==0]) / len(train_y[train_y==1]), #
        ↪replaces class_weight='balanced'
        eval_metric='logloss',
        random_state=42
    ))
])

grid_search = RandomizedSearchCV(
    estimator=price_xgb,
    param_distributions=param_grid,
    n_iter=50,
    cv=5,
    scoring=scoring,
    refit='f1',
    return_train_score=True,
    random_state=42,
    verbose=2
)

price_train_X = cleaned_ts_train.drop(columns=['Stock Price Direction', 'Dates'])
price_train_y = cleaned_ts_train['Stock Price Direction']
grid_search.fit(price_train_X, price_train_y)

print("Best Hyperparameters:", grid_search.best_params_)

results = pd.DataFrame(grid_search.cv_results_)
summary = results[['params', 'mean_test_accuracy', 'mean_test_f1',
    ↪'mean_train_accuracy', 'mean_train_f1']]
print(summary.sort_values(by='mean_test_f1', ascending=False))

price_xgb_tuned = Pipeline([
    ('xgb', XGBClassifier(
        n_estimators=grid_search.best_params_['xgb__n_estimators'],
        max_depth=grid_search.best_params_['xgb__max_depth'],

```

```

        learning_rate=grid_search.best_params_['xgb__learning_rate'],
        scale_pos_weight=len(price_train_y[price_train_y==0]) /
→ len(price_train_y[price_train_y==1]),
        eval_metric='logloss',
        random_state=42
    ))
])

fit_model_price(price_xgb_tuned, cleaned_ts_train, cleaned_ts_val)

```

Fitting 5 folds for each of 36 candidates, totalling 180 fits

```

[CV] END xgb__learning_rate=0.01, xgb__max_depth=3, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=3, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=3, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=3, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=3, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=3, xgb__n_estimators=400; total
time= 0.2s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=3, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=3, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=3, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=3, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=3, xgb__n_estimators=750; total
time= 0.2s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=3, xgb__n_estimators=750; total
time= 0.2s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=3, xgb__n_estimators=750; total
time= 0.2s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=3, xgb__n_estimators=750; total
time= 0.2s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=3, xgb__n_estimators=750; total
time= 0.2s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=4, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=4, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=4, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.01, xgb__max_depth=4, xgb__n_estimators=100; total

```



```
time=    0.0s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=4, xgb__n_estimators=100; total  
time=    0.0s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=4, xgb__n_estimators=400; total  
time=    0.1s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=4, xgb__n_estimators=400; total  
time=    0.1s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=4, xgb__n_estimators=400; total  
time=    0.1s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=4, xgb__n_estimators=400; total  
time=    0.1s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=4, xgb__n_estimators=750; total  
time=    0.2s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=4, xgb__n_estimators=750; total  
time=    0.2s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=4, xgb__n_estimators=750; total  
time=    0.2s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=4, xgb__n_estimators=750; total  
time=    0.2s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=5, xgb__n_estimators=100; total  
time=    0.1s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=5, xgb__n_estimators=100; total  
time=    0.0s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=5, xgb__n_estimators=100; total  
time=    0.0s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=5, xgb__n_estimators=100; total  
time=    0.0s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=5, xgb__n_estimators=400; total  
time=    0.1s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=5, xgb__n_estimators=400; total  
time=    0.1s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=5, xgb__n_estimators=400; total  
time=    0.1s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=5, xgb__n_estimators=400; total  
time=    0.2s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=5, xgb__n_estimators=750; total  
time=    0.2s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=5, xgb__n_estimators=750; total  
time=    0.2s  
[CV] END xgb__learning_rate=0.01, xgb__max_depth=5, xgb__n_estimators=750; total
```



```

time= 0.0s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=3, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=3, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=3, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=3, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=3, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=3, xgb__n_estimators=750; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=3, xgb__n_estimators=750; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=3, xgb__n_estimators=750; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=3, xgb__n_estimators=750; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=4, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=4, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=4, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=4, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=4, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=4, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=4, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=4, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=4, xgb__n_estimators=750; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=4, xgb__n_estimators=750; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=4, xgb__n_estimators=750; total

```

```

time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=4, xgb__n_estimators=750; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=4, xgb__n_estimators=750; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=100; total
time= 0.0s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=750; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=750; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=750; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=750; total
time= 0.1s
Best Hyperparameters: {'xgb__n_estimators': 400, 'xgb__max_depth': 4,
'xgb__learning_rate': 0.01}

```

				params	
	mean_test_accuracy	mean_test_f1	mean_train_accuracy	mean_train_f1	
8	{'xgb__n_estimators': 750, 'xgb__max_depth': 5, 'xgb__learning_rate': 0.01}	0.512737	0.480362	0.549710	0.521211
7	{'xgb__n_estimators': 400, 'xgb__max_depth': 5, 'xgb__learning_rate': 0.01}	0.512737	0.480362	0.549710	0.521211
5	{'xgb__n_estimators': 750, 'xgb__max_depth': 4, 'xgb__learning_rate': 0.01}	0.512737	0.480362	0.549710	0.521211
4	{'xgb__n_estimators': 400, 'xgb__max_depth': 4, 'xgb__learning_rate': 0.01}	0.512737	0.480362	0.549710	0.521211
34	{'xgb__n_estimators': 400, 'xgb__max_depth': 5, 'xgb__learning_rate': 0.1}				

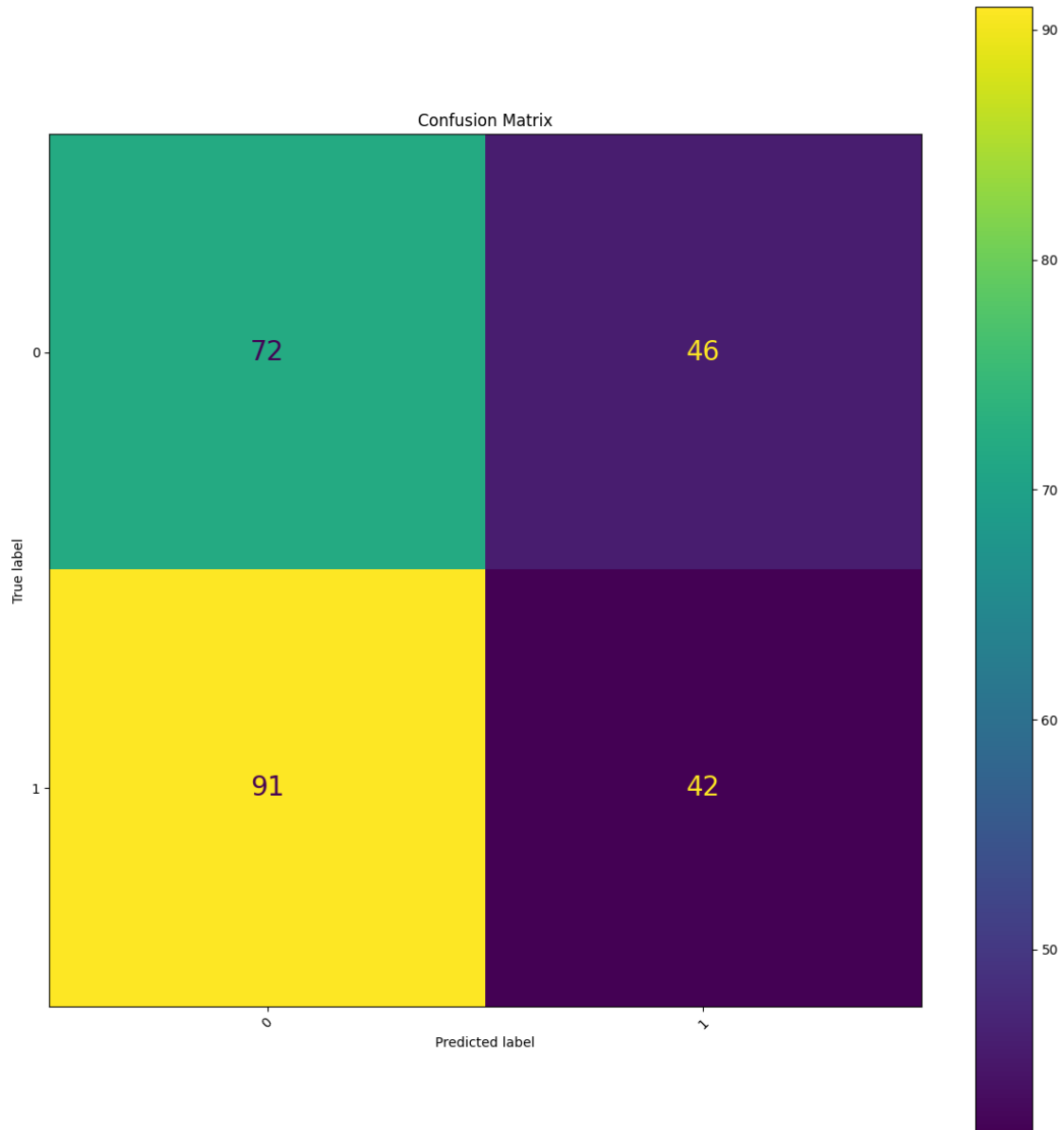
	0.512737	0.480362	0.549710	0.521211
35	{'xgb__n_estimators': 750, 'xgb__max_depth': 5, 'xgb__learning_rate': 0.1}			
	0.512737	0.480362	0.549710	0.521211
33	{'xgb__n_estimators': 100, 'xgb__max_depth': 5, 'xgb__learning_rate': 0.1}			
	0.512737	0.480362	0.549710	0.521211
32	{'xgb__n_estimators': 750, 'xgb__max_depth': 4, 'xgb__learning_rate': 0.1}			
	0.512737	0.480362	0.549710	0.521211
15	{'xgb__n_estimators': 100, 'xgb__max_depth': 5, 'xgb__learning_rate': 0.02}			
	0.512737	0.480362	0.549710	0.521211
14	{'xgb__n_estimators': 750, 'xgb__max_depth': 4, 'xgb__learning_rate': 0.02}			
	0.512737	0.480362	0.549710	0.521211
13	{'xgb__n_estimators': 400, 'xgb__max_depth': 4, 'xgb__learning_rate': 0.02}			
	0.512737	0.480362	0.549710	0.521211
12	{'xgb__n_estimators': 100, 'xgb__max_depth': 4, 'xgb__learning_rate': 0.02}			
	0.512737	0.480362	0.549710	0.521211
16	{'xgb__n_estimators': 400, 'xgb__max_depth': 5, 'xgb__learning_rate': 0.02}			
	0.512737	0.480362	0.549710	0.521211
17	{'xgb__n_estimators': 750, 'xgb__max_depth': 5, 'xgb__learning_rate': 0.02}			
	0.512737	0.480362	0.549710	0.521211
25	{'xgb__n_estimators': 400, 'xgb__max_depth': 5, 'xgb__learning_rate': 0.05}			
	0.512737	0.480362	0.549710	0.521211
26	{'xgb__n_estimators': 750, 'xgb__max_depth': 5, 'xgb__learning_rate': 0.05}			
	0.512737	0.480362	0.549710	0.521211
24	{'xgb__n_estimators': 100, 'xgb__max_depth': 5, 'xgb__learning_rate': 0.05}			
	0.512737	0.480362	0.549710	0.521211
23	{'xgb__n_estimators': 750, 'xgb__max_depth': 4, 'xgb__learning_rate': 0.05}			
	0.512737	0.480362	0.549710	0.521211
22	{'xgb__n_estimators': 400, 'xgb__max_depth': 4, 'xgb__learning_rate': 0.05}			
	0.512737	0.480362	0.549710	0.521211
21	{'xgb__n_estimators': 100, 'xgb__max_depth': 4, 'xgb__learning_rate': 0.05}			
	0.512737	0.480362	0.549710	0.521211
30	{'xgb__n_estimators': 100, 'xgb__max_depth': 4, 'xgb__learning_rate': 0.1}			
	0.512737	0.480362	0.549710	0.521211
31	{'xgb__n_estimators': 400, 'xgb__max_depth': 4, 'xgb__learning_rate': 0.1}			
	0.512737	0.480362	0.549710	0.521211
6	{'xgb__n_estimators': 100, 'xgb__max_depth': 5, 'xgb__learning_rate': 0.01}			
	0.516852	0.479943	0.548889	0.512562
3	{'xgb__n_estimators': 100, 'xgb__max_depth': 4, 'xgb__learning_rate': 0.01}			
	0.516852	0.479943	0.548889	0.512562
9	{'xgb__n_estimators': 100, 'xgb__max_depth': 3, 'xgb__learning_rate': 0.02}			
	0.498742	0.446241	0.542110	0.495078
0	{'xgb__n_estimators': 100, 'xgb__max_depth': 3, 'xgb__learning_rate': 0.01}			
	0.498742	0.433769	0.541905	0.489749
27	{'xgb__n_estimators': 100, 'xgb__max_depth': 3, 'xgb__learning_rate': 0.1}			
	0.496266	0.432350	0.543958	0.482885
2	{'xgb__n_estimators': 750, 'xgb__max_depth': 3, 'xgb__learning_rate': 0.01}			
	0.496266	0.432350	0.543958	0.482885
28	{'xgb__n_estimators': 400, 'xgb__max_depth': 3, 'xgb__learning_rate': 0.1}			


```

0.496266      0.432350      0.543958      0.482885
19 {'xgb__n_estimators': 400, 'xgb__max_depth': 3, 'xgb__learning_rate': 0.05}
0.496266      0.432350      0.543958      0.482885
10 {'xgb__n_estimators': 400, 'xgb__max_depth': 3, 'xgb__learning_rate': 0.02}
0.496266      0.432350      0.543958      0.482885
11 {'xgb__n_estimators': 750, 'xgb__max_depth': 3, 'xgb__learning_rate': 0.02}
0.496266      0.432350      0.543958      0.482885
29 {'xgb__n_estimators': 750, 'xgb__max_depth': 3, 'xgb__learning_rate': 0.1}
0.496266      0.432350      0.543958      0.482885
20 {'xgb__n_estimators': 750, 'xgb__max_depth': 3, 'xgb__learning_rate': 0.05}
0.496266      0.432350      0.543958      0.482885
1 {'xgb__n_estimators': 400, 'xgb__max_depth': 3, 'xgb__learning_rate': 0.01}
0.500381      0.415173      0.540673      0.462349
18 {'xgb__n_estimators': 100, 'xgb__max_depth': 3, 'xgb__learning_rate': 0.05}
0.501204      0.413177      0.541494      0.463589
Model: Pipeline(steps=[('xgb',
                        XGBClassifier(base_score=None, booster=None, callbacks=None,
                                      colsample_bylevel=None, colsample_bynode=None,
                                      colsample_bytree=None, device=None,
                                      early_stopping_rounds=None,
                                      enable_categorical=False, eval_metric='logloss',
                                      feature_types=None, feature_weights=None,
                                      gamma=None, grow_policy=None,
                                      importance_type=None,
                                      interaction_constraints=None, learning_rate=0.01,
                                      max_bin=None, max_cat_threshold=None,
                                      max_cat_to_onehot=None, max_delta_step=None,
                                      max_depth=4, max_leaves=None,
                                      min_child_weight=None, missing=nan,
                                      monotone_constraints=None, multi_strategy=None,
                                      n_estimators=400, n_jobs=None,
                                      num_parallel_tree=None, ...))])

```

	precision	recall	f1-score	support
0	0.61	0.44	0.51	163
1	0.32	0.48	0.38	88
accuracy			0.45	251
macro avg	0.46	0.46	0.45	251
weighted avg	0.51	0.45	0.47	251



8.3 Combined Dataset

```
[127]: # XGBoost for Combined Dataset
xgb_boost = XGBClassifier(random_state=42)

fit_model(xgb_boost, cleaned_com_train, cleaned_com_val)
```

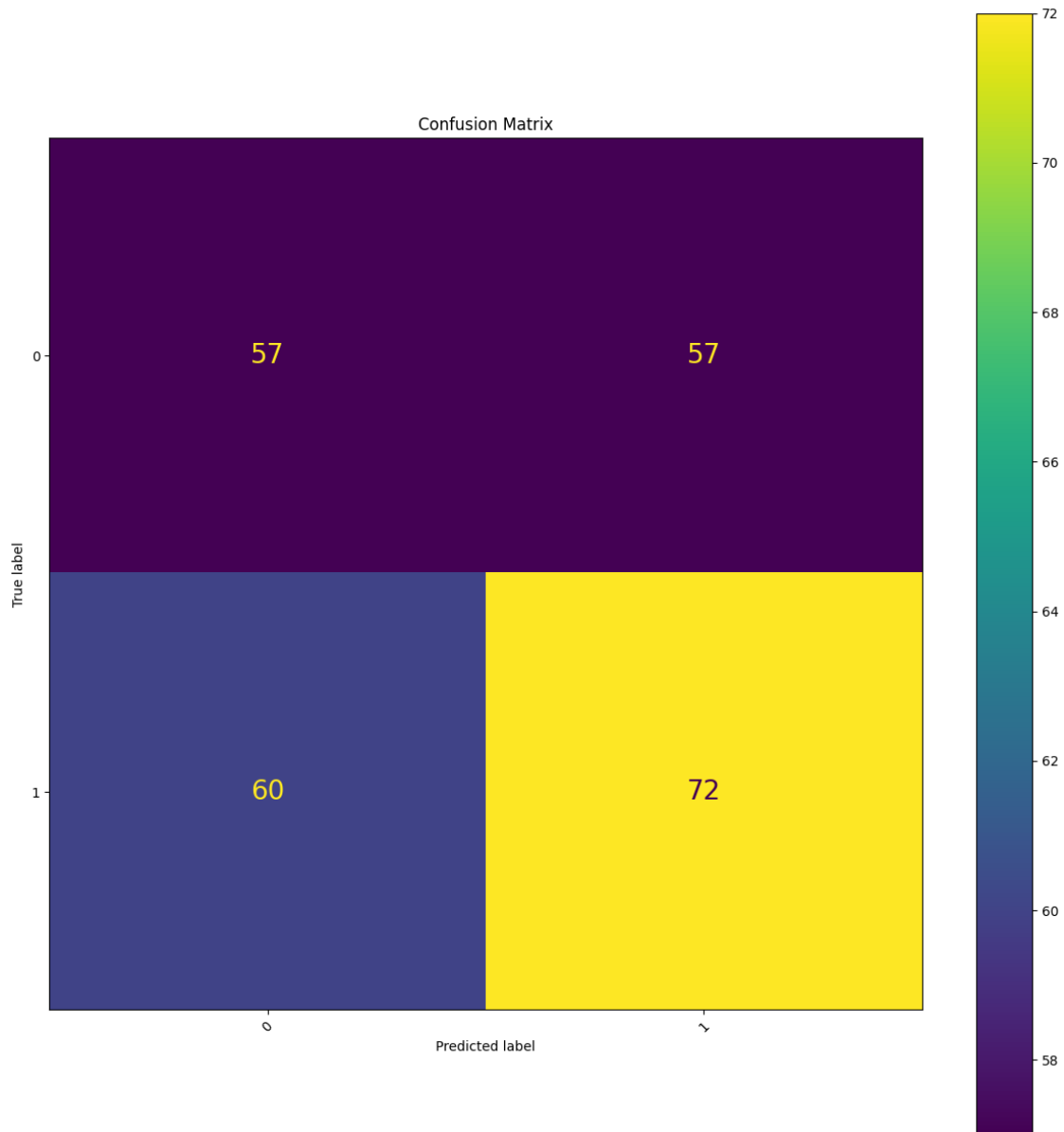
```
Model: XGBClassifier(base_score=None, booster=None, callbacks=None,
  colsample_bylevel=None, colsample_bynode=None,
  colsample_bytree=None, device=None, early_stopping_rounds=None,
  enable_categorical=False, eval_metric=None, feature_types=None,
```

```

feature_weights=None, gamma=None, grow_policy=None,
importance_type=None, interaction_constraints=None,
learning_rate=None, max_bin=None, max_cat_threshold=None,
max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
max_leaves=None, min_child_weight=None, missing=nan,
monotone_constraints=None, multi_strategy=None, n_estimators=None,
n_jobs=None, num_parallel_tree=None, ...)
precision    recall  f1-score   support

```

0	0.50	0.49	0.49	117
1	0.55	0.56	0.55	129
accuracy			0.52	246
macro avg	0.52	0.52	0.52	246
weighted avg	0.52	0.52	0.52	246



```
[128]: # Define the hyperparameter grid to search

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('xgb', XGBClassifier(eval_metric='logloss'))
])

param_grid = {
    'xgb_n_estimators': [100, 400, 750],
    'xgb_max_depth': [3, 4, 5],
```

```

    'xgb__learning_rate': [0.01, 0.02, 0.05, 0.10]
}

scoring = {
    'accuracy': make_scorer(accuracy_score),
    'f1': make_scorer(f1_score, pos_label=1)    # or pos_label=1 if encoded
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Create the GridSearchCV object
grid_search = GridSearchCV(
    estimator=pipe,
    param_grid=param_grid,
    cv = cv,                                # 5-fold cross-validation
    scoring=scoring,    # print f1 and accuracy
    refit = 'accuracy',
    return_train_score=True,
    verbose=2
)

# Fit to training data
cleaned_com_train_df = pd.DataFrame(cleaned_com_train)
grid_search.fit(cleaned_com_train_df, train_y)

# Best hyperparameters
print("Best Hyperparameters:", grid_search.best_params_)

# Print performance metrics for cross-fold validation
pd.set_option('display.max_colwidth', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 2000)
results = pd.DataFrame(grid_search.cv_results_)
summary = results[['params', 'mean_test_accuracy', 'mean_test_f1',
    → 'mean_train_accuracy', 'mean_train_f1']]
print(summary.sort_values(by='mean_test_accuracy', ascending=False))

# Best model
best_xgb = grid_search.best_estimator_

fit_model(best_xgb, cleaned_com_train, cleaned_com_val)

```

Fitting 5 folds for each of 36 candidates, totalling 180 fits
 [CV] END xgb__learning_rate=0.01, xgb__max_depth=3, xgb__n_estimators=100; total
 time= 0.1s
 [CV] END xgb__learning_rate=0.01, xgb__max_depth=3, xgb__n_estimators=100; total

[illegible]

[illegible]

[illegible]


```

time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=400; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=750; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=750; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=750; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=750; total
time= 0.1s
[CV] END xgb__learning_rate=0.1, xgb__max_depth=5, xgb__n_estimators=750; total
time= 0.1s
Best Hyperparameters: {'xgb__learning_rate': 0.01, 'xgb__max_depth': 3,
'xgb__n_estimators': 100}

```

	params			
	mean_test_accuracy	mean_test_f1	mean_train_accuracy	mean_train_f1
0	{'xgb__learning_rate': 0.01, 'xgb__max_depth': 3, 'xgb__n_estimators': 100}			
1.0	1.0	1.0	1.0	1.0
1	{'xgb__learning_rate': 0.01, 'xgb__max_depth': 3, 'xgb__n_estimators': 400}			
1.0	1.0	1.0	1.0	1.0
2	{'xgb__learning_rate': 0.01, 'xgb__max_depth': 3, 'xgb__n_estimators': 750}			
1.0	1.0	1.0	1.0	1.0
3	{'xgb__learning_rate': 0.01, 'xgb__max_depth': 4, 'xgb__n_estimators': 100}			
1.0	1.0	1.0	1.0	1.0
4	{'xgb__learning_rate': 0.01, 'xgb__max_depth': 4, 'xgb__n_estimators': 400}			
1.0	1.0	1.0	1.0	1.0
5	{'xgb__learning_rate': 0.01, 'xgb__max_depth': 4, 'xgb__n_estimators': 750}			
1.0	1.0	1.0	1.0	1.0
6	{'xgb__learning_rate': 0.01, 'xgb__max_depth': 5, 'xgb__n_estimators': 100}			
1.0	1.0	1.0	1.0	1.0
7	{'xgb__learning_rate': 0.01, 'xgb__max_depth': 5, 'xgb__n_estimators': 400}			
1.0	1.0	1.0	1.0	1.0
8	{'xgb__learning_rate': 0.01, 'xgb__max_depth': 5, 'xgb__n_estimators': 750}			
1.0	1.0	1.0	1.0	1.0
9	{'xgb__learning_rate': 0.02, 'xgb__max_depth': 3, 'xgb__n_estimators': 100}			
1.0	1.0	1.0	1.0	1.0
10	{'xgb__learning_rate': 0.02, 'xgb__max_depth': 3, 'xgb__n_estimators': 400}			
1.0	1.0	1.0	1.0	1.0
11	{'xgb__learning_rate': 0.02, 'xgb__max_depth': 3, 'xgb__n_estimators': 750}			

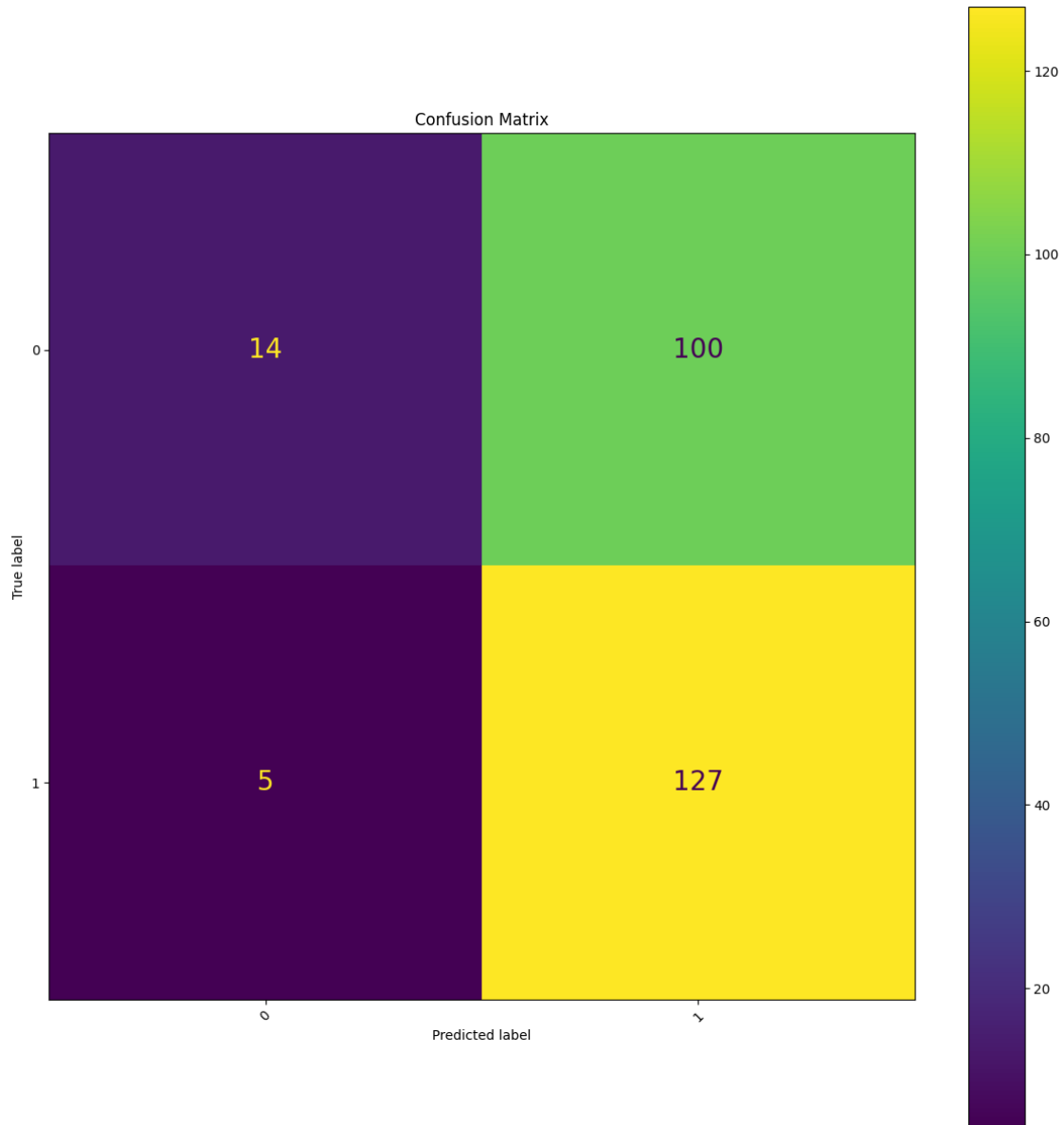
1.0	1.0	1.0	1.0
12	{'xgb__learning_rate': 0.02, 'xgb__max_depth': 4, 'xgb__n_estimators': 100}		
1.0	1.0	1.0	1.0
13	{'xgb__learning_rate': 0.02, 'xgb__max_depth': 4, 'xgb__n_estimators': 400}		
1.0	1.0	1.0	1.0
14	{'xgb__learning_rate': 0.02, 'xgb__max_depth': 4, 'xgb__n_estimators': 750}		
1.0	1.0	1.0	1.0
15	{'xgb__learning_rate': 0.02, 'xgb__max_depth': 5, 'xgb__n_estimators': 100}		
1.0	1.0	1.0	1.0
16	{'xgb__learning_rate': 0.02, 'xgb__max_depth': 5, 'xgb__n_estimators': 400}		
1.0	1.0	1.0	1.0
17	{'xgb__learning_rate': 0.02, 'xgb__max_depth': 5, 'xgb__n_estimators': 750}		
1.0	1.0	1.0	1.0
18	{'xgb__learning_rate': 0.05, 'xgb__max_depth': 3, 'xgb__n_estimators': 100}		
1.0	1.0	1.0	1.0
19	{'xgb__learning_rate': 0.05, 'xgb__max_depth': 3, 'xgb__n_estimators': 400}		
1.0	1.0	1.0	1.0
20	{'xgb__learning_rate': 0.05, 'xgb__max_depth': 3, 'xgb__n_estimators': 750}		
1.0	1.0	1.0	1.0
21	{'xgb__learning_rate': 0.05, 'xgb__max_depth': 4, 'xgb__n_estimators': 100}		
1.0	1.0	1.0	1.0
22	{'xgb__learning_rate': 0.05, 'xgb__max_depth': 4, 'xgb__n_estimators': 400}		
1.0	1.0	1.0	1.0
23	{'xgb__learning_rate': 0.05, 'xgb__max_depth': 4, 'xgb__n_estimators': 750}		
1.0	1.0	1.0	1.0
24	{'xgb__learning_rate': 0.05, 'xgb__max_depth': 5, 'xgb__n_estimators': 100}		
1.0	1.0	1.0	1.0
25	{'xgb__learning_rate': 0.05, 'xgb__max_depth': 5, 'xgb__n_estimators': 400}		
1.0	1.0	1.0	1.0
26	{'xgb__learning_rate': 0.05, 'xgb__max_depth': 5, 'xgb__n_estimators': 750}		
1.0	1.0	1.0	1.0
27	{'xgb__learning_rate': 0.1, 'xgb__max_depth': 3, 'xgb__n_estimators': 100}		
1.0	1.0	1.0	1.0
28	{'xgb__learning_rate': 0.1, 'xgb__max_depth': 3, 'xgb__n_estimators': 400}		
1.0	1.0	1.0	1.0
29	{'xgb__learning_rate': 0.1, 'xgb__max_depth': 3, 'xgb__n_estimators': 750}		
1.0	1.0	1.0	1.0
30	{'xgb__learning_rate': 0.1, 'xgb__max_depth': 4, 'xgb__n_estimators': 100}		
1.0	1.0	1.0	1.0
31	{'xgb__learning_rate': 0.1, 'xgb__max_depth': 4, 'xgb__n_estimators': 400}		
1.0	1.0	1.0	1.0
32	{'xgb__learning_rate': 0.1, 'xgb__max_depth': 4, 'xgb__n_estimators': 750}		
1.0	1.0	1.0	1.0
33	{'xgb__learning_rate': 0.1, 'xgb__max_depth': 5, 'xgb__n_estimators': 100}		
1.0	1.0	1.0	1.0
34	{'xgb__learning_rate': 0.1, 'xgb__max_depth': 5, 'xgb__n_estimators': 400}		
1.0	1.0	1.0	1.0
35	{'xgb__learning_rate': 0.1, 'xgb__max_depth': 5, 'xgb__n_estimators': 750}		

```

1.0          1.0          1.0          1.0
Model: Pipeline(steps=[('scaler', StandardScaler()),
                        ('xgb',
                         XGBClassifier(base_score=None, booster=None, callbacks=None,
                                       colsample_bylevel=None, colsample_bynode=None,
                                       colsample_bytree=None, device=None,
                                       early_stopping_rounds=None,
                                       enable_categorical=False, eval_metric='logloss',
                                       feature_types=None, feature_weights=None,
                                       gamma=None, grow_policy=None,
                                       importance_type=None,
                                       interaction_constraints=None, learning_rate=0.01,
                                       max_bin=None, max_cat_threshold=None,
                                       max_cat_to_onehot=None, max_delta_step=None,
                                       max_depth=3, max_leaves=None,
                                       min_child_weight=None, missing=nan,
                                       monotone_constraints=None, multi_strategy=None,
                                       n_estimators=100, n_jobs=None,
                                       num_parallel_tree=None, ...))])

```

	precision	recall	f1-score	support
0	0.12	0.74	0.21	19
1	0.96	0.56	0.71	227
accuracy			0.57	246
macro avg	0.54	0.65	0.46	246
weighted avg	0.90	0.57	0.67	246



9 Week 9 - Picking the Best Model

This section will loop through all the models of the past 3 weeks, and compare validation accuracy as well as F1 score. The accuracy will be compiled into a bar chart with error on the y-axis and model name on the x-axis. Finally, the model with the superior validation performance will be used to generate predictions on the test dataset, and performance will be evaluated.

```
[129]: #make sure we have training and validation datasets available on hand
ts_train_X = cleaned_ts_train.drop(columns = ['Stock Price Direction', 'Dates'])
ts_train_y = cleaned_ts_train['Stock Price Direction']
```



```

ts_val_X = cleaned_ts_val.drop(columns = ['Stock Price Direction', 'Dates'])
ts_val_y = cleaned_ts_val['Stock Price Direction']

com_train_X = cleaned_com_train.drop(columns = ['Stock Price Direction'])
com_train_y = cleaned_com_train['Stock Price Direction']
com_val_X = cleaned_com_val.drop(columns = ['Stock Price Direction'])
com_val_y = cleaned_com_val['Stock Price Direction']
com_test_X = cleaned_com_test.drop(columns = ['Stock Price Direction'])
com_test_y = cleaned_com_test['Stock Price Direction']

#to store errors for charts later
train_errors = []
val_errors = []
train_f1s = []
val_f1s = []

def get_error(model, train_X, train_y, val_X, val_y):
    '''Returns the errors of a model given training and validation sets'''

    # predictions
    train_pred = model.predict(train_X)
    val_pred = model.predict(val_X)

    # errors
    train_error = 1 - accuracy_score(train_y, train_pred)
    val_error = 1 - accuracy_score(val_y, val_pred)

    # Macro F1
    train_f1 = f1_score(train_y, train_pred, average='macro')
    val_f1 = f1_score(val_y, val_pred, average='macro')

    train_errors.append(train_error)
    val_errors.append(val_error)
    train_f1s.append(train_f1)
    val_f1s.append(val_f1)

    return train_errors, val_errors, train_f1s, val_f1s

# Price Logistic Regression:
get_error(price_lr_tuned, ts_train_X, ts_train_y, ts_val_X, ts_val_y)

#Sentiment Logistic Regression:
get_error(best_lr_sentiment, X_train_combined, y_train_sentiment, X_val_combined,
    ↪y_val_sentiment)

#Combined Dataset Logistic Regression
get_error(lr2, com_train_X, com_train_y, com_val_X, com_val_y)

```

```

#Price Support Vector Machine
get_error(price_svm_tuned, ts_train_X, ts_train_y, ts_val_X, ts_val_y)

#Sentiment Support Vector Machine
get_error(best_sentiment_svm, X_train_combined, y_train_sentiment,
↪X_val_combined, y_val_sentiment)

#Combined Support Vector Machine
get_error(best_svm, com_train_X, com_train_y, com_val_X, com_val_y)

#Price XGBoost
get_error(price_xgb_tuned, ts_train_X, ts_train_y, ts_val_X, ts_val_y)

#Combined XGBoost
get_error(best_xgb, com_train_X, com_train_y, com_val_X, com_val_y)

#Sentiment FINBERT
get_error(best_lr, X_train_finbert_lr, y_train_finbert_lr, X_val_finbert_lr,
↪y_val_finbert_lr)

```

```

[129]: ([0.485620377978636,
        0.47165160230073955,
        0.4678217821782178,
        0.4847986852917009,
        0.47165160230073955,
        0.4521452145214522,
        0.456861133935908,
        0.39191419141914197,
        0.4552177485620378],
        [0.4581673306772909,
        0.4701195219123506,
        0.4878048780487805,
        0.5258964143426295,
        0.4701195219123506,
        0.44715447154471544,
        0.545816733067729,
        0.426829268292683,
        0.4581673306772909],
        [0.43010304540574373,
        0.3456989247311828,
        0.4740255714542665,
        0.5152009873822487,
        0.3456989247311828,
        0.44094999848490457,
        0.5367427976777304,
        0.5225477390060335,

```

```

0.41743567942709414],
[0.4512044413179459,
0.34635416666666667,
0.4576719576719577,
0.4740284480568961,
0.34635416666666667,
0.43592095388976904,
0.4462730068758957,
0.4590236035771881,
0.4594265595445437])

```

```

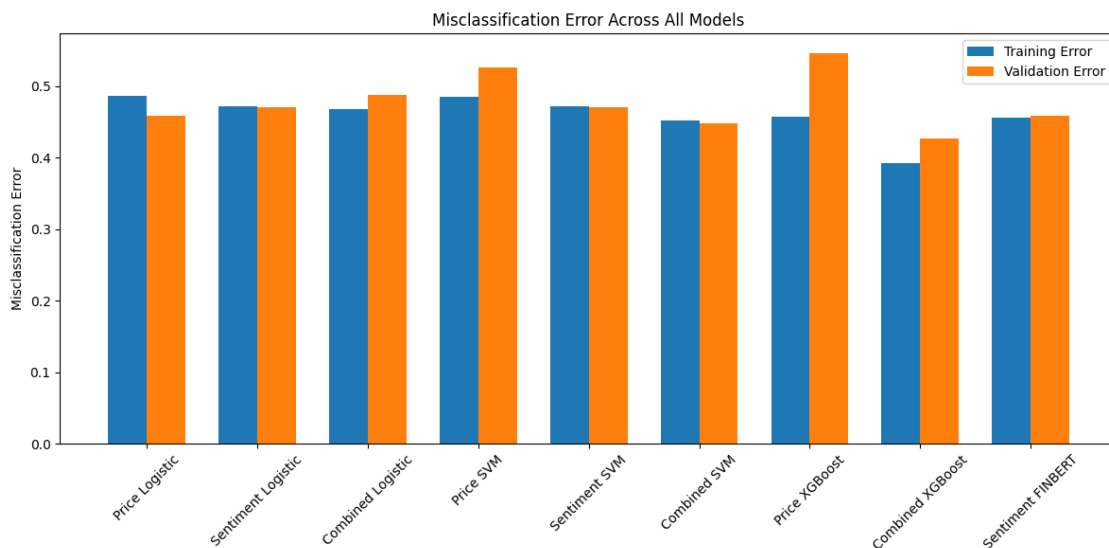
[130]: #create chart with error on y-axis, models on x-axis
model_names = [
    "Price Logistic", "Sentiment Logistic", "Combined Logistic", "Price SVM",
    "Sentiment SVM", "Combined SVM", "Price XGBoost", "Combined XGBoost"
    , "Sentiment FINBERT"
]

x = np.arange(len(model_names))
width = 0.35

plt.figure(figsize=(12, 6))
plt.bar(x - width/2, train_errors, width, label='Training Error')
plt.bar(x + width/2, val_errors, width, label='Validation Error')

plt.xticks(x, model_names, rotation=45)
plt.ylabel("Misclassification Error")
plt.title("Misclassification Error Across All Models")
plt.legend()
plt.tight_layout()
plt.show()

```



```
[131]: def test_metrics(model, val_X, val_y, print_matrix = True):

    #Generate predictions and produce accuracy metrics
    my_pred = model.predict(val_X)
    print(f'Model: {model}')
    print(CR(my_pred, val_y))

    #generate confusion matrix
    cm = confusion_matrix(val_y, my_pred, labels=model.classes_)

    #print confusion matrix
    if print_matrix:
        disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=model.
→classes_)

        fig, ax = plt.subplots(figsize=(12, 12)) # Adjust size for readability
        disp.plot(ax=ax, cmap='viridis', xticks_rotation=45)

        for row in disp.text_:
            for text in row:
                text.set_fontsize(20)

        plt.title("Confusion Matrix")
        plt.tight_layout()
        plt.show()

test_metrics(best_xgb, com_test_X, com_test_y)
```

```
Model: Pipeline(steps=[('scaler', StandardScaler()),
                        ('xgb',
                         XGBClassifier(base_score=None, booster=None, callbacks=None,
                                         colsample_bylevel=None, colsample_bynode=None,
                                         colsample_bytree=None, device=None,
                                         early_stopping_rounds=None,
                                         enable_categorical=False, eval_metric='logloss',
                                         feature_types=None, feature_weights=None,
                                         gamma=None, grow_policy=None,
                                         importance_type=None,
                                         interaction_constraints=None, learning_rate=0.01,
                                         max_bin=None, max_cat_threshold=None,
                                         max_cat_to_onehot=None, max_delta_step=None,
                                         max_depth=3, max_leaves=None,
                                         min_child_weight=None, missing=nan,
                                         monotone_constraints=None, multi_strategy=None,
```

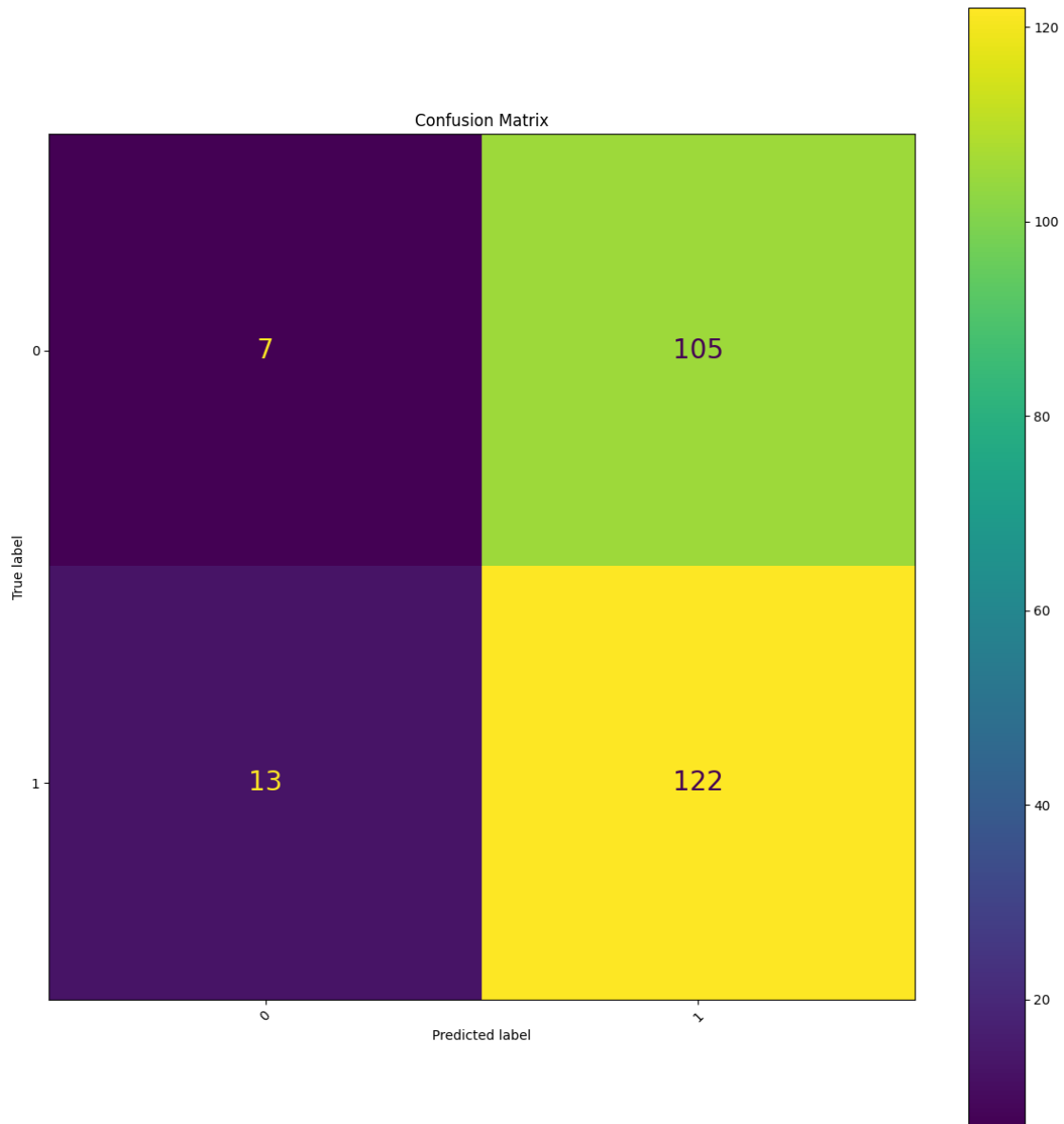
```

n_estimators=100, n_jobs=None,
num_parallel_tree=None, ...)))
precision    recall  f1-score   support

0           0.06      0.35      0.11         20
1           0.90      0.54      0.67        227

accuracy          0.52         247
macro avg          0.48      0.44      0.39         247
weighted avg          0.84      0.52      0.63         247

```



10 Week 10 - Data Centric AI

```
[132]: # Final data cleaning function for the data-centric approach, this will be used
        ↳ to create the X and y datasets for the price, sentiment, and combined models.
        # This function will also create lagged variables for the sentiment polarity,
        ↳ which will be used in the models.
def data_centric_cleaning(sentiment_data, ts_data):
    # Create a copy to avoid modifying the original DataFrame
    sentiment_data = sentiment_data.copy()

    # Drop 'Stock Price Direction' from sentiment_data if it exists, since we
    ↳ will be merging with ts_data which has the target variable
    if 'Stock Price Direction' in sentiment_data.columns:
        sentiment_data.drop(columns = 'Stock Price Direction', inplace = True)

    #join on date
    df = pd.merge(sentiment_data, ts_data, how = 'right', left_on = 'Dates',
    ↳ right_on = 'Dates')

    # Drop 'tokens' column if present (contains lists which are unhashable)
    if 'tokens' in df.columns:
        df = df.drop(columns=['tokens'])

    # Sort by date to ensure correct order for lag creation
    df = df.sort_values('Dates').reset_index(drop=True)

    for i in range(1, 6):
        #create lags for both price and sentiment polarity variables
        df[f'sentiment_polarity_lag{i}'] = df['sentiment_polarity'].shift(i)
        df[f'sentiment_pos_lag{i}'] = df['sentiment_pos'].shift(i)
        df[f'sentiment_neu_lag{i}'] = df['sentiment_neu'].shift(i)
        df[f'sentiment_neg_lag{i}'] = df['sentiment_neg'].shift(i)

    #Create lags for the AAPL Share Price
    for i in range(1, 6):
        df[f'aapl_price_direction_lag{i}'] = df['AAPL Share Price'].shift(i)

    #drop rows that are missing lagged variables
    df.dropna(inplace = True)

    #drop duplicates, but there shouldn't be any because of the primary key date
    df.drop_duplicates(inplace = True)

    #pick the final columns for our dataset
```

```

    final_df = df[['Stock Price Direction', 'AAPL Share Price',
    ↪ 'sentiment_polarity', 'sentiment_polarity_lag1', 'sentiment_polarity_lag2',
    ↪ 'sentiment_polarity_lag3', 'sentiment_polarity_lag4',
    ↪ 'sentiment_polarity_lag5', 'sentiment_pos_lag1', 'sentiment_pos_lag2',
    ↪ 'sentiment_pos_lag3', 'sentiment_pos_lag4', 'sentiment_pos_lag5',
    ↪ 'sentiment_neu_lag1', 'sentiment_neu_lag2', 'sentiment_neu_lag3',
    ↪ 'sentiment_neu_lag4', 'sentiment_neu_lag5', 'sentiment_neg_lag1',
    ↪ 'sentiment_neg_lag2', 'sentiment_neg_lag3', 'sentiment_neg_lag4',
    ↪ 'sentiment_neg_lag5', 'aapl_price_direction_lag1',
    ↪ 'aapl_price_direction_lag2', 'aapl_price_direction_lag3',
    ↪ 'aapl_price_direction_lag4', 'aapl_price_direction_lag5']]

    X_data = final_df.drop(columns = ['Stock Price Direction'])
    y_data = final_df['Stock Price Direction']

    return X_data, y_data

```

```

[133]: X_train_centric, y_train_centric = data_centric_cleaning(Final_Raw_X_train,
    ↪ cleaned_ts_train)

```

```

[134]: #Apply SMOTE, this weights minority class heavier
X_train_resampled, y_train_resampled = SMOTE().fit_resample(X_train_centric,
    ↪ y_train_centric)

#Validation creation
X_val_centric, y_val_centric = data_centric_cleaning(Final_Raw_X_val,
    ↪ cleaned_ts_val)

#Test creation
X_test_centric, y_test_centric = data_centric_cleaning(Final_Raw_X_test,
    ↪ cleaned_ts_test)

```

```

[140]: param_grid = {
    'xgb__n_estimators': [100, 400, 750],
    'xgb__max_depth': [3, 4, 5],
    'xgb__learning_rate': [0.01, 0.02, 0.05, 0.10]
}

grid_search = GridSearchCV(XGBClassifier(random_state=42), param_grid,
    ↪ scoring='accuracy', cv=3, n_jobs=1)
grid_search.fit(X_train_resampled, y_train_resampled)
best_params = grid_search.best_params_
print(f"Best Parameters: {best_params}")

```

```

Best Parameters: {'xgb__learning_rate': 0.01, 'xgb__max_depth': 3,
'xgb__n_estimators': 100}

```

```
[141]: # XGBoost for Combined Dataset with bootstrapping
bootstrap_models = []

for i in range(100):
    X_boot, y_boot = resample(X_train_resampled, y_train_resampled,
    ↪replace=True, random_state=i)
    xgb_boot = XGBClassifier(**best_params, random_state=42)
    xgb_boot.fit(X_boot, y_boot)
    bootstrap_models.append(xgb_boot)

# Aggregate predictions on validation set
val_probab = np.array([m.predict_proba(X_val_centric)[: , 1] for m in
    ↪bootstrap_models]).mean(axis=0)

val_preds = (val_probab >= 0.5).astype(int)
```

```
[144]: # Validation evaluation
print(f"Model: XGBClassifier (bootstrapped, n=100)")
print(CR(val_preds, y_val_centric))

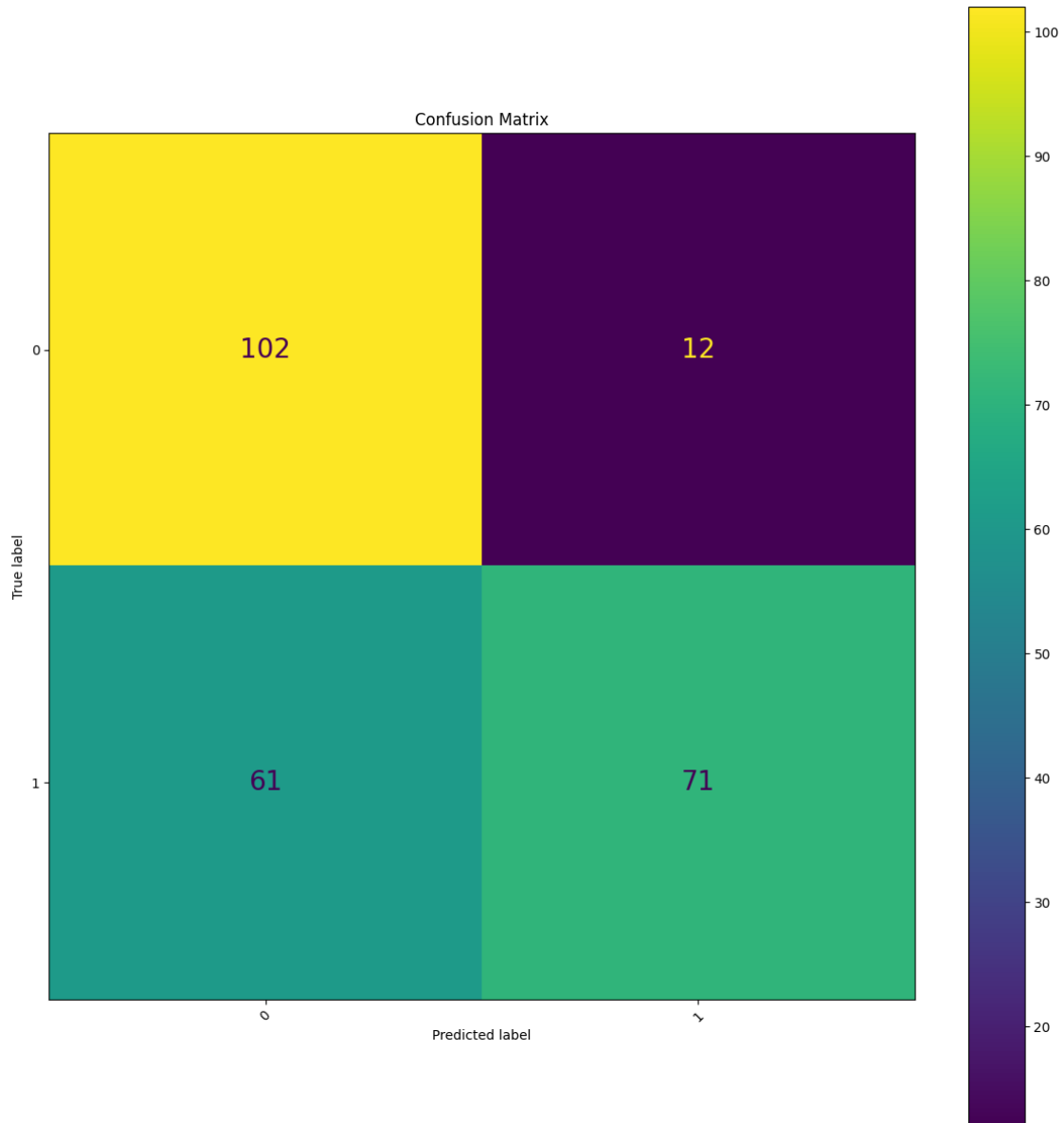
cm = confusion_matrix(y_val_centric, val_preds, labels=[0, 1])
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[0, 1])
fig, ax = plt.subplots(figsize=(12, 12))
disp.plot(ax=ax, cmap="viridis", xticks_rotation=45)

for row in disp.text_:
    for text in row:
        text.set_fontsize(20)

plt.title("Confusion Matrix")
plt.tight_layout()
plt.show()
```

```
Model: XGBClassifier (bootstrapped, n=100)
```

	precision	recall	f1-score	support
0	0.89	0.63	0.74	163
1	0.54	0.86	0.66	83
accuracy			0.70	246
macro avg	0.72	0.74	0.70	246
weighted avg	0.77	0.70	0.71	246



```
[145]: # Test evaluation
test_probab = np.array([m.predict_proba(X_test_centric)[: , 1] for m in
↳ bootstrap_models]).mean(axis=0)
test_preds = (test_probab >= 0.5).astype(int)

print(f"Model: XGBClassifier (bootstrapped, n=100)")
print(CR(test_preds, y_test_centric))

cm = confusion_matrix(y_test_centric, test_preds, labels=[0, 1])
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[0, 1])
```

```

fig, ax = plt.subplots(figsize=(12, 12))
disp.plot(ax=ax, cmap='viridis', xticks_rotation=45)

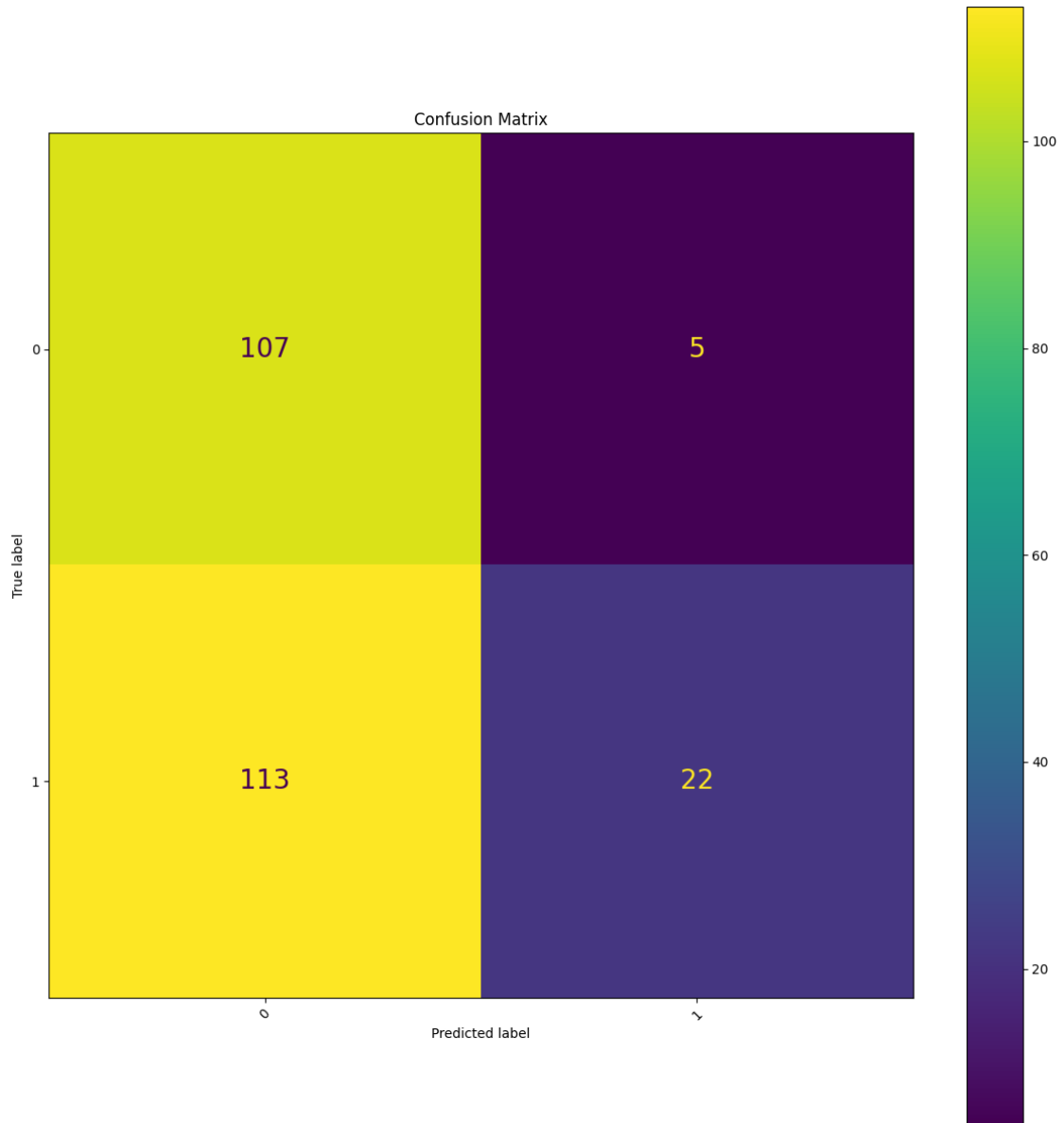
for row in disp.text_:
    for text in row:
        text.set_fontsize(20)

plt.title("Confusion Matrix")
plt.tight_layout()
plt.show()

```

Model: XGBClassifier (bootstrapped, n=100)

	precision	recall	f1-score	support
0	0.96	0.49	0.64	220
1	0.16	0.81	0.27	27
accuracy			0.52	247
macro avg	0.56	0.65	0.46	247
weighted avg	0.87	0.52	0.60	247



[]: